

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ  
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

=====

Дата генерации: 11.09.2026, 20:30:37  
Файлов исходного кода: 87  
Суммарный объём кода: 1.20 MB  
Глав/билетов в пособии: 30

=====

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

=====

1. 1. Дерево отрезков с операциями на отрезках [id: tree-seg]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-dfs]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

24. src/data/vizStepBus.ts  
25. src/data/vizSync.ts

# Билеты (учебный контент)

26. src/data/tickets/ticket1\_5.ts  
27. src/data/tickets/ticket14\_20.ts  
28. src/data/tickets/ticket21\_24.ts  
29. src/data/tickets/ticket6\_13.ts

# Интерактивные компоненты и симуляторы

30. src/components/ArticulationPointsViz.tsx  
31. src/components/BellmanFordViz.tsx  
32. src/components/BfsViz.tsx  
33. src/components/ChapterImage.tsx  
34. src/components/ChapterNav.tsx  
35. src/components/ChapterView.tsx  
36. src/components/DfsBridgesSimulator.tsx  
37. src/components/DijkstraViz.tsx  
38. src/components/DPVisualizer.tsx  
39. src/components/EulerSimulator.tsx  
40. src/components/ExpressQuiz.tsx  
41. src/components/FloydViz.tsx  
42. src/components/GraphTraversalViz.tsx  
43. src/components/HeapViz.tsx  
44. src/components/hints/demoKit.tsx  
45. src/components/hints/demosExtra.tsx  
46. src/components/hints/demosGraph.tsx  
47. src/components/hints/demosStruct.tsx  
48. src/components/hints/MiniDemo.tsx  
49. src/components/hints/SimulatorModal.tsx  
50. src/components/hints/TermPopover.tsx  
51. src/components/JohnsonViz.tsx  
52. src/components/KosarajuViz.tsx  
53. src/components/KruskalSimulator.tsx  
54. src/components/MnemonicCards.tsx  
55. src/components/MobileToc.tsx  
56. src/components/Navbar.tsx  
57. src/components/PlanarityDemo.tsx

```
86. .github/workflows/deploy-pages.yml
87. .github/workflows/rebuild-pdf.yml
```

---

## РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

---

ФАЙЛ 1/87: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

## Изменённые файлы

```text

```
A    .github/workflows/deploy-pages.yml
M    .github/workflows/rebuild-pdf.yml
M    index.html
M    public/export/full_code_all_pages.pdf
M    public/export/full_code_all_pages.txt
A    public/favicon.svg
M    src/App.tsx
M    src/components/Navbar.tsx
A    src/components/PythonCompiler.tsx
M    vite.config.ts
```
```

## `github/workflows/deploy-pages.yml`

### Полное содержимое после изменений

```yaml

```
name: Deploy app to GitHub Pages
```

```
on:
```

```
  push:
```

```
    branches:
```

```
- name: Upload Pages artifact
  uses: actions/upload-pages-artifact@v5
  with:
    path: ./dist

deploy:
  name: Publish to GitHub Pages
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${ steps.deployment.outputs.page_url }
  steps:
    - name: Deploy
      id: deployment
      uses: actions/deploy-pages@v5
....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
```



```
sudo apt-get update -qq
sudo apt-get install -y --no-install-recommen
# На Ubuntu политика ImageMagick иногда запре
# разрешаем то, что нужно пайплайну (text ->
for policy in /etc/ImageMagick-6/policy.xml /
    if [ -f "$policy" ]; then
        sudo sed -i 's/rights="none" pattern="\{P
    fi
done
convert -version
```

- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
EXPORT\_DENSITY: \${github.event.inputs.densi  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`public/export/full\_code\_all\_pages  
 echo "| \`public/export/full\_code\_all\_pages

index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DejaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB\_STEP\_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

- name: Upload artifacts (PNG-страницы)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

Универсальное пособие • полный исходный код

```
-----  
    <body class="bg-slate-950 text-slate-100 min-h-screen  
....
```

```
## `public/favicon.svg`
```

```
### Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64  
  <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#  
  <circle cx="47" cy="21" r="4" fill="#34d399"/>  
</svg>  
....
```

```
### Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg  
new file mode 100644  
index 00000000..3bbbba0  
--- /dev/null  
+++ b/public/favicon.svg  
@@ -0,0 +1,5 @@  
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64  
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#  
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>  
+</svg>  
....
```

```
## `src/App.tsx`
```

```
### Полное содержимое после изменений
```

```
````tsx
```

```
import { useCallback, useEffect, useMemo, useState } from  
import { chapters } from "../data/content";
```



```

    }, [goTo, next, prev]));

const progress = useMemo(() => ((index + 1) / chapters.length) * 100);

return (
  <div className="min-h-screen bg-slate-950 text-slate-50">
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
      {activeTab === "guide" ? (
        <>
          <MobileToc
            selectedChapterId={activeChapterId}
            setSelectedChapterId={goTo}
            currentTitle={activeChapter.title}
            index={index}
            total={chapters.length}
          />

          <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
            {/* Сайдбар только на десктопе – на мобильном скрывается */}
            <div className="hidden lg:block lg:w-80 shrink-0">
              <Sidebar selectedChapterId={activeChapterId}>
            </div>

            <main id="main-content" className="flex-1 min-w-0">
              {/* Шапка главы с быстрыми переходами */}
              <div className="flex items-center justify-between">
                <div className="min-w-0">
                  <div className="text-[10px] uppercase">
                    {activeChapter.category || "Универсальное пособие"}
                  </div>
                  <h2 className="text-base sm:text-xl font-medium">
                    {activeChapter.title}
                  </h2>
                </div>
                <ChapterNav prev={prev} next={next} index={index}>
              </div>
            </main>
          </div>
        </>
      ) : (
        <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
          <div className="hidden lg:block lg:w-80 shrink-0">
            <Sidebar selectedChapterId={activeChapterId}>
          </div>

          <main id="main-content" className="flex-1 min-w-0">
            {/* Шапка главы с быстрыми переходами */}
            <div className="flex items-center justify-between">
              <div className="min-w-0">
                <div className="text-[10px] uppercase">
                  {activeChapter.category || "Универсальное пособие"}
                </div>
                <h2 className="text-base sm:text-xl font-medium">
                  {activeChapter.title}
                </h2>
              </div>
              <ChapterNav prev={prev} next={next} index={index}>
            </div>
          </main>
        </div>
      )}
    </Navbar>
  </div>
);

```

```

        </footer>
    </div>
  );
}
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
  const [activeChapterId, setActiveChapterId] = useState<string>("");
  const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
    </main>
    </div>
  </>
-  ) : (
+  ) : activeTab === "simulator" ? (
    <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
      <SimulatorHub
        onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
      </>
    </main>
  ) : (

```

```

<div className="flex items-center gap-3 w-full" >
  <div className="bg-gradient-to-tr from-indigo-400 to-indigo-600 w-6 h-6" >
    <Sparkles className="w-6 h-6" />
  </div>
  <div className="min-w-0 flex-1" >
    <h1 className="text-xl font-extrabold text-slate-900" >
      Универсальное пособие
    </h1>
    <p className="text-xs text-indigo-400 font-medium" >
      Подготовка к экзаменам: Алгоритмы, Структуры данных
    </p>
  </div>
</div>

```

```

<div className="flex items-center w-full lg:w-auto" >
  >
  <button
    id="tab-guide-btn"
    onClick={() => setActiveTab('guide')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
      activeTab === 'guide'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
        : 'text-slate-400 hover:text-white'
    }`}
  >
    <BookOpen className="w-4 h-4" /> Учебное пособие
  </button>
  <button
    id="tab-simulator-btn"
    onClick={() => setActiveTab('simulator')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
      activeTab === 'simulator'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
        : 'text-slate-400 hover:text-white'
    }`}
  >
    <CodeIcon className="w-4 h-4" /> Симулятор
  </button>
</div>

```

```

        </a>
        <button
          id="download-html-btn"
          onClick={saveBook}
          disabled={busy}
          className="flex items-center justify-center
            er:bg-amber-600/20 border border-amber-500/
          title="Скачать всё пособие одним автономным
        >
          {busy ? <Loader2 className="w-4 h-4 animate
        </button>
      </div>
    </div>
  </header>
);
};
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

```

### Полное содержимое после изменений

```
````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в интерпретатор
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
      .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL }));
  }
  return runtimePromise;
}
```

```

        const captured = [...stdout, ...stderr].join("");
        setOutput(`${captured}${captured && !captured.end`
        setRuntimeMessage("Не удалось выполнить код – про
    } finally {
        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
        <div className="max-w-6xl mx-auto">
            <div className="flex flex-col sm:flex-row sm:items-center">
                <div>
                    <div className="inline-flex items-center gap-2">
                        <Terminal className="w-4 h-4" /> Боковая панель
                    </div>
                    <h2 className="text-2xl sm:text-3xl font-extrabold">
                        <p className="text-slate-400 mt-2 max-w-2xl">
                            Пишите небольшой код и запускайте его прямо в браузере,
                            который переводит CPython в WebAssembly.
                        </p>
                    </div>
                    <a
                        href="https://pyodide.org/"
                        target="_blank"
                        rel="noreferrer"
                        className="inline-flex items-center gap-1.5"
                    >
                        Как это работает <ExternalLink className="w-4 h-4" />
                    </a>
                </div>
            </div>
        </div>
    </main>

```

```

    }}
    spellCheck={false}
    aria-label="Код Python"
    className="block w-full min-h-[360px] res
    -none focus:ring-2 focus:ring-inset focus:r
    placeholder="Напишите Python-код..."
  />

  <div className="border-t border-slate-800 b
    <label className="block px-4 pt-3 text-[1
      Ввод для input() • по одной строке на к
    </label>
    <textarea
      id="python-stdin"
      value={stdin}
      onChange={(event) => setStdin(event.tar
      spellCheck={false}
      rows={2}
      placeholder={'Например:\nАлиса'}
      className="block w-full resize-y bg-tra
    eholder:text-slate-700"
    />
  </div>
</section>

<section className="rounded-2xl border border
  <div className="flex items-center justify-b
    <div className="flex items-center gap-2 t
      <Terminal className="w-4 h-4 text-emera
    </div>
    <span className={`inline-flex items-cente
      {runtimeReady && <CheckCircle2 classNam
      {runtimeReady ? "готов" : "не загружен"
    </span>
  </div>
  <pre className="min-h-[360px] max-h-[560px]
  x] leading-6 text-slate-300">
    {output}
  
```

Универсальное пособие • полный исходный код

```
-----  
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/pyodide.js`  
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/index.html`  
+  
+const STARTER_CODE = `# Первый запуск загрузит Python и выполнит код  
+name = "алгоритмы"  
+  
+for number in range(1, 4):  
+    print(f"{number}. Учим {name}!")  
+`;  
+  
+type PyodideRuntime = {  
+  runPythonAsync: (source: string) => Promise<unknown>  
+  setStdout: (options: { batched: (message: string) => void }) => void  
+  setStderr: (options: { batched: (message: string) => void }) => void  
+  setStdin: (options: { stdin: () => string | number | null }) => void  
+};  
+  
+type PyodideModule = {  
+  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>  
+};  
+  
+let runtimePromise: Promise<PyodideRuntime> | null = null;  
+  
+function getPythonRuntime() {  
+  if (!runtimePromise) {  
+    runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL).then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));  
+  }  
+  return runtimePromise;  
+}  
+  
+function errorText(error: unknown) {  
+  if (error instanceof Error) return error.message;  
+  return String(error);  
+}  
+  
+export function PythonCompiler() {
```



```

+
+  const reset = () => {
+    setCode(STARTER_CODE);
+    setStdin("");
+    setOutput("Нажмите «Запустить», чтобы увидеть резу.
+  };
+
+  return (
+    <main className="max-w-screen-2xl mx-auto px-4 sm:p
+      <div className="max-w-6xl mx-auto">
+        <div className="flex flex-col sm:flex-row sm:i
+          <div>
+            <div className="inline-flex items-center g
+              <Terminal className="w-4 h-4" /> Боковая
+            </div>
+            <h2 className="text-2xl sm:text-3xl font-e
+            <p className="text-slate-400 mt-2 max-w-2x
+              Пишите небольшой код и запускайте его пр
+              который переводит CPython в WebAssembly.
+            </p>
+          </div>
+          <a
+            href="https://pyodide.org/"
+            target="_blank"
+            rel="noreferrer"
+            className="inline-flex items-center gap-1.5
+          >
+            Как это работает <ExternalLink className="
+          </a>
+        </div>
+
+        <div className="grid xl:grid-cols-[minmax(0,1.
+          <section className="rounded-2xl border borde
+            <div className="flex flex-wrap items-cente
+              <div className="flex items-center gap-2"
+                <span className="w-2.5 h-2.5 rounded-f
+                <span className="text-sm font-bold tex
+                <span className="text-[11px] text-slate

```

```

+
+         <div className="border-t border-slate-800 p-4">
+             <label className="block px-4 pt-3 text-slate-600">
+                 Ввод для input() • по одной строке на строку
+             </label>
+             <textarea
+                 id="python-stdin"
+                 value={stdin}
+                 onChange={(event) => setStdin(event.target.value)}
+                 spellCheck={false}
+                 rows={2}
+                 placeholder={'Например:\nАлиса'}
+                 className="block w-full resize-y bg-tran
+                 placeholder:text-slate-700"
+             />
+         </div>
+     </section>
+
+     <section className="rounded-2xl border border-slate-800 p-4">
+         <div className="flex items-center justify-between">
+             <div className="flex items-center gap-2">
+                 <Terminal className="w-4 h-4 text-emerald-500">
+                     {stdin}
+                 </div>
+                 <span className={`inline-flex items-center gap-2`}
+                     {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500"/>
+                     {runtimeReady ? "готов" : "не загружен"}}
+                 </span>
+             </div>
+             <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+                 {output}
+             </pre>
+             <div className="border-t border-slate-800 p-4">
+                 </div>
+         </section>
+     </div>
+
+     <div className="mt-5 grid md:grid-cols-3 gap-3">
+         <div className="flex gap-2 rounded-xl border

```

```
plugins: [react(), tailwindcss(), viteSingleFile()],
resolve: {
  alias: {
    "@": path.resolve(__dirname, "src"),
  },
},
server: {
  host: "0.0.0.0",
  port: 5173,
  strictPort: true,
  // предпросмотр может открываться через внешний прокси
  allowedHosts: true,
},
preview: {
  host: "0.0.0.0",
  port: 4173,
  strictPort: true,
  allowedHosts: true,
},
});
```
```

### Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локально в /
+ // VITE_BASE можно переопределить, если репозиторий не на GitHub
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {

```

Универсальное пособие • полный исходный код

- 
2. **\*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна иметь объект `{ "content": ... }` для вставки в ``src/data/content.ts``.
  3. **\*\*Строгая структура контента:\*\***
    - Заголовок с цветным бейджем (например, ``bg-indigo-100``)
    - Определение/правило в центре (``border-blue-500`` или ``border-blue-200``)
    - Обязательная сетка аналогий (2 колонки: неформальный текст и код)
    - "Душные" детали (код, формулы, сложные доказательства)
  4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы. Используй только классы в файлах ``src/data/content.ts``. Не используй чистый `markdown`.

---

ФАЙЛ 4/87: `index.html`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 Б

---

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
    <title> Дискретка для тупых – Интерактивный гид по дискретке
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

---

ФАЙЛ 5/87: `metadata.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 310 Б

---

```
{
  "name": "Remix Remix: ExamPrep Reader",
  "version": "1.0.0",
  "description": "A guide to the ExamPrep Reader, a tool for reading and understanding the ExamPrep Reader.",
  "keywords": ["ExamPrep Reader", "Remix Remix", "ExamPrep Reader", "Remix Remix"],
  "author": "Remix Remix",
  "license": "MIT",
  "repository": {
    "type": "git",
    "url": "https://github.com/remix-remix/exam-prep-reader"
  },
  "scripts": {
    "dev": "vite dev",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.14.2",
    "vite": "^4.3.9"
  },
  "devDependencies": {
    "@types/react": "^18.2.14",
    "@types/react-dom": "^18.2.6",
    "@types/react-router-dom": "^6.14.2",
    "typescript": "^5.0.4"
  }
}
```

```
"devDependencies": {
  "@tailwindcss/vite": "4.1.17",
  "@types/node": "22.19.17",
  "@types/pdfkit": "^0.17.6",
  "@types/react": "19.2.7",
  "@types/react-dom": "19.2.3",
  "@vitejs/plugin-react": "5.1.1",
  "esbuild": "^0.28.2",
  "tailwindcss": "4.1.17",
  "typescript": "5.9.3",
  "vite": "7.3.2",
  "vite-plugin-singlefile": "2.3.0"
},
"description": "Интерактивное пособие по алгоритмам и структурам данных",
}
```

---

ФАЙЛ 7/87: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.2 KB

---

# algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с подсветкой кода и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash
npm install
npm run dev          # предпросмотр на http://0.0.0.0:5174
npm run build        # сборка в dist/ (single-file)
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в PDF-файл.

- \* **\*\*Содержание\*\*** – на десктопе боковая панель с поиском на телефоне – липкая строка «Содержание · N из M» и в
- \* **\*\*Переходы между темами\*\*** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре
- \* **\*\*Всплывающие подсказки по терминам\*\*** – текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор Словарь – `src/data/glossary.ts`, мини-анимации – `src
- \* **\*\*Реестр интерактивов\*\*** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

## ## Структура

...

|                    |                                           |
|--------------------|-------------------------------------------|
| src/               |                                           |
| App.tsx            | – оболочка, привязка глав к визуализации  |
| components/        | – интерактивные симуляторы (Действия)     |
| data/              | – контент пособия (главы/билеты)          |
| scripts/export/    | – пайплайн код → txt → png → pdf          |
| public/export/     | – сгенерированные TXT и PDF (обновляемые) |
| .github/workflows/ | – автоматизация                           |
| ...                |                                           |

## # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и
- \* Подзаголовки: Используют тег `

### `.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.

Универсальное пособие • полный исходный код

```
-----
> * **Аналогия** – в HTML главы есть блок «Аналогия ...»
> * **2D** – к главе привязан интерактивный визуализатор
>
> Всего глав в пособии: **25**.
```

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава              | id        | Что делать           |
|--------------------|-----------|----------------------|
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| Билет         | Тема                           | Статус                                    |
|---------------|--------------------------------|-------------------------------------------|
| ---           | ---                            | ---                                       |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. Книжка читается вхолостую.     |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной главы нет; есть блок аналогии**. |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть блок аналогии**. |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; есть блок аналогии**. |

Дополнительно – «осиротевшие» визуализаторы без глав (книжки)

- \* `stack-dfs` → `StackViz.tsx` – **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` – **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` – **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` – **\*\*Динамическое программирование\*\***

|    |                                               |   |       |
|----|-----------------------------------------------|---|-------|
| 9  | 9. Графы. Топологическая сортировка           | 1 | -     |
| 10 | 10. Графы. Компоненты сильной связности (SCC) |   |       |
| 12 | 12. Графы. Точки сочленения                   | 1 | -   - |
| 14 | 14. Графы. Кратчайший путь. Дейкстра          | 3 | -     |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман      | 1 |       |
| 16 | 16. Графы. Кратчайший путь. Флойд             | - | -     |
| 17 | 17. Графы. Алгоритм Джонсона                  | - | -     |
| 18 | 18. Графы. Остовное дерево. Краскал           | 1 | -     |
| 19 | 19. Графы. Остовное дерево. Прима             | 1 | -     |
| 20 | 20. Графы. Остовное дерево. Борувка           | 1 | -     |
| 21 | 21. Префикс-функция (π), КМП                  | 4 | -   - |
| 22 | 22. Z-функция                                 | 4 | -   - |
| 23 | 23. Алгоритм Ахо–Корасик                      | 2 | -   - |
| 24 | 24. Классы сложности, сведение задач          | 4 | -   - |
| -  | Обзор: Кратчайшие пути и Графы                | - | -   - |
| -  | Алгоритмы на карте                            | - | -   - |
| -  | Эйлер: Путь ≠ Цикл                            | - | -   - |
| -  | Мосты в графах: код + визуализация            | 1 | -   - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» — минимум `inli`
4. **\*\*Решить судьбу `alg-map`\*\*** — раскрыть или удалить.

ФАЙЛ 9/87: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
  "compilerOptions": {
```



```
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algv0/, локально
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

```

const openCompilerWithViz = useCallback(() => {
  setCompilerOpen(true);
  requestAnimationFrame(() => {
    requestAnimationFrame(() => {
      document.getElementById("chapter-viz)?.scrollIntoView();
    });
  });
}, []);

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
  const onKey = (e: KeyboardEvent) => {
    const target = e.target as HTMLElement | null;
    if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target?.tagName)) return;
    if (e.metaKey || e.ctrlKey || e.altKey) return;
    if (e.key === "ArrowRight" && next) goTo(next.id);
    if (e.key === "ArrowLeft" && prev) goTo(prev.id);
  };
  window.addEventListener("keydown", onKey);
  return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapter.length) * 100, [index, chapter.length]);

return (
  <div className="min-h-screen bg-slate-950 text-slate-50">
    <Navbar
      activeTab={activeTab}
      setActiveTab={setActiveTab}
      compilerOpen={compilerOpen}
      onToggleCompiler={() => setCompilerOpen((o) => !o)}
    />

    <div className={compilerOpen ? "max-lg:pb-[58dvh]" : "max-lg:pb-[58dvh]"}>
      {activeTab === "guide" ? (
        <div>
          <MobileToc

```

```

        next={next}
        index={index}
        total={chapters.length}
        onGo={goTo}
        onOpenSimulator={setModalVizId}
        onOpenCompiler={openCompilerWithViz}
      />
    </main>
  </div>
</>
) : (
  <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
    <SimulatorHub
      onOpen={setModalVizId}
      onGoChapter={({id}) => {
        setActiveTab("guide");
        goTo(id);
      }}
    />
  </main>
)}

<footer className="p-8 text-center text-slate-600" >
  © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => {
  compilerOpen && (
    <PythonCompiler
      chapterId={activeChapter.id}
      chapterTitle={activeChapter.title}
      onOpenGuide={() => setActiveTab("guide")}
      onClose={() => setCompilerOpen(false)}
    />
  )}
/>
</div>

```

```
* <span class="term-hint" data-term-id="...">, чтобы на
* повесить всплывающую карточку (как превью ссылок в В
*/
export function annotateTerms(root: HTMLElement): void {
  if (!cachedRegex) cachedRegex = buildRegex();
  const re = cachedRegex;

  const perTerm = new Map<string, number>();
  const perSurface = new Map<string, number>();
  const walker = document.createTreeWalker(root, NodeFilter.
    acceptNode(node) {
      const parent = (node as Text).parentElement;
      if (!parent) return NodeFilter.FILTER_REJECT;
      if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
      if (!node.nodeValue || node.nodeValue.trim().length === 0) return NodeFilter.FILTER_REJECT;
      return NodeFilter.FILTER_ACCEPT;
    },
  );

  const targets: Text[] = [];
  let current = walker.nextNode();
  while (current) {
    targets.push(current as Text);
    current = walker.nextNode();
  }

  for (const textNode of targets) {
    const text = textNode.nodeValue ?? "";
    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
      const surface = m[1].toLowerCase();
    }
  }
}
```

```
    try {
      annotateTerms(el);
    } catch {
      /* если браузер не умеет lookbehind – просто оста
    }
  };

  // при смене главы
  // eslint-disable-next-line react-hooks/exhaustive-dep
  useEffect(run, deps);

  // страховка: контент перерисовали, а подсказок в нём
  useEffect(() => {
    const el = ref.current;
    if (!el) return;
    if (!el.querySelector(".term-hint")) run();
  });
}
```

---

ФАЙЛ 13/87: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 KB

---

```
@import "tailwindcss";

@layer utilities {
  .neon-glow-blue {
    box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
  }
  .neon-glow-emerald {
    box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
  }
  .neon-glow-rose {
    box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
  }
  .neon-glow-yellow {
```

```

    from {
      opacity: 0;
      transform: translateY(4px);
    }
    to {
      opacity: 1;
      transform: translateY(0);
    }
  }

.term-popover {
  animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
  cursor: help;
  border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
  color: #c7d2fe;
  border-radius: 3px;
  padding: 0 1px;
  transition:
    background-color 0.15s ease,
    color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
  background: rgba(99, 102, 241, 0.22);
  color: #fff;
  outline: none;
}
@media (hover: none) {
  .term-hint {
    border-bottom-style: solid;
    background: rgba(99, 102, 241, 0.12);
  }
}

```

```
.chapter-body :where(h2) {
  font-size: 1.25rem !important;
  line-height: 1.3 !important;
}
.chapter-body :where(h3) {
  font-size: 1.05rem !important;
  line-height: 1.35 !important;
}
.chapter-body :where(.p-6, .p-8) {
  padding: 0.9rem !important;
}
.chapter-body :where(.text-3xl) {
  font-size: 1.4rem !important;
}
.chapter-body :where(.text-2xl) {
  font-size: 1.2rem !important;
}
.chapter-body :where(.text-xl) {
  font-size: 1.05rem !important;
}
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #0f172a;
}
::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #475569;
}
```

```
    description?: string;
    category?: string;
}
```

---

ФАЙЛ 16/87: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {
  const src: string;
  export default src;
}
declare module '*.jpg?url' {
  const src: string;
  export default src;
}
declare module '*.jpeg' {
  const src: string;
  export default src;
}
declare module '*.png' {
  const src: string;
  export default src;
}
```

---

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

---

ФАЙЛ 17/87: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 934 | РАЗМ

---



## Аналогия 2: Ступени

```

    </div>
    <p class="text-slate-300 text-sm mb-6">
        нужно покрыть отрезок, мы берем две самые бо
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg border
    <summary class="p-4 font-bold text-slate-300
        -b border-slate-700/50">
            Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <pre class="bg-slate-950 p-4 rounded"
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky)
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R
int minimum = min(ans1, ans2);</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "treap",
        title: "3. Декартово дерево (Treap)",
        type: "html",
        description: "Дерево поиска по ключу и Куча по приори
        category: "Продвинутые структуры",
        content: `
<section id="treap" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border

```

```

    } else {
      auto [L, R] = split(t->left, x);
      t->left = R;
      return {L, t};
    }
  }</pre>
</div>
</details>
</div>
</div>
</section>`,
  },
  {
    id: "splay-tree",
    title: "4. Splay-дерево",
    type: "html",
    description: "Дерево поиска, которое чинит само себя",
    category: "Продвинутые структуры",
    content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">

    <!-- 0. Определение -->
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-lg sm:text-xl font-mono">
        поиска <span class="text-slate-500">без</span>
        : серия поворотов, которая поднимает v в кор
        <p class="text-sm text-slate-400 text-c">
        т – поворот.</p>
      </div>

```

```

<ul class="list-disc list-inside text-slate-300">
  <li><span class="text-white font-bold">Важно!</span>
    каждой вставки чинят дерево, чтобы оно <em>не</em>
  </li>
  <li><span class="text-white font-bold">Важно!</span>
    менно быть кривым. Но каждый раз, когда мы
    ровнее становится.</li>
</ul>
</div>

```

<!-- 2. Аналогии -->

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
    Аналогия 1: стопка бумаг на столе
  </div>
  <p class="text-slate-300 text-sm">У тебя
    скиваешь его и, поработав, кладёшь <span class="text-white font-bold">Важно!</span>
    неделю самые нужные бумаги сами собой оказыва
    ет ровно это: «взял → положил наверх».</p>
  </div>
  <div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md">
    Аналогия 2: тропинка через газон
  </div>
  <p class="text-slate-300 text-sm">Ты идёшь по дорожке,
    ода дорожка сама укорачивалась вдвое: чем чаще
    оплачивает все следующие. Пойдёшь один раз
  </p>
</div>

```

<!-- 3. Поворот -->

```

<div class="bg-slate-800/60 p-6 rounded-xl border border-emerald-500 shadow-md">
  <h3 class="text-lg font-bold text-white mb-4">Поворот
  <p class="text-slate-300 text-sm mb-4">Поворот — это движение,
    движение, три перевешенные ссылки</span> и

```

```

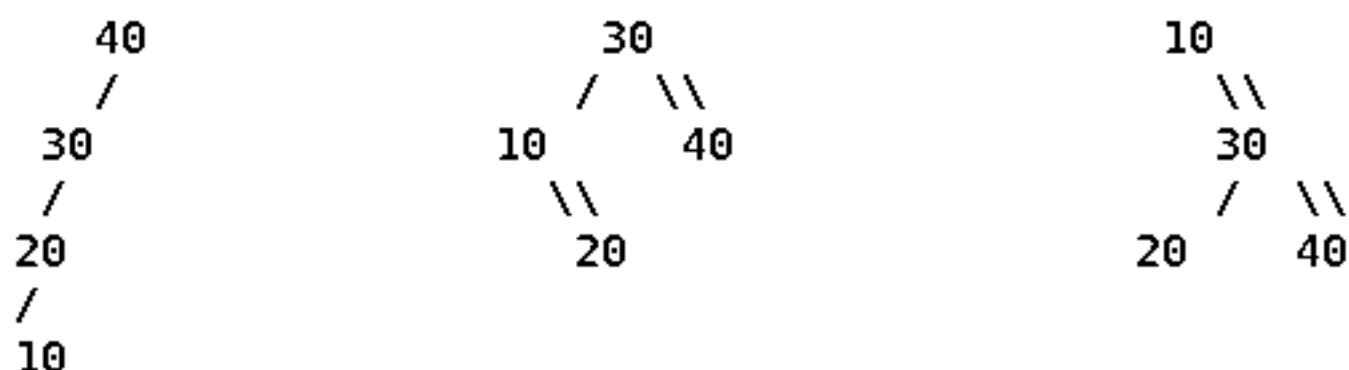
        </tr>
    </thead>
    <tbody class="text-slate-300">
        <tr class="border-b border-slate-200">
            <td class="py-3 pr-3 font-bolder">Знаменитый
            <td class="py-3 pr-3">деда
            <td class="py-3 pr-3">сам х
            <td class="py-3">х стал королем
        </tr>
        <tr class="border-b border-slate-200">
            <td class="py-3 pr-3 font-bolder">Знаменитый
            <td class="py-3 pr-3">х и п
аво-право)</td>
            <td class="py-3 pr-3"><span class="font-bolder">Знаменитый
            <td class="py-3">х поднялся
        </tr>
        <tr>
            <td class="py-3 pr-3 font-bolder">Знаменитый
            <td class="py-3 pr-3">х и п
            <td class="py-3 pr-3"><span class="font-bolder">Знаменитый
            <td class="py-3">п и г стали
        </tr>
    </tbody>
</table>
</div>

<p class="text-slate-300 text-sm mt-4">И та
нова смотрим на тройку. Zig может случиться
<p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделатъ руками.</p>
</div>

<!-- 5. Главная ловушка -->
<div class="bg-rose-950/30 p-6 rounded-xl border">
    <h3 class="text-lg font-bold text-rose-300">
    <p class="text-slate-300 text-sm mb-4">Это
ми.</p>
    <div class="grid grid-cols-1 lg:grid-cols-2"

```

```
<p class="text-slate-300 text-sm mb-4">Дерево  
3 шага, а потом делаем Splay(10).</p>  
<pre class="bg-slate-950 p-4 rounded-lg overflow-x-  
0 leading-relaxed">старт           после Zi  
        глубина 3 → 1           10 в корне
```



итог: было 4 узла в линию (высота 4) → стало высота 3,  
а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра  
инили дерево по пути</span>. Все узлы, мимо  
</div>
```

```
<!-- 7. Амортизация -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2  
<h3 class="text-lg font-bold text-white mb-3">  
<p class="text-slate-300 text-sm mb-3">Стро  
оддерева) по всем узлам, и Zig-Zig/Zig-Zag  
ах идея такая:</p>  
<div class="grid grid-cols-1 md:grid-cols-3  
  <div class="bg-slate-900 rounded-lg p-4  
    <p class="text-white font-bold mb-1">  
    <p class="text-slate-400 text-xs">c  
  </div>  
  <div class="bg-slate-900 rounded-lg p-4  
    <p class="text-white font-bold mb-1">  
    <p class="text-slate-400 text-xs">H  
  </div>  
  <div class="bg-slate-900 rounded-lg p-4  
    <p class="text-white font-bold mb-1">  
    <p class="text-slate-400 text-xs">д
```

```

-----
// один поворот: x встаёт на место своего родителя
function rotate(x) {
    const p = x.parent, g = p.parent;

    if (p.left === x) {                // правый поворот
        p.left = x.right;
        if (x.right) x.right.parent = p;
        x.right = p;
    } else {                          // левый поворот (зеркало)
        p.right = x.left;
        if (x.left) x.left.parent = p;
        x.left = p;
    }

    p.parent = x;                    // родитель уехал вниз
    x.parent = g;                    // x занял его место
    if (g) { if (g.left === p) g.left = x; else g.right = x; }
}

// поднимаем x в корень
function splay(x) {
    while (x.parent) {
        const p = x.parent, g = p.parent;

        if (!g) {                    // ZIG: деда нет
            rotate(x);
        } else if ((g.left === p) === (p.left === x)) {
            rotate(p);                // ZIG-ZIG: сн
            rotate(x);
        } else {
            rotate(x);                // ZIG-ZAG: сн
            rotate(x);
        }
    }
    return x;                        // теперь x —
}

// поиск: спустились и подняли найденное наверх

```

```

<details class="bg-slate-900/40 rounded-lg border-
  <summary class="p-4 font-bold text-slate-400
  open:border-b border-slate-700/50">
    Вопросы, которые задают на экзамене (
  </summary>
  <div class="p-5 text-sm text-slate-300 space-
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Последний
    енный или преемник). Splay делают всегда, да
    бы, и амортизация сломалась бы.</p>
    </div>
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Потому что
    еворачивают бамбук в другую сторону (см. бл
    </div>
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Чередовани
    вает один из них в корень, превращая дерево
    ступает только для <em>одной</em> конкретной
    </div>
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Treap дер
    play не хранит вообще ничего и держит балан
    p>
    </div>
  </div>
</details>

</div>
</section>`,
  },
  {
    id: "prefix-sums-2d",
    title: "5. Двумерная матрица префиксных сумм",
  }

```

```

        content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-blue-400 m
      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-lg text-blue-300 italic m
        <p class="text-xl font-mono text-white"
    ый граф можно раскрасить 4 цветами.</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4 l
        rder border-emerald-500 shadow-md">
          Аналогия 1: Карта мира
        </div>
        <p class="text-slate-300 text-sm mb
      . Границы не пересекаются в одной точке по-
      ы не сливались цветами, Всегда можно всего
    </div>
  </div>
</div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">

```



```

        description: "Разбиение орграфа на макро-вершины. А
        category: "Графы. Продвинутые",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        нвертированному в порядке top-sort.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Улицы с односторонним движением.
          </div>
          <p class="text-slate-300 text-sm mb-4">
            БЫХ направлениях по односторонним улицам. Как это отразится
            у на карте.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
      },
      {
        id: "graph-bridges",
        title: "11. Графы. Мосты",
        type: "html",
        description: "Рёбра, удаление которых расхреначивает граф",
        category: "Графы. Продвинутые",
        content: `

```

```

</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-rose-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-4"
      <p class="text-lg text-rose-300 italic"
      <p class="text-xl font-mono text-white"
тей > 1).</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия 1: Центральный роутер
        </div>
        <p class="text-slate-300 text-sm mb-4"
правильный роутер - и два сегмента офиса б
      </div>
    </div>
  </div>
</div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-indigo-400

```

```

        </div>
        <div class="grid grid-cols-1 gap-6">
            <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                    Аналогия: Копаем туннель под .
                </div>
                <p class="text-slate-300 text-sm mb-4">
                    ней земли. Но если начать копать одновременно
                    в 2 РАЗА меньше работы (геометрически: площ
                </div>
            </div>
        </div>
    </div>
</section>`,
    },
    {
        id: "mst-kruskal",
        title: "18. Графы. Остовное дерево. Краскал",
        type: "html",
        description: "",
        category: "Графы. Остовные",
        content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
                <h3 class="text-xl font-bold text-emerald-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4">
                    <p class="text-lg text-emerald-300 italic">
                    <p class="text-xl font-mono text-white">
                        (проверяем через DSU) - берем в ответ.</p>
                </div>
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">

```

```

        </div>
        <p class="text-slate-300 text-sm mb-4">
ли вирус). Мы стартуем из одного города, и
Сеть растет только из одного связного куска
        </div>
    </div>
</div>
</div>
</section>`,
    },
    {
      id: "mst-boruvka",
      title: "20. Графы. Остовное дерево. Борувка",
      type: "html",
      description: "",
      category: "Графы. Остовные",
      content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
ается с соседом.</p>
      </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border-emerald-500 shadow-md">
          Аналогия: Племена
        </div>
        <p class="text-slate-300 text-sm mb-4">
изкому племени для объединения. К вечеру пл

```

```

        </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg border-1
        <summary class="p-4 font-bold text-slate-300
        -b border-slate-700/50">
            Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300
            <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "string-z-func",
        title: "22. Z-функция",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border-1">
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-6">

```

```

description: "",
category: "Теория сложности",
content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1 rounded-md">
    <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-purple-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">
        <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Судoku (P vs NP)
          </div>
          <p class="text-slate-300 text-sm mb-4">
пример сортировка чисел). Класс <b>NP</b> —
енную сетку (ответ), он БЫСТРО (за класс P)
        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-purple-500 shadow-md">
            Аналогия 2: Машина-переводчик
          </div>
          <p class="text-slate-300 text-sm mb-4">
ь отличный переводчик на английский (Сведен
аче B, то B — это <b>NP-Полная</b> (сосредо
        </div>
      </div>
    </div>
  </div>
`

```

ин раз. Вершины повторять можно.</p>

```
<p><strong class="text-white">Эйлер</strong>
<div class="p-3 bg-slate-950 rounded-lg border border-rose-500 shadow-md">
  <p class="text-sm text-blue-300">
    <p><strong class="text-white">Путь:
    <p><strong class="text-white">Цикл:
  </div>
</div>
</div>
```

```
<p class="text-slate-300 text-sm">
  <strong>Билет 13 (Различие пути и цикла)
  <br>• <span class="text-green-400 font-weight-bold">шел-вошёл-вышел).
  <br>• <span class="text-yellow-400 font-weight-bold">финиш).
</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-rose-500 shadow-md">
    <div class="absolute -top-3 left-4 bg-slate-950 border border-green-500 shadow-md">
      Путь: Нормальная прогулка
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Ты вышел из дома <strong>(нечётная)</strong>.
      талый пришёл в бар <strong>(вторая нечётная)</strong>.
      Домой вернуться не смог, потому что
    </p>
  </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md">
  <div class="absolute -top-3 left-4 bg-rose-500 border-rose-500 shadow-md bg-slate-950">
    Цикл: Гамильтонов синдром (болезнь)
  </div>
  <p class="text-slate-300 text-sm mb-4">
```

```
<div class="flex items-center mb-6">
  <span class="bg-emerald-600 text-white px-4 py-4">
    <h2 class="text-3xl font-bold text-white">Мосты
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500">
    <h3 class="text-xl font-bold text-emerald-400">Мост: Единственная веревка

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">Представь, что ты альпинист. Ты спустился в пещеру.
      <div class="text-left font-mono text-sm">
        <p><strong class="text-white">tin[v]
        <p><strong class="text-white">low[v]
        <div class="p-3 bg-slate-950 rounded">
          <span class="text-rose-400 font-mono text-sm">
            <p class="text-xl font-bold text-white">Мост: Единственная веревка
            <p class="text-xs text-slate-400">
          </div>
        </div>
      </div>
    </div>
```

```
<p class="text-slate-300 text-sm">
  Если из сына u и всех его потомков <strong>
  Единственная ниточка. Порвётся — граф р
</p>
```

```
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500">
    <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
      Мост: Единственная веревка
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь, что ты альпинист. Ты спустился в пещеру.
      Из пещеры и нет других выходов — только веревка.
      Если верёвка (ребро v-u) оборвётся —
```



```

int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;           // устанавливаем

    for (int to : adj[v]) {
        if (to == p) continue;         // не идём назад

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v);                 // рекурсивный вызов
            low[v] = min(low[v], low[to]);

            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
}

```

</div>

<p class="text-xs text-slate-400">Занес

</div>

```
order-green-500 shadow-md">
```

К5: Полный граф на 5 вершинах

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У К5: `class="text-white">V=5, -white">10 &gt; 9</code> – БАХ, непланарен!`

Все 5 вершин соединены со всеми. На

Теорема Куратовского: К5 – эталон не

Если внутри графа спрятан К5 – забудь

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border
```

```
<div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
```

К3,3: 3 дома и 3 колодца

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Три дома хотят соединиться с тремя

Нарисовать без пересечений невозможно

У него `class="text-white">V=6 xt-white">9 ≤ 12</code> – проходит, но он в`

Почему? Потому что у двудольного графа

Отсюда более строгое ограничение: `<`

`class="text-white">9 &gt; 8</code>`

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border
```

```
<summary class="p-4 font-bold text-slate-400
order-slate-700/50">
```

Доказательство непланарности К5 (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space
```

```
<p class="text-blue-400">Доказательство
```

```
<p>
```

Пусть К5 планарен.  $V = 5$ ,  $E = 10$ .

```
<p class="text-slate-300 text-sm">
  <strong>Важно:</strong> Это работает то
  епятся к точкам сочленения. То есть, <strong>
  ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
    <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
      Аналогия: Главный перекресток
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь город, где два района соединены
      её перекроют (удалят вершину) – районы будут
      <strong>хрупкость</strong> системы.
    </p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
    <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
      Телеком: Центральный свитч
    </div>
    <p class="text-slate-300 text-sm mb-4">
      У тебя несколько компьютеров в одной
      абелем (мостом). Сами свитчи – это точки со
    </p>
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-800">
  <summary class="p-4 font-bold text-slate-400">
    order-slate-700/50">
      Душный мод: Код и корень DFS (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300 space-y-2">
```

```
        <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
        <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
        <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
</ul>
    </div>
</div>
</section>`,
    },
];
```

---

ФАЙЛ 19/87: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

---

```
import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as newChapters } from "./content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
  ...graphChapters,
  ...additionalTickets,
  ...tickets6to13,
  ...tickets14to20,
  ...tickets21to24
].filter(c => c && c.id);
```

```
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

export interface DebugStep {
  /** Номер строки в коде пользователя (1-based). */
  line: number;
  /** Имя функции-фрейма ("<module>" – верхний уровень) */
  func: string;
  /** Локальные переменные: имя → герг (усечённый). */
  locals: Record<string, string>;
}

export interface DebugResult {
  steps: DebugStep[];
  /** Текст последней необработанной ошибки (или пустая) */
  error: string;
  /** true, если уперлись в лимит шагов. */
  truncated: boolean;
}

/** Лимит шагов трассы – защита от бесконечных циклов и
export const DEBUG_STEP_CAP = 500;

/**
 * Строит самодостаточный python-скрипт: выполняет userSrc
 * и возвращает JSON с шагами (строка + func + locals)
 * последнего выражения (его вернёт runPythonAsync).
 */
export function buildDebugRunner(userSrc: string): string {
  const srcLiteral = JSON.stringify(userSrc); // JSON-сериализация
  return `import sys as __sys, json as __json`
```

```

__OUT = __json.dumps({
    "steps": __steps,
    "error": __err,
    "truncated": len(__steps) >= __CAP,
})
__OUT
\;
}

/**
 * Какие переменные изменились на этом шаге относительно
 * они подсвечиваются (как amber-подсветка изменений в
 * Новые переменные тоже считаются изменёнными.
 */
export function changedVars(
    prev: Record<string, string> | null | undefined,
    cur: Record<string, string>
): Record<string, boolean> {
    const out: Record<string, boolean> = {};
    for (const k of Object.keys(cur)) {
        out[k] = !prev || !(k in prev) || prev[k] !== cur[k];
    }
    return out;
}

/** Базовое имя из карточки переменной: "tin[v]" → "tin"
export const baseVarName = (name: string): string => name.slice(0, -1);

/**
 * Порядок показа в панели переменных: сначала переменные
 * страницы (те, что подсвечивает визуализация), остальные
 */
export function orderVars(
    cur: Record<string, string>,
    syncNames: string[]
): [string, string][] {
    const inSync = new Set(syncNames);

```

```
export const GLOSSARY: GlossaryEntry[] = [
  {
    id: "bfs",
    labels: ["BFS", "обход в ширину", "поиск в ширину",
    title: "BFS – обход в ширину",
    short:
      "Волна от источника: сначала все соседи, потом со
    formal:
      "Очередь (FIFO). Кладём старт, достаём вершину –
        у рёбер.",
    complexity: "O(V + E)",
    demo: "bfs",
    simulator: "queue-bfs",
  },
  {
    id: "dfs",
    labels: ["DFS", "обход в глубину", "поиск в глубину",
    title: "DFS – обход в глубину",
    short:
      "Прёшь вперёд до упора, упёрся – откатываешься на
    formal:
      "Стек (или рекурсия). Из DFS растут: времена вход
    complexity: "O(V + E)",
    demo: "dfs",
    simulator: "stack-dfs",
  },
  {
    id: "heap",
    labels: ["куча", "кучу", "кучи", "куче", "кучей", "к
    title: "Куча (priority queue)",
    short:
      "Не отсортированный список, а «мешок, из которого
    formal:
      "Полное бинарное дерево в массиве: дети узла i ле
    complexity: "вставка/извлечение O(log n), «посмотре
    demo: "heap",
    simulator: "heap-beam-search",
  },
],
```

```

    demo: "trie",
    simulator: "aho-corasick",
},
{
    id: "bfs-on-trie",
    labels: [
        "обход дерева",
        "обход бора",
        "обход в ширину по бору",
        "какой-то обход",
        "обходом дерева",
        "конечный автомат",
        "конечного автомата",
        "автомат",
        "автомата",
    ],
    title: "Обход бора в ширину (тот самый «какой-то обход»)",
    short:
        "В Ахо–Корасике бор обходят именно BFS: суффиксную очередь  

        о есть строго по уровням.",
    formal:
        "Кладём в очередь детей корня (их ссылка = корень).  

        Кладём с в очередь.",
    complexity: "O(суммарной длины словаря × алфавит)",
    demo: "trie-bfs",
    simulator: "aho-corasick",
},
{
    id: "suffix-link",
    labels: ["суффиксная ссылка", "суффиксные ссылки"],
    title: "Суффиксная ссылка",
    short:
        "«Куда откатиться, если следующая буква не подошла»",
    formal: "link(v) = go(link(parent(v)), ch). Считает",
    demo: "suffix-link",
    simulator: "aho-corasick",
},

```



```

        "разреженная таблица", "разреженная таблица", "sparse-table",
        "разреженная", "двоичные подьёмы", "двоичный подьём",
    ],
    title: "Разреженная таблица (метод двоичных подьёмов)",
    short:
        "Заранее считаем ответ для всех отрезков длины 1, 2, ...,
        М.",
    formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
    complexity: "препроцессинг O(n log n), запрос O(1)",
    demo: "sparse-table",
    simulator: "sparse-table",
},
{
    id: "segment-tree",
    labels: ["дерево отрезков", "дереве отрезков", "дерево отрезков"],
    title: "Дерево отрезков",
    short:
        "Матрёшка из отрезков: корень отвечает за весь массив,
        левый кусок.",
    formal: "Строится за O(n), запрос и обновление — O(log n)",
    demo: "segment-tree",
    simulator: "segment-trees",
},
{
    id: "bridge",
    labels: ["мост", "моста", "мосты", "мостов", "мостовые"],
    title: "Мост",
    short:
        "Ребро, после удаления которого граф разваливается на
        две части.",
    formal: "Ребро (u,v) — мост ⇔ low[v] > tin[u], где tin[u] — время
        первого посещения u, low[v] — минимальное tin среди всех
        ребро в родителя — нет).",
    complexity: "O(V + E) одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "articulation

```

```

        "класс Р", "полином", "полиномиальное",
    ],
    title: "Класс NP и NP-полнота",
    short:
        "NP — «решение проверить легко, найти трудно» (судно
        не в NP.",
    formal: "Задача NP-полна, если она в NP и к ней полнота",
    demo: "np",
},
{
    id: "potential",
    labels: ["потенциал", "потенциалы", "потенциалов",
    title: "Потенциалы (перевзвешивание Джонсона)",
    short:
        "Трюк «поднять все дороги на одинаковую высоту»:
    formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$ , где  $h$  — раск
        к путей сохраняется.",
    demo: "relax",
    simulator: "johnson-algo",
},
{
    id: "scc",
    labels: ["компонента сильной связности", "компоненты",
    title: "Компоненты сильной связности",
    short:
        "Группы вершин, где из каждой можно дойти до каждо
    formal: "Косарайю: DFS с сохранением порядка выхода",
    complexity: " $O(V + E)$ ",
    demo: "scc",
    simulator: "scc-kosaraju",
},
{
    id: "prefix-function",
    labels: [
        "префикс-функция", "префикс-функции", "префикс-функ
        "префикс", "префикса", "суффиксом", "суффикс", "суб
        "подстроки", "подстроку", "подстрока",
    ],

```

```

    },
    {
      id: "shortest-path",
      labels: ["кратчайший путь", "кратчайшие пути", "кра",
      title: "Кратчайший путь",
      short:
        "Самый дешёвый маршрут из А в В. «Дешёвый» – по с
      formal:
        "Дейкстра – неотрицательные веса,  $O(E \log V)$ . Форд
          ы,  $O(V^3)$ .",
      demo: "relax",
      simulator: "dijkstra",
    },
    {
      id: "negative-cycle",
      labels: ["отрицательный цикл", "отрицательного цикла",
      title: "Отрицательный цикл",
      short:
        "Круг, пройдя по которому вы становитесь «богаче»
      formal:
        "Форд–Беллман: если на  $V$ -й итерации ещё удастся с
      demo: "relax",
      simulator: "bellman-ford",
    },
    {
      id: "greedy",
      labels: [
        "жадный алгоритм", "жадный", "жадно", "жадная",
        "самое дешевое ребро", "самое дешёвое ребро", "са
      ],
      title: "Жадный алгоритм",
      short:
        "На каждом шаге хватаем то, что лучше прямо сейчас
      formal:
        "Корректность обычно доказывается «леммой о разре
      demo: "mst",
      simulator: "mst-kruskal",
    },
  ],

```



```
{
  id: "kruskal",
  labels: ["Краскал", "Краскала", "Краскалу", "Kruska"],
  title: "Алгоритм Краскала",
  short:
    "Сортируем все рёбра по весу и берём подряд те, ч
  formal:
    "Сортировка  $O(E \log E) + E$  операций union/find. C
  complexity: " $O(E \log E)$ ",
  demo: "mst",
  simulator: "mst-kruskal",
},
{
  id: "components",
  labels: [
    "компонента связности", "компоненты связности", "
    "компоненте связности", "связности", "связный", "
  ],
  title: "Компоненты связности",
  short:
    "Граф-архипелаг: острова, между которыми нет мост
    бой.",
  formal:
    "Считаются одним проходом: пока есть непосещённая
  complexity: " $O(V + E)$ ",
  demo: "components",
  simulator: "graph-dfs-bfs",
},
{
  id: "graph",
  labels: [
    "граф", "графа", "графе", "графы", "графов", "гра
    "вершины", "вершина", "вершин", "ребра с весом",
  ],
  title: "Граф",
  short:
    "Точки (вершины) и связи между ними (рёбра). Горо
  formal:
```

```

    complexity: "O(высоты): от  $O(\log n)$  до  $O(n)$ ",
    demo: "segment-tree",
},
{
    id: "inclusion-exclusion",
    labels: [
        "включений-исключений", "включения-исключения", "вырезание прямоугольников",
    ],
    title: "Включения-исключения на префиксных суммах",
    short:
        "Чтобы получить сумму прямоугольника, берём большой отрезали дважды.",
    formal:
        "Sum( $x1..x2$ ,  $y1..y2$ ) =  $P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]$ ",
    complexity: "запрос  $O(1)$  после  $O(nm)$  предподсчёта",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "reduction",
    labels: ["сведение", "сведения", "сведении", "сведё"],
    title: "Сведение задач",
    short:
        "«Я не умею решать A, зато умею B». Если A можно",
    formal:
        " $A \leq_p B$ : есть полиномиальная функция, переводящая NP-полную задачу к новой.",
    demo: "np",
},
{
    id: "range",
    labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
    title: "Запрос на отрезке",
    short:
        "Спрашиваем не про один элемент, а про кусок массива предподсчёты.",
    formal:

```

```

    demo: "zig",
    simulator: "splay-rotations",
  },
  {
    id: "zig-zig",
    labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
    title: "Zig-Zig – x и родитель с одной стороны",
    short:
      "Дерево вытянулось в «бамбук»: g → p → x идут в од
      м (p, x). Именно это вдвое сокращает глубину вс
    formal:
      "Если крутить наивно (два раза поднимать x), бамб
    demo: "zig-zig",
    simulator: "splay-rotations",
  },
  {
    id: "zig-zag",
    labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
    title: "Zig-Zag – x и родитель с разных сторон",
    short:
      "Узлы идут «змейкой»: p – левый сын g, а x – прав
      p и g становятся двумя детьми x.",
    formal: "Эквивалентно двойному повороту AVL-дерева
    demo: "zig-zag",
    simulator: "splay-rotations",
  },
];

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) :

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string; }[] =
  e.labels.map((label) => ({ label, id: e.id }))
).sort((a, b) => b.label.length - a.label.length);

```

---

```
</div>
```

```
<div class="space-y-8">
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border
```

```
    <h3 class="text-xl font-bold text-emerald-4
```

```
  <div class="bg-slate-900 p-4 rounded-lg mb-1
```

```
    <p class="text-lg text-emerald-300 italic
```

```
    <p class="text-xl font-mono text-white"
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2
```

```
  <!-- Аналогия 1 -->
```

```
  <div class="bg-slate-800 p-6 rounded-lg
```

```
    <div class="absolute -top-3 left-4 l
  rder border-emerald-500 shadow-md">
```

```
      Аналогия 1: Позитивное плани
```

```
    </div>
```

```
    <p class="text-slate-300 text-sm mb
  (нельзя поехать и заработать). Он всегда в
```

```
    <div id="slot-chapter-image-dijkstr
  </div>
```

```
  <!-- Аналогия 2 -->
```

```
  <div class="bg-slate-900 p-6 rounded-lg
    <div class="absolute -top-3 left-4 l
  border-rose-500 shadow-md">
```

```
      Ограничения
```

```
    </div>
```

```
    <p class="text-slate-300 text-sm mb
  охождение улицы), Дейкстра сломается, так к
  </div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
```

```
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">
```

```
    Строгое доказательство (Скрыто)
```



```

        </div>
        <p class="text-slate-300 text-sm mb-6">
        еряет ВСЕ возможные дороги V-1 раз подряд.
        <div id="slot-chapter-image-bellman" class="img-fluid">
        </div>

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4
            border-rose-500 shadow-md">
                Отрицательный цикл
            </div>
            <p class="text-slate-300 text-sm mb-6">
            г станет ЕЩЕ короче – значит мы попали во в
            </div>
        </div>

        <div id="slot-bellman-viz" class="mt-8"></div>
    </div>
</div>
</section>`
    },
    {
        id: "floyd",
        title: "Билет 16. Флойд-Уоршелл",
        type: "html",
        category: "Графы",
        content: `<section id="floyd-content" class="mb-12">
        <div class="flex items-center mb-6">
            <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-3xl font-bold text-white">Флойд
        </div>

        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border">
                <h3 class="text-xl font-bold text-indigo-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-6">

```

```
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-l">
    <h3 class="text-xl font-bold text-blue-400 mb-4">

    <div class="bg-slate-900 p-4 rounded-lg mb-6 text">
      <p class="text-sm text-blue-300 italic mb-2">0с
      <p class="text-lg font-mono text-white">Бор (Tr
    </div>
    <p class="text-slate-300 text-sm">Позволяет найти
      -mono text-amber-300 bg-black/30 px-1 rounded">
      y.</p>
  </div>
```

<!-- Аналогии -->

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo">
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      мы движемся по веткам. Если нужного продолж
      ("нити страховки") в самый длинный из извест
    </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2">
  <div class="absolute -top-3 left-4 bg-rose-900
    -500 shadow-md">
    Аналогия 2: Матёшка (Терминальные)
  </div>
  <p class="text-slate-300 text-sm mb-4">Представ
    нашли и "he", потому что оно спрятано внутри
    терминальные ссылки</b> (term link) – прямые мос
```

```

        // Если суффиксная ссылка сама является словом
        // Иначе наследуем terminal_link
        if (nodes[nodes[child].link].is_terminal) {
            nodes[child].term_link = nodes[child].link;
        } else {
            nodes[child].term_link = nodes[nodes[child].link].term_link;
        }

        q.push(child);
    }
}

</div>
</div>
</details>
</div>
</section>`
},
{
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-12">
<div class="flex items-center mb-6">
    <span class="bg-purple-600 text-white px-4 py-1 rounded">Алгоритмы
    <h2 class="text-3xl font-bold text-white">Алгоритмы на карте
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-purple-400">Алгоритмы на карте
        <p class="text-slate-300 text-sm mb-4">Когда мы имеем дело с графами, которые могут
        содержать очень большие и маленькие числа (долгота и широта), мы часто
        работаем с числами, выходящими за пределы диапазона от 0 до N, и уже по ним строим графы.</p>
    </div>
    </div>
</div>
</section>`

```

```

        ого критерия. Это NP-полная задача, которую не
    },
    {
        question: "Куда ведёт эйлеров путь, если нечётных
        options: [
            "Всё ещё можно пройти по всем рёбрам ровно раз",
            "Такого графа не существует",
            "Эйлерова пути нет — нужно удалять лишние рёбра",
        ],
        correctIndex: 2,
        explanation: "Если нечётных вершин больше 2, эйлер
            ровно 2 нечётные или 0."
    }
],
"bridges-code": [
    {
        question: "После DFS у нас low[u] = 4, а tin[v] =
        options: [
            "Да, так как low[u] (4) > tin[v] (2)",
            "Нет, low[u] должно быть меньше или равно",
            "Только если граф связный и неориентированный"
        ],
        correctIndex: 0,
        explanation: "Условие моста: low[u] > tin[v]. В н
    },
    {
        question: "Для чего нужен массив low[v]?",
        options: [
            "Для того чтобы было больше переменных в коде",
            "Чтобы знать минимальный tin предка, до которого
            "Чтобы хранить глубину рекурсии"
        ],
        correctIndex: 1,
        explanation: "low[v] хранит минимальное время вхо
            поиска мостов и точек сочленения."
    },
    {
        question: "Какова временная сложность алгоритма п

```

```
        correctIndex: 0,
        explanation: "У K5: V=5, E=10, 3V - 6 = 9. 10 > 9
    }
}
};
```

---

ФАЙЛ 24/87: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 113 | РАЗМ

---

```
import { createContext, useContext, useEffect } from "r
```

```
/**
```

```
 * Связь «компилятор ↔ открытая визуализация».
```

```
 *
```

```
 * Панель компилятора в режиме пошагового отладчика тра
```

```
 * текущего шага трассы вместе с id текущей главы; поша
```

```
 * главы (через useVizStepSync) подхватывают его и прыг
```

```
 * визуализация идёт в ногу с листингом (каждая исполня
```

```
 * = очередной шаг подсветки, см. vizSync.ts).
```

```
 */
```

```
export const VizChapterContext = createContext("");
```

```
const CHANNEL = "algo:viz-step";
```

```
export interface VizStepEventDetail {
```

```
  chapterId: string;
```

```
  step: number;
```

```
}
```

```
/** Транслировать текущий шаг отладчика визуализациям т
```

```
export function emitVizStep(chapterId: string, step: num
```

```
  if (typeof window === "undefined") return;
```

```
  window.dispatchEvent(new window.CustomEvent<VizStepEve
```

```
}
```

```

-----
    for (let t = 0; t <= idx && t < steps.length; t++) {
        const content = srcLines[steps[t].line - 1];
        if (content !== undefined && markedContents.has(content)) {
            hits++;
        }
    }
    return Math.max(0, hits - 1);
}

/** Сколько шаговых («помеченных») строк успело выполниться до idx
export function vizHitsAtTrace(
    steps: ReadonlyArray<{ line: number }>,
    idx: number,
    srcLines: readonly string[],
    markedContents: ReadonlySet<string>
): number {
    let hits = 0;
    for (let t = 0; t <= idx && t < steps.length; t++) {
        const content = srcLines[steps[t].line - 1];
        if (content !== undefined && markedContents.has(content)) {
            hits++;
        }
    }
    return hits;
}

/* — Выбор демонстрации внутри страницы (вкладки ID / chapterId)

const DEMO_CHANNEL = "algo:viz-demo";

export interface VizDemoEventDetail {
    chapterId: string;
    /** base-запись страницы – demoId пустой; подписанные демо-страницы
    demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта
export function emitVizDemo(chapterId: string, demoId: string) {
    if (typeof window === "undefined") return;
    window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>("viz-demo", {
        detail: { chapterId, demoId },
    }));
}

```

```

-----
/** Заголовок демонстрации на странице (если интерактив)
vizTitle?: string;
/** Чему соответствует один шаг визуализации (для тул)
stepNote?: string;
/** Переменные синхронизации. Пусто – на странице нет
variables: SyncVariable[];
/**
 * Минимальный исполняемый скелет для редактора: стро
 * шаг-в-шаг с подсветкой визуализации (без словесных
 */
code?: string;
/**
 * Номера строк ВНУТРИ code (с 1), каждое выполнение
 * шаг визуализации: по ним отладчик точно переводит
 * шаг демонстрации (иначе – грубо «строчный индекс =
 */
stepCodeLines?: number[];
/**
 * Может ли демонстрация страницы ходить за отладчиком
 * false/нет – интерактив руками (клики/наведение): ч
 */
stepDriven?: boolean;
}

/** Справочные значения, указанные внутри самих симулято
export const PAGE_SYNC: Record<string, PageSync> = {
  "segment-trees": {
    vizTitle: "Дерево отрезков",
    stepNote: "Шаг = спуск по вершине v: либо читаем tr
    code: `n = 8                                # длина массива
tree = [0] * (2 * n)
i = 0                                           # лист: tree[n + i]

v, l, r = 1, 0, n - 1                         # вершина и её отрезок
m = (l + r) // 2                               # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
    variables: [
      { name: "n", role: "длина исходного массива; лист

```

```

        { name: "g", role: "дедушка узла x – нужен в случ
    ],
},
"sparse-table": {
    vizTitle: "Разрезанная таблица (1D и 2D)",
    stepNote: "Шаг = одна ячейка  $st[i][j] = \min(st[i][j - 1], st[i + (1 \ll (j - 1))][j])$ ",
    stepDriven: true,
    code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
i, j = 0, 0
st = [[v] + [None] * (LOG - 1) for v in a] # пустые кл

for j in range(1, LOG):
    for i in range(n - (1 << j) + 1):
        st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j])
    stepCodeLines: [4, 8],
    variables: [
        { name: "n", role: "длина массива / сторона квадрата"},
        { name: "k", role: "уровень таблицы: ячейка хранит минимальное значение"},
        { name: "i", role: "начало блока в строке уровня"},
        { name: "j", role: "номер уровня при построении (1-based)"},
        { name: "r, c", role: "строка и столбец ячейки в таблице"},
        { name: "kx, ky", role: "уровни по вертикали и горизонтали"},
    ],
},
"prefix-sums-2d": {
    vizTitle: "Префиксные суммы (1D и 2D)",
    stepNote: "Шаг = заполнение очередного  $P[i] = P[i - 1] + a[i - 1]$ ",
    stepDriven: true,
    code: `a = [3, 1, 4, 1, 5, 9, 2, 6]
i = 0

P = [0]
for i in range(1, len(a) + 1):
    P.append(P[i - 1] + a[i - 1])
print("P =", P)`
    stepCodeLines: [4, 6],
    variables: [

```



```

code: `a = [7, 3, 5, 1]                                # куча-массивом
n = len(a)
i = n - 1                                              # всплываемый элемент

parent = (i - 1) // 2                                # родитель
kids = (2 * i + 1, 2 * i + 2)                        # дети
print(f"i={i} parent={parent} kids={kids}")`,
variables: [
    { name: "n", role: "текущее число элементов в куче" },
    { name: "i", role: "индекс элемента, который сейчас всплывает" },
    { name: "(i-1)//2", role: "индекс родителя вершины" },
    { name: "2*i+1, 2*i+2", role: "индексы левого и правого ребенка" },
],
},
"stack-dfs": {
    vizTitle: "Стек и обход в глубину",
    stepNote: "Шаг = push новой вершины на стек или pop",
    code: `st = []                                     # стек = рекурсия
st.append("A")                                         # push — зашли в вершину
st.append("B")
top = st.pop()                                         # pop — возврат на уровень
print("pop →", top, "стек:", st)`,
variables: [
    { name: "top", role: "вершина стека — то, откуда мы вышли" },
    { name: "v", role: "текущая вершина графа, которую мы рассматриваем" },
],
},
"queue-bfs": {
    vizTitle: "Очередь и обход в ширину",
    stepDriven: true,
    stepNote: "Шаг = dequeue вершины из головы очереди",
    code: `from collections import deque

q = deque(["A"])
q.append("B")                                          # enqueue соседа
v = q.popleft()                                       # шаг: обрабатываем голову
print("v =", v, "очередь:", list(q))`,
variables: [

```

```

        { name: "order", role: "порядок, в котором обход",
      },
    ],
  },
  "graph-components": {
    // отдельного виджета нет, тема опирается на DFS/BFS
    variables: [],
  },
  "top-sort": {
    vizTitle: "Топологическая сортировка",
    stepDriven: true,
    stepNote: "Шаг = выход рекурсии из вершины v: она дана",
    code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2], 4: []}
used, order = set(), []

def dfs(v):
    used.add(v)
    for to in adj[v]:
        # шаг: ребро v → to
        if to not in used:
            dfs(to)
    order.append(v)
    # выход из v!

for v in adj:
    if v not in used:
        dfs(v)
print(order[::-1])
# разворот – ответ`,
    variables: [
      { name: "v", role: "текущая вершина DFS", range: 0..4 },
      { name: "to", role: "вершина, куда ведёт ребро из v", range: 0..4 },
      { name: "i", role: "перебор стартовых вершин во входе", range: 0..4 },
      { name: "order", role: "список выхода из рекурсии", range: 0..4 },
    ],
  },
  "scc-kosaraju": {
    vizTitle: "Компоненты сильной связности (Косарайю)",
    stepDriven: true,
    stepNote: "Шаг = вершина из order (в обратном порядке)",
    code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [], 4: []}
adjT = {v: [] for v in adj}
    `,
  },
}

```

```

    },
    "bridges-code": {
        vizTitle: "Мосты: DFS + tin/low",
        stepDriven: true,
        stepNote: "Шаг = строка псевдокода слева; ребро (v,
        code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1
timer = 3                                # как на демо

v, to = "B", "G"                        # шаг: возврат из G
is_bridge = low[to] > tin[v]             # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
        variables: [
            { name: "v", role: "текущая вершина DFS", range:
            { name: "to", role: "сосед; ребро (v, to) проверя
            { name: "tin[v]", role: "время входа в вершину –
            { name: "low[v]", role: "минимум tin, куда можно
            { name: "timer", role: "глобальный счётчик времени
        ],
    },
    "euler-path-vs-cycle": {
        vizTitle: "Эйлеров путь и цикл",
        stepNote: "Шаг = один клик по следующей вершине мар
        code: `edges = [(0,1), (0,2), (1,3), (2,4), (1,2),
deg = {}
for u, v in edges:                      # степени вершин
    deg[u] = deg.get(u, 0) + 1
    deg[v] = deg.get(v, 0) + 1

odd = [v for v in deg if deg[v] % 2]
print(deg, "нечётных:", len(odd))
# 0 → цикл • 2 → путь • иначе – нельзя`,
        variables: [
            { name: "path", role: "текущий маршрут – последов
            { name: "last", role: "последняя вершина пути – о
            { name: "deg(v)", role: "степень вершины: чётная
        ],
    },
    "planarity-euler-formula": {

```

```

        heapq.heappush(pq, (dist[v], v))
print("итог:", dist)\,
    stepCodeLines: [4, 13, 17, 19, 22],
    variables: [
        { name: "u", role: "вершина с минимальным dist среди",
        { name: "v", role: "сосед вершины u, до которого про",
        { name: "w", role: "вес ребра (u, v)", range: "по",
        { name: "dist[v]", role: "лучшее известное расстояние"},
    ],
},
"bellman-ford": {
    vizTitle: "Форд–Беллман",
    stepDriven: true,
    stepNote: "Шаг = одна релаксация ребра (u, v, w) внутри",
    code: `n, m = 7, 10                                # вершин, рёбер
dist = {"S": 0}

for i in range(1, n):                                # итерация i = 1 ... n-1
    u, v, w = "S", "A", 5                            # шаг: очередное ребро
    if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
        dist[v] = dist[u] + w                        # релаксация
    if i == 1:
        print(f"итерация {i}: {dist}")
        break`,
    variables: [
        { name: "n", role: "число вершин графа – ровно n-1",
        { name: "m", role: "число рёбер, по которым проходят",
        { name: "i", role: "номер итерации (после i итераций)",
        { name: "u, v, w", role: "текущее ребро: откуда, куда, вес",
        { name: "dist[v]", role: "расстояние; если обновилось, то новое"},
    ],
},
floyd: {
    vizTitle: "Флойд–Уоршелл",
    stepDriven: true,
    stepNote: "Шаг = инициализация матрицы, смена промежуточных",
    code: `INF = 999

```

```

    ],
},
"mst-kruskal": {
    vizTitle: "Краскал и DSU",
    stepNote: "Шаг = следующее ребро из отсортированных",
    code: `edges = [(3, "B", "C"), (1, "A", "B"), (2, "A", "C"),
edges.sort()                # по весу

parent = {v: v for v in "ABC"}    # DSU
def find(x):
    while parent[x] != x:
        x = parent[x]
    return x

mst = []
for w, u, v in edges:            # шаг: очередное ребро
    if find(u) != find(v):      # разные компоненты → берём
        parent[find(u)] = find(v)
        mst.append((w, u, v))
print(mst)\`,
    variables: [
        { name: "n", role: "число вершин; в остов войдёт 1 вершина"},
        { name: "m", role: "число рёбер, сортируем по весу"},
        { name: "i", role: "ребро сортируем по весу w; ра"},
        { name: "parent[v]", role: "DSU: кто представитель"},
        { name: "rank[v]", role: "высота дерева DSU – по 1"},
    ],
},
"mst-prim": {
    vizTitle: "Прим",
    stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
    code: `import heapq

start = "A"
heap = [(0, start, "start")]    # (вес, вершина, откуда)

w, v, parent = heapq.heappop(heap) # шаг: берём минимум
taken = {start}                # дерево растёт от start

```

```

    if s[i] == s[j]:
        j += 1
    pi[i] = j
print(pi)`,
    variables: [
        { name: "n", role: "длина строки s, для которой с"},
        { name: "i", role: "индекс текущего символа – счи"},
        { name: "j", role: "длина текущего наилучшего сов"},
        { name: "π[i]", role: "префикс-функция: длина наи"},
        { name: "ица" },
    ],
},
"string-z-func": {
    vizTitle: "Z-функция",
    stepDriven: true,
    stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]",
    code: `s = "abacaba"

n = len(s)
i, l, r = 1, 0, 0          # правый Z-блок [l, r]
z = [None] * n           # пустые клетки: значения p
z[0] = 0

for i in range(1, n):     # шаг: вычисляем z[i]
    if i <= r:
        z[i] = min(r - i + 1, z[i - l]) # из блока
    else:
        z[i] = 0           # свежая клетка: с нуля
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1       # досчёт в лоб
        if i + z[i] - 1 > r:
            l, r = i, i + z[i] - 1
print(z)`,
    variables: [
        { name: "n", role: "длина строки s", range: "n = 1..7"},
        { name: "i", role: "позиция, для которой считаем z[i]"},
        { name: "l, r", role: "правый крайний Z-блок: отрезок [l, r]"},
        { name: "z[i]", role: "длина наибольшего общего префикса s[0..i] и s[i..n-1]"},
    ],
},

```

```

    for c in range(8):
        st2[(r, c, 0, 0)] = A[r][c]
for k in range(1, 4):
    print(f"уровень k={k}: блоки {1 << k}x{1 << k}")
    size = 9 - (1 << k)
    half = 1 << (k - 1)
    for r in range(size):
        for c in range(size):
            st2[(r, c, k, k)] = min(st2[(r, c, k - 1, k - 1),
            + half, c + half, k - 1, k - 1]))`,
    stepCodeLines: [1, 8],
    variables: [
        { name: "r, c", role: "левый верхний угол блока, ",
        { name: "k", role: "уровень: блоки 2^k x 2^k (шаг",
        { name: "st2[(r,c,k,k)]", role: "минимум квадратн
    ],
},
"sparse-table#2d-query": {
    vizTitle: "Разрезанная таблица 2D: запрос",
    stepNote: "Демонстрация кликабельна вручную: выбери
    code: `# st2[(r, c, kx, ky)] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
    variables: [
        { name: "r1, c1, r2, c2", role: "углы запрашиваем
        { name: "kx, ky", role: "уровень самого крупного
    ],
},
"prefix-sums-2d#2d": {
    vizTitle: "Префиксные суммы 2D (прямоугольники)",
    stepNote: "Демонстрация кликабельна вручную: наведи
    code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0

```

```

    if (!sync.code) return head;
    return `${head}\n${sync.code}\n`;
}

/**
 * Инициализация скелета: только объявление демо-данных
 * (без for/while/def/if/print). Именно это показывается
 * умолчанию – дальше пользователь пишет свой вариант,
 * подсматривает через кнопку-«глаз».
 */
function extractInit(code: string): string {
    const keep: string[] = [];
    for (const line of code.split("\n")) {
        const t = line.trim();
        if (/^(for|while|def|class|if\s|elif\s|else|try|exc
        keep.push(line);
    }
    // убрать пустые строки с конца
    while (keep.length > 0 && keep[keep.length - 1].trim()
    return keep.join("\n");
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО иници
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, ch
    const sync = getPageSync(chapterId, demoId);

    const head = sync.vizTitle
        ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#
          опкой-«глазом»\n`
        : `# «${chapterTitle}»\n# демо на странице нет – св

    if (!sync.code) return head;
    const init = extractInit(sync.code);
    return `${head}\n${init}\n`;

```



```

<div class="grid grid-cols-1 xl:grid-cols-2
  <div class="bg-slate-800 p-6 rounded-lg
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Корпоративная пре
    </div>
    <p class="text-slate-300 text-sm mb
ы рассылать 10 000 писем, он пишет одно пис
счетах сотрудников только тогда, когда сотр
женное обновление).</p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg
    <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
      Аналогия 2: Матрешки-коробки
    </div>
    <p class="text-slate-300 text-sm mb
ше, и так далее. Если нам нужно покрасить п
Внутри всё синее!" на большую коробку. Опус
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
      Строгое доказательство / Код (Скр
    </summary>
    <div class="p-5 text-sm text-slate-300
      <pre class="bg-slate-950 p-4 rounder
function push(node) {
  if (lazy[node] !== 0) {
    lazy[2 * node] += lazy[node];
    tree[2 * node] += lazy[node];
    lazy[2 * node + 1] += lazy[node];
    tree[2 * node + 1] += lazy[node];
    lazy[node] = 0;
  }
}

```

```

        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
                Аналогия 2: Ступени
            </div>
            <p class="text-slate-300 text-sm mb-4">
                нужно покрыть отрезок, мы берем две самые бо
            </div>
        </div>
        <details class="bg-slate-900/40 rounded-lg">
            <summary class="p-4 font-bold text-slate-300 border-slate-700/50">
                Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300">
                <pre class="bg-slate-950 p-4 rounded">
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
            </div>
        </details>
    </div>
</div>
</section>`
    },
    {
        id: "treap",
        title: "3. Декартово дерево (Treap)",
        type: "html",
        description: "Дерево поиска по ключу и Куча по приоритету",
        category: "Продвинутые структуры",
        content: `
<section id="treap" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
                </div>
    </div>
    <div class="space-y-8">

```



```

        <div class="space-y-4 text-sm text-slate-500">
          <div>
            <strong class="text-white block">
корень?</strong>
            <p>Оно поднимет узел, на котором
ска. За счет сдвига этого листа в корень, д
/p>
          </div>

          <div>
            <strong class="text-white block">
            <p>Если агент постоянно переключ
находящимися на разных концах дерева, его
ащая дерево в вытянутый бамбук для другого.
. Постоянное свинг-переключение превратит пр
            </div>

          <div>
            <strong class="text-white block">
            <p>Хотя один поиск может стоять
емах обладают прекрасным свойством: они не
по пути. "Палочное" дерево прессуется в сб
осов.</p>
          </div>
        </div>
      </div>
    </div>
  </section>`
  },
  {
    id: "prefix-sums-2d",
    title: "5. Двумерная матрица префиксных сумм",
    type: "html",
    description: "Сжатие суммы подматрицы за  $O(1)$ ",
    category: "Алгоритмы матрицы",
    content: `

```

```

export const tickets14to20: Chapter[] = [
  {
    id: "dijkstra",
    title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">14
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Графы. Поиск кратчайшего пути. Дейкстра
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Позитивное планирование
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Дейкстра — это алгоритм поиска кратчайшего пути в графе с неотрицательными весами. Он работает так:
        <p class="text-xl font-mono text-white">1. Выбираем стартовую точку (источник) и помечаем ее как «обработанную».
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Позитивное планирование
          </div>
          <p class="text-slate-300 text-sm mb-4">Дейкстра — это алгоритм поиска кратчайшего пути в графе с неотрицательными весами. Он работает так:
          (нельзя поехать и заработать). Он всегда выбирает ближайшую точку, которую еще не обработал.
        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Ограничения
          </div>
          <p class="text-slate-300 text-sm mb-4">Дейкстра — это алгоритм поиска кратчайшего пути в графе с неотрицательными весами. Он работает так:
          дный и не умеет пересматривать уже "зафиксированные" узлы.
        </div>
      </div>
    </div>
  </div>
`
  }
]

```

```

        <p class="text-slate-300 text-sm mb-4">
опали во временную петлю.</p>
      </div>
    </div>
    <div id="slot-bellman-viz" class="mt-8"></div>
  </div>
</div>
</section>`
},
{
  id: "floyd",
  title: "16. Графы. Поиск кратчайшего пути. Флойд",
  type: "html",
  category: "Графы. Пути",
  content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500 shadow-md">
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>

    <div class="bg-slate-800 p-6 rounded-lg border-emerald-500 shadow-md">
      <div class="absolute -top-3 left-4 bg-slate-900
border-emerald-500 shadow-md">
        Суть фокуса (По шагам)
      </div>
      <p class="text-slate-300 text-sm mb-4">
        Главный герой – <b>внешний цикл</b>
        шаг <code class="bg-slate-900 text-pink-400">k?</code>
        шу себе проходить через вершину k?</i>
      </p>
    </div>
  </div>
</section>
`

```

```

<p class="text-slate-300 text-sm mb-6">
  Дейкстра (Билет 14) быстрая, но работает очень медленно. Мы хотим сделать Джонсона по пути остались теми же.
</p>
<ol class="text-sm text-slate-300 space-y-4">
  <li>Добавляем фиктивную вершину
  <li>Запускаем от неё медленно «потенциалы» <code class="text-pink-400">
  <li>Меняем старые веса: новое р
  <li><b>Магия!</b> Все веса тепер
  <li>Теперь смело запускаем б
</li>
</ol>
</div>
</div>
<div id="slot-johnson-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
  id: "mst-kruskal",
  title: "18. Графы. Остовное дерево. Краскал",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">

```

```

<div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
    * Аналогия: Заражение вирусом
</div>
<p class="text-slate-300 text-sm mb
ли вирус). Мы стартуем из одного города, и
Сеть растет только из одного связного куска
</div>
</div>
</div>
</section>`
},
{
  id: "mst-boruvka",
  title: "20. Графы. Остовное дерево. Борувка",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-emerald-4
      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-lg text-emerald-300 ital
        <p class="text-xl font-mono text-white"
ается с соседом.</p>
      </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия: Племена
        </div>

```



```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2
```

```
  <!-- Аналогия 1 -->
```

```
  <div class="bg-slate-800 p-6 rounded-lg
```

```
    <div class="absolute -top-3 left-4 l  
  rder border-emerald-500 shadow-md">
```

```
    ♂ Аналогия 1: Поиск без отка
```

```
  </div>
```

```
  <p class="text-slate-300 text-sm mb
```

```
    Мы идём по строке слева направо. К  
строки СЕЙЧАС закончился под моим пальцем?»
```

```
    Представь, ты ищешь в тексте слов  
х</code>. Тупой алгоритм начнет искать зано  
</code>, а это начало нашего слова! Не надо
```

```
  </p>
```

```
</div>
```

```
  <!-- Аналогия 2 -->
```

```
  <div class="bg-slate-900 p-6 rounded-lg
```

```
    <div class="absolute -top-3 left-4 l  
border-rose-500 shadow-md">
```

```
    Аналогия 2: LLM стриминг
```

```
  </div>
```

```
  <p class="text-slate-300 text-sm mb
```

```
    Для LLM, которая стримит токены  
каждом шаге. Если мы частично совпали с зап  
акого места этого слова мы можем продолжить
```

```
  </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
```

```
  <summary class="p-4 font-bold text-slate  
-b border-slate-700/50">
```

```
    Пример вычисления (Скрыто)
```

```
  </summary>
```

```
  <div class="p-5 text-sm text-slate-300
```



```

    if (i &lt;= r) z[i] = min(r - i + 1, z[i - 1]);
    while (i + z[i] &lt; n && s[z[i]] == s[i + z[i]]) z
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}

```

```

    </div>

```

```

    </details>

```

```

    </div>

```

```

    </div>

```

```

</section>`

```

```

},

```

```

{

```

```

    id: "aho-corasick",

```

```

    title: "23. Алгоритм Ахо-Корасик",

```

```

    type: "html",

```

```

    category: "Строки",

```

```

    content: `

```

```

<section id="aho-corasick" class="mb-12 scroll-mt-10">

```

```

    <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

        <span class="bg-indigo-600 text-white px-4 py-1 rounded">

```

```

        <h2 class="text-2xl sm:text-3xl font-bold text-white">

```

```

    </div>

```

```

<div class="space-y-8">

```

```

    <div class="bg-slate-700/50 p-6 rounded-xl border-l">

```

```

        <h3 class="text-xl font-bold text-blue-400 mb-4">

```

```

        <div class="bg-slate-900 p-4 rounded-lg mb-6 text">

```

```

            <p class="text-sm text-blue-300 italic mb-2">Осн

```

```

            <p class="text-lg font-mono text-white">Бор (Tr

```

```

        </div>

```

```

        <p class="text-slate-300 text-sm">Позволяет найти
        го один проход.</p>

```

```

    </div>

```

```

<!-- Аналогии -->

```

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">

```

```

    <div class="bg-slate-800 p-6 rounded-lg border bo

```

```

        <div class="absolute -top-3 left-4 bg-slate-700

```

```

} </pre>
    </div>
  </details>
</div>
</section>`
  },
  {
    id: "complexity-classes",
    title: "24. Классы сложности, сведение задач",
    type: "html",
    category: "Теория сложности",
    content: `
<section id="complexity-classes" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-purple-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">
        <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 l
            order border-emerald-500 shadow-md">
              Аналогия 1: Судoku (P vs NP)
            </div>
            <p class="text-slate-300 text-sm mb-4">
пример сортировка чисел). Класс <b>NP</b> –
енную сетку (ответ), он БЫСТРО (за класс P)
          </div>
          <!-- Аналогия 2 -->
          <div class="bg-slate-900 p-6 rounded-lg

```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md">
  <h3 class="text-xl font-bold text-slate-300">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-slate-300 italic">
        <p class="text-xl font-mono text-white">
          (V + E).</p>
        </p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Матрица = Таблица
          </div>
          <p class="text-slate-300 text-sm mb-4">
            глупо, но проверка "знает ли Вася Петю" мгно
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
              Аналогия: Списки = Контакты
            </div>
            <p class="text-slate-300 text-sm mb-4">
              т. Оптимально для почти всех задач.</p>
            </div>
          </div>
        </div>
      </div>
    </h3>
  </div>
</div>
```

```
{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md">
  <h3 class="text-xl font-bold text-indigo-400">

  <div class="grid grid-cols-1 xl:grid-cols-2">
    <!-- Аналогия DFS -->
    <div class="bg-slate-800 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 border border-indigo-500 shadow-md">
        ♂ DFS (Поиск в глубину) = Ж
      </div>
```

```

        </summary>
        <div class="grid grid-cols-1 md:grid-co
            <pre class="bg-slate-950 p-3 rounde
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
    vis[v] = true;
    for (int u : adj[v]) {
        if (!vis[u]) dfs(u);
    }
}
}
}

        <pre class="bg-slate-950 p-3 rounde
// BFS (Очередь)
queue<int> q;
q.push(start_node);
vis[start_node] = true;

while(!q.empty()) {
    int v = q.front(); q.pop();
    for (int u : adj[v]) {
        if (!vis[u]) {
            vis[u] = true;
            q.push(u);
        }
    }
}
}

        </div>
        </details>
    </div>
</div>
</section>`,
    },
    {
        id: "graph-planar-colors",
        title: "7. Графы. Планарные. Покраска",
        type: "html",
    }

```

```

<section id="graph-components" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Графы
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-emerald-400">Графы
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Графы – это абстрактная модель, состоящая из
        <p class="text-xl font-mono text-white">узлов и ребер. Узлы представляют объекты, а ребра – отношения между
        ество компонент.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Острова
          </div>
          <p class="text-slate-300 text-sm mb-4">Если представить карту – остались ли серые (неисследованные)
          ь через океан – столько и компонент.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы</span>

```





```

        <div class="bg-slate-900 p-6 rounded-lg"
          <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
            Связь с Билетом 12
          </div>
          <p class="text-slate-300 text-sm mb"
            <b>SCC (Билет 10)</b> склеивает
        </br><br>
            <b>Точки сочленения (Билет 12)</b>
        ость</strong> монолита. Вырвешь точку сочле
          </p>
        </div>
      </div>

      <details class="bg-slate-900/40 rounded-lg"
        <summary class="p-4 font-bold text-slate-
        -b border-slate-700/50">
          Душный мод: Код Алгоритм Косарайю
        </summary>
        <div class="p-5 text-sm text-slate-300"
          <p class="text-emerald-400 font-bol"
          <ol class="list-decimal list-inside"
            <li>Запускаем DFS на исходном графе
          </li>
            <li>Разворачиваем этот список (только вершины)
            <li>Переворачиваем все ребра в обратном направлении
            <li>Идем по <code>order</code>:
          </li>
        </ol>
        </div>
      </details>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-bridges",
    title: "11. Графы. Мосты",

```

```
-----
<section id="graph-articulation" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-rose-400 m

      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-lg text-rose-300 italic m
        <div class="text-left font-mono text-sm
          <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
        <div class="p-3 bg-slate-950 rounde
          <span class="text-rose-400 font
          <p class="text-xl font-bold tex
          <p class="text-xs text-slate-40
        </div>
      </div>
    </div>
    <div class="p-6 rounded-xl border-rose-500 shadow-md">
      <p class="text-slate-300 text-sm">
        <strong>Важно:</strong> Это работает то
еются к точкам сочленения. То есть, <strong>
ег - это тупик со степенью 1).
      </p>
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2 gap
      <div class="bg-slate-800 p-6 rounded-lg bord
        <div class="absolute -top-3 left-4 bg-s
        <div class="text-rose-500 shadow-md">
          Аналогия: Главный перекресток
        </div>
        <p class="text-slate-300 text-sm mb-4">
          Представь город, где два района сое
```



-----  
=====

ФАЙЛ 30/87: src/components/ArticulationPointsViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

-----

```
import React, { useState, useEffect } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import { Play, Pause, RotateCcw, Info } from "lucide-react";
```

```
interface Node {
  id: number;
  x: number;
  y: number;
  label: string;
}
```

```
interface Edge {
  u: number;
  v: number;
}
```

```
const nodes: Node[] = [
  { id: 0, x: 250, y: 50, label: "0" },
  { id: 1, x: 100, y: 150, label: "1" },
  { id: 2, x: 250, y: 250, label: "2" },
  { id: 3, x: 400, y: 150, label: "3" },
  { id: 4, x: 550, y: 150, label: "4" },
  { id: 5, x: 700, y: 50, label: "5" },
  { id: 6, x: 700, y: 250, label: "6" },
];
```

```
const edges: Edge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 },
```

```
const recordStep = (desc: string, u: number | null) => {
  newSteps.push({
    desc,
    visited: new Set(visited),
    ap: new Set(ap),
    currentNode: u,
    tin: [...currentTin],
    low: [...currentLow],
  });
};
```

recordStep("Начало DFS (Тарьян) для поиска точек со

```
const dfs = (v: number, p: number = -1) => {
  visited.add(v);
  currentTin[v] = currentLow[v] = timer++;
  let children = 0;

  recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}`);

  for (const to of adj[v]) {
    if (to === p) continue;

    if (visited.has(to)) {
      currentLow[v] = Math.min(currentLow[v], currentLow[to]);
      recordStep(
        `Обратное ребро ${v}-${to}. Обновляем low[${v}]`,
        v,
      );
    } else {
      children++;
      dfs(to, v);
      currentLow[v] = Math.min(currentLow[v], currentLow[to]);

      recordStep(
        `Возврат в ${v} из ${to}. Обновляем low[${v}]`,
        v,
      );
    }
  }
};
```

```

    visited: new Set(),
    ap: new Set(),
    currentNode: null,
    tin: [],
    low: [],
  };

  useEffect(() => {
    let interval: NodeJS.Timeout;
    if (isPlaying && currentStepIndex < steps.length - 1) {
      interval = setInterval(() => {
        setCurrentStepIndex((prev) => prev + 1);
      }, 1500);
    } else if (currentStepIndex >= steps.length - 1) {
      setIsPlaying(false);
    }
    return () => clearInterval(interval);
  }, [isPlaying, currentStepIndex, steps.length]);

  const togglePlay = () => setIsPlaying(!isPlaying);
  const reset = () => {
    setIsPlaying(false);
    setCurrentStepIndex(0);
  };

  return (
    <div className="bg-slate-900 rounded-xl border border-
      <div className="bg-slate-800 p-4 border-b border-
        <h3 className="font-bold text-white flex items-
          <Info className="w-5 h-5 text-rose-400" />
          Моделирование поиска точек сочленения
        </h3>

        <div className="flex items-center gap-2 bg-slate-
          <button
            onClick={togglePlay}
            className="flex items-center gap-2 px-3 py-
            text-white"

```

```

        stroke={isBridge ? "#f43f5e" : "#475568"}
        strokeWidth={isBridge ? "4" : "2"}
        strokeDasharray={isBridge ? "8,4" : "0"}
      />
    );
  }}}

```

```

{/* Nodes */}
{nodes.map((node) => {
  const isCurrent = currentStep.currentNode === node.id;
  const isVisited = currentStep.visited.has(node.id);
  const isAp = currentStep.ap.has(node.id);

```

```

    const fill = isCurrent
      ? "#3b82f6"
      : isAp
        ? "#e11d48"
        : isVisited
          ? "#10b981"
          : "#1e293b";
    const stroke = isCurrent
      ? "#60a5fa"
      : isAp
        ? "#f43f5e"
        : isVisited
          ? "#34d399"
          : "#334155";
    const r = isAp ? 24 : 20;

```

```

    return (
      <g key={node.id}>
        <circle
          cx={node.x}
          cy={node.y}
          r={r}
          fill={fill}
          stroke={stroke}
          strokeWidth="3"

```

```

                ? currentStep.low[node.id]
                : "-"}
            </text>
        </>
    })
  </g>
);
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
  <h4 className="text-sm font-bold text-slate-4
    Статус алгоритма
  </h4>

  <div className="bg-slate-900 border border-sl
    <p className="text-white font-medium min-h-
      {currentStep.desc}
    </p>
  </div>

  <div className="space-y-4 flex-1 overflow-y-a
    <div className="mb-4">
      <h5 className="text-xs text-slate-500 fon
      <div className="flex items-center gap-2 t
        <div className="w-3 h-3 rounded-full bg
          Точка сочленения
        </div>
      <div className="flex items-center gap-2 t
        <div className="w-3 h-3 rounded-full bg
          Посещена
        </div>
      <div className="flex items-center gap-2 t
        <div className="w-3 h-3 rounded-full bg
          Текущая
        </div>
      <div className="flex items-center gap-2 t

```



```
    });  
  };
```

---

ФАЙЛ 31/87: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';  
import { useVizStepSync } from '../data/vizStepBus';  
  
interface Node { id: string; x: number; y: number; name: string; }  
interface Edge { u: string; v: string; w: number; }  
interface Step {  
  iteration: number;  
  checkingEdge: { u: string; v: string; w: number } | null;  
  distances: { [key: string]: number };  
  previous: { [key: string]: string | null };  
  log: string;  
  hasNegativeCycle?: boolean;  
  activeCodeLine: number;  
}  
  
export default function BellmanFordViz() {  
  const [hasCycle, setHasCycle] = useState(false);  
  const [steps, setSteps] = useState<Step[]>([]);  
  const [currentStepIndex, setCurrentStepIndex] = useState(0);  
  useVizStepSync(currentStepIndex, setCurrentStepIndex, setSteps);  
  const [isPlaying, setIsPlaying] = useState(false);  
  
  const nodes: Node[] = [  
    { id: 'S', x: 60, y: 150, name: 'База (S)' },  
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },  
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },  
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },  
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },  
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },  
  ];
```

```

    { line: 7, text: '      for (u, v, weight) in E: // Пройдем по всем ребрам
    { line: 8, text: '          if dist[u] + weight < dist[v]:
    { line: 9, text: '              return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"
    { line: 10, text: '      return dist' },
  ];

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];
  let dist: { [key: string]: number } = { S: 0, A: 999 };
  let prev: { [key: string]: string | null } = { S: null };

```

```

  simulationSteps.push({
    iteration: 0, checkingEdge: null,
    distances: { ...dist }, previous: { ...prev },
    log: '    Фура заведена. Начинаем с Базы (S). Расстояние до А: 999
    activeCodeLine: 2,
  });

```

```

  const vCount = nodes.length;
  for (let iter = 1; iter < vCount; iter++) {
    let anyUpdate = false;
    for (const edge of edges) {
      const uDist = dist[edge.u];
      const vDist = dist[edge.v];

      let updated = false;
      if (uDist !== 999 && uDist + edge.w < vDist) {
        dist = { ...dist, [edge.v]: uDist + edge.w };
        prev = { ...prev, [edge.v]: edge.u };
        updated = true;
        anyUpdate = true;
      }

```

```

    simulationSteps.push({
      iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
      distances: { ...dist }, previous: { ...prev },
      log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
        updated ? ` Релаксация успешна! dist[${edge.v}] = ${dist[edge.v]}

```

```

    }

    setSteps(simulationSteps);
    setCurrentStepIndex(0);
    setIsPlaying(false);
  }, [hasCycle]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Зарядка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border" >
    <div className="flex flex-wrap items-center justify-between" >
      <div className="flex items-center gap-3" >
        <div className="p-2.5 bg-blue-600 text-white" >
          <span className="text-2xl" > </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white" >
            <p className="text-sm text-blue-300" >Полный
          </p>
        </div>
      </div>

      <div className="flex items-center gap-3" >
        <div className="bg-slate-900 p-1 rounded-lg border" >
          <button onClick={() => setHasCycle(false)} >
            'bg-blue-600 text-white' : 'text-slate-400'
          <button onClick={() => setHasCycle(true)} >

```

```

    {edges.map((edge, idx) => {
      const uNode = nodes.find((n) => n.id === u);
      const vNode = nodes.find((n) => n.id === v);
      const isChecking = currentStep.checkingEdge === idx;
      const isNeg = edge.w < 0;
      const isCyclePart = hasCycle && ((edge.u === u && edge.v === 'D') || (edge.v === v && edge.u === 'D'));

      let strokeColor = '#475569';
      let marker = 'url(#arrow-bf-normal)';
      if (isChecking) { strokeColor = '#f59e0b'; }
      else if (currentStep.hasNegativeCycle && isCyclePart) { strokeColor = '#f43f5e'; }

      return (
        <g key={idx}>
          <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} strokeWidth={
            isChecking && isCyclePart ? 4 : 2} marker={isChecking ? 'url(#arrow-bf-normal)' : 'none'} />
          <circle cx={({uNode.x + vNode.x} / 2) cy={({uNode.y + vNode.y} / 2)} stroke={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#475569'} strokeDash={isChecking ? [4, 4] : []} />
          <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2) + 10} text={isChecking ? 'Checking' : isNeg ? 'Neg' : ''} font-size="11" font-weight="bold" fill="white" />
        </g>
      );
    })
  );
}

{nodes.map((node) => {
  const dist = currentStep.distances[node.id];
  const isCheckingNode = currentStep.checkingNode === node.id;

  return (
    <g key={node.id} className="transition-all duration-300">
      <circle cx={node.x} cy={node.y} r={isCheckingNode ? 10 : 5} fill={isCheckingNode ? '#7dd3fc' : '#64748b'} stroke="black" />
      <text x={node.x} y={node.y + 5} fill="white" font-size="10" font-weight="bold">{node.id}</text>
      <text x={node.x} y={node.y - 28} fill="white" font-size="10" font-weight="bold">{dist}</text>
    </g>
  );
})
}

```

```

    { /* Таблица расстояний */ }
    <div className="w-full lg:w-1/2 bg-rose-950/10">
      <div>
        <h4 className="text-lg font-bold text-rose-950">
          <div className="grid grid-cols-4 gap-2 text-rose-950">
            {nodes.map(n => {
              const d = currentStep.distances[n.id];
              const isTgt = currentStep.checkingEdge === n.id;
              return (
                <div key={n.id} className={`p-2 rounded-lg border-slate-700/60 bg-slate-800/40`} >
                  <div className="text-xs text-slate-400">
                    <div className={`font-mono font-bold`} >
                      {n.id}
                    </div>
                    <div>
                      {d}
                    </div>
                  </div>
                </div>
              );
            })}
          </div>
        </div>
      </div>
    </div>

    { /* Код алгоритма */ }
    <div className="w-full lg:w-1/2 bg-slate-900/60">
      <div>
        <h4 className="text-lg font-bold text-slate-400">
          <div className="bg-slate-950/80 rounded-lg p-4">
            {codeLines.map(item => (
              <div key={item.line} className={`py-1.5 px-4`} >
                {item.line === currentStep.start ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'}
                <span className="text-slate-600 w-5 shrink-0">{item.line}</span>
                <span className="whitespace-pre flex-grow-1">{item.code}</span>
              </div>
            ))}
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

```

```
    logs: string;
}

const generateBfsSteps = (): Step[] => {
    const steps: Step[] = [];
    const queue: number[] = [];
    const visited = new Set<number>();

    // Init
    steps.push({
        activeLine: 1,
        queue: [],
        visited: [],
        currentNode: null,
        logs: "Инициализация: начинаем с корня (0).",
    });

    queue.push(0);
    visited.add(0);
    steps.push({
        activeLine: 2,
        queue: [...queue],
        visited: Array.from(visited),
        currentNode: null,
        logs: "Добавляем стартовую вершину 0 в очередь и по",
    });

    while (queue.length > 0) {
        steps.push({
            activeLine: 4,
            queue: [...queue],
            visited: Array.from(visited),
            currentNode: null,
            logs: `Очередь не пуста, элементов: ${queue.length}`,
        });

        const curr = queue.shift()!;
        steps.push({
```

```

        });
    }
}

steps.push({
  activeLine: 12,
  queue: [],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
  const [stepIdx, setStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);
  useVizStepSync(stepIdx, setStepIdx, STEPS.length - 1)

  const step = STEPS[stepIdx];

  const handleNext = useCallback(() => {
    setStepIdx(prev => Math.min(prev + 1, STEPS.length
  }, []));

  // @ts-expect-error зарезервировано под кнопку «назад»
  const handlePrev = useCallback(() => {
    setStepIdx(prev => Math.max(prev - 1, 0));
  }, []);

  useEffect(() => {
    if (!isPlaying) return;
    if (stepIdx >= STEPS.length - 1) {
      setIsPlaying(false);
      return;
    }
  }, [stepIdx, isPlaying]);
}

```

```

    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-2 gap-4" >
    {/** Визуал графа */}
    <div className="bg-slate-800 border border-slate-600 border-radius-50px]" >
      <svg className="w-full h-full min-h-[350px]" >
        {/** Рёбра */}
        {EDGES.map((e, idx) => {
          const n1 = NODES.find(n => n.id === e.from)
          const n2 = NODES.find(n => n.id === e.to)
          const isVisited = step.visited.includes(n1.id)
          return (
            <line
              key={idx}
              x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
              className={`transition-all duration-500 ${
                isVisited ? 'stroke-slate-600' : 'stroke-blue-400'
              }`}
            />
          )
        })}

        {/** Вершины */}
        {NODES.map(n => {
          const isCurrent = step.currentNode === n.id
          const isInQueue = step.queue.includes(n.id)
          const isVisited = step.visited.includes(n.id)

          let fill = '#1e293b'; // slate-800
          let stroke = '#475569'; // slate-600
          let scale = 1;

          void scale;

          if (isCurrent) {
            fill = '#3b82f6'; // blue-500
            stroke = '#60a5fa'; // blue-400
            scale = 1.3;
          }
        })}
      </svg>
    </div>
  </div>

```



```

<div className="bg-slate-900 border border-slate-800" style={{ padding: 10px }}>
  <div className="bg-slate-800 text-slate-400 p-4" style={{ border: 1px solid #333, border-radius: 5px, margin: 10px 0 10px 10px; font-family: monospace; font-size: 0.9em; line-height: 1.2; white-space: pre-wrap; width: 100%; }}>
    <span>bfs(graph, start)</span>
    <span className="text-blue-400">Mar {step}</span>
  </div>
  <div className="p-4">
    {
      { line: 1, code: 'queue = Queue()' },
      { line: 2, code: 'queue.push(start); visited.add(start)' },
      { line: 3, code: '' },
      { line: 4, code: 'while not queue.isEmpty():' },
      { line: 5, code: '    node = queue.pop()' },
      { line: 6, code: '    for neighbor in graph[node]:' },
      { line: 7, code: '        if neighbor not in visited:' },
      { line: 8, code: '            visited.add(neighbor)' },
      { line: 9, code: '            queue.push(neighbor)' },
    }.map((cl) => {
      const isActive = step.activeLine === cl.line
      return (
        <div key={cl.line} className={`px-2 py-1 border-l-2 border-blue-400 -ml-0.5` : `text-${isActive ? 'blue-400' : 'slate-400'}`} style={{ border-left: 2px solid #333, padding-left: 10px, margin-left: -0.5px, font-family: monospace; font-size: 0.8em; line-height: 1.2; white-space: pre-wrap; width: 100%; }}>
          <span className="opacity-40 w-6 inline-block" style={{ display: inline-block, width: 60px, height: 1em, background-color: #333, vertical-align: middle; margin-right: 5px; }}></span>
          <span className="whitespace-pre">{cl.code}</span>
        </div>
      )
    })
  </div>
</div>

<div className="bg-slate-800/80 border border-slate-700 p-4" style={{ border: 1px solid #333, border-radius: 5px, margin: 10px 0 10px 10px; font-family: monospace; font-size: 0.9em; line-height: 1.2; white-space: pre-wrap; width: 100%; }}>
  <div className="text-sm font-bold text-emerald-400" style={{ margin-bottom: 10px; }}>
    <span>Состояние Очереди (Queue):</span>
    <span className="text-xs text-slate-500" style={{ margin-left: 10px; }}>{step.queue.length}</span>
  </div>
  <div className="flex items-center gap-2" style={{ font-family: monospace; font-size: 0.8em; }}>
    {step.queue.length === 0 ? (

```

```

    },
    'bellman-ford': {
      src: bellmanImg,
      alt: 'Фура по болоту',
      caption: ' Тяжёлая, но ей пофиг на ямы!',
      borderColor: 'border-rose-500/30',
      captionColor: 'text-rose-400',
    },
    floyd: {
      src: floydImg,
      alt: 'Все ко Всем Флойд',
      caption: ' Все связаны со всеми!',
      borderColor: 'border-emerald-500/30',
      captionColor: 'text-emerald-400',
    },
  };

export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;

  return (
    <div>
      <div className={`rounded-2xl overflow-hidden border-2 border-${info.borderColor}`}>
        <img src={info.src} alt={info.alt} className="w-full h-full" />
      </div>
      <p className={`text-center text-xs mt-2 font-bold text-${info.captionColor}`}>
        {info.caption}
      </p>
    </div>
  );
}

```

---

ФАЙЛ 34/87: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React from "react";
```

```

        :bg-slate-700 enabled:hover:text-white disabled:opacity-50
      >
      <span className="hidden sm:inline">Вперёд</span>
      <ChevronRight className="w-4 h-4" />
    </button>
  </div>
);
}

return (
  <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
    {prev ? (
      <button
        onClick={() => onGo(prev.id)}
        className="group text-left p-4 rounded-xl bg-white shadow-sm transition-all"
      >
        <div className="flex items-center gap-1.5 text-indigo-400 mb-1">
          <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая тема
        </div>
        <div className="text-sm font-bold text-slate-700">
          {prev.title}
        </div>
      </button>
    ) : (
      <div className="hidden sm:block" />
    )}

    {next && (
      <button
        onClick={() => onGo(next.id)}
        className="group text-right p-4 rounded-xl bg-white shadow-sm transition-all sm:col-start-2"
      >
        <div className="flex items-center justify-end text-indigo-400 mb-1">
          Следующая тема <ChevronRight className="w-3.5 h-3.5" />
        </div>
        <div className="text-sm font-bold text-slate-700">
          {next.title}
        </div>
      </button>
    )}
  </nav>
);

```

```

const [anchor, setAnchor] = useState<HintAnchor | null>(null);
const hideTimer = useRef<number | null>(null);
const [saving, setSaving] = useState<"idle" | "work">("idle");

useTermHints(contentRef, [chapter.id]);

const cancelHide = useCallback(() => {
  if (hideTimer.current) {
    window.clearTimeout(hideTimer.current);
    hideTimer.current = null;
  }
}, []);

const scheduleHide = useCallback(() => {
  cancelHide();
  hideTimer.current = window.setTimeout(() => setAnchor(anchor), 2500);
}, [cancelHide]);

useEffect(() => {
  setAnchor(null);
  setSaving("idle");
}, [chapter.id]);

const saveHtml = useCallback(async () => {
  setSaving("work");
  try {
    await downloadChapterHtml(chapter);
    setSaving("done");
    window.setTimeout(() => setSaving("idle"), 2500);
  } catch {
    setSaving("idle");
  }
}, [chapter]);

const open = useCallback(
  (el: HTMLElement) => {
    const termId = el.dataset.termId;
    if (!termId) return;
  },
);

```

```

<button
  onClick={saveHtml}
  disabled={saving === "work"}
  title="Сохранить эту тему одним автономным файлом"
  className="flex items-center gap-1.5 text-x-300 hover:text-white hover:border-emerald-500"
>
  {saving === "work" ? (
    <Loader2 className="w-3.5 h-3.5 animate-spin" />
  ) : saving === "done" ? (
    <Check className="w-3.5 h-3.5 text-emerald-500" />
  ) : (
    <Download className="w-3.5 h-3.5" />
  )}
  {saving === "done" ? "Файл сохранён" : "Скачать"}
</button>
<button
  onClick={() => window.print()}
  title="Распечатать или сохранить в PDF средствами браузера"
  className="flex items-center gap-1.5 text-x-300 hover:text-white hover:border-indigo-500"
>
  <Printer className="w-3.5 h-3.5" /> Печать
</button>
<span className="text-[11px] text-slate-500">
  </div>

<div
  ref={contentRef}
  className="prose prose-invert max-w-none"
  onMouseOver={handlePointerOver}
  onMouseOut={handlePointerOut}
  onClick={handleClick}
  onFocus={handleFocus}
  dangerouslySetInnerHTML={{ __html: chapter.content }}
/>

<p className="mt-6 flex items-start gap-2 text-slate-500">

```

```

        <viz.Component />
      </VizChapterContext.Provider>
    </section>
  )}

  <ChapterNav prev={prev} next={next} index={index} />
</article>

<TermPopover
  anchor={anchor}
  onClose={() => setAnchor(null)}
  onOpenSimulator={(id) => {
    setAnchor(null);
    onOpenSimulator(id);
  }}
  onCardEnter={cancelHide}
  onCardLeave={scheduleHide}
/>
</>
);
};

```

---

ФАЙЛ 36/87: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import { Undo, Play, Pause, SkipForward, RefreshCw } from 'react-native-svg';

```

```

interface CodeLine {
  line: string;
  highlight: string;
}

```

```

const CODE_LINES: CodeLine[] = [

```

```

    { id: 3, label: "D", x: 180, y: 300 },
    { id: 4, label: "E", x: 100, y: 370 },
    { id: 5, label: "F", x: 260, y: 370 },
    { id: 6, label: "G", x: 340, y: 200 }
  ];

const GRAPH_EDGES = [
  { u: 0, v: 1, key: "0-1" },
  { u: 1, v: 2, key: "1-2" },
  { u: 2, v: 0, key: "0-2" },
  { u: 2, v: 3, key: "2-3" },
  { u: 3, v: 4, key: "3-4" },
  { u: 4, v: 5, key: "4-5" },
  { u: 5, v: 3, key: "3-5" },
  { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
  {
    currentNode: 0, visitedIndices: [0],
    tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
    activeEdge: null, backEdge: null,
    log: "Входим в A. tin[A]=1, low[A]=1.",
    activeCodeLine: [0,1,2]
  },
  {
    currentNode: 1, visitedIndices: [0,1],
    tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
    activeEdge: [0,1], backEdge: null,
    log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
    activeCodeLine: [4,5,7,10,11]
  },
  {
    currentNode: 6, visitedIndices: [0,1,6],
    tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
    activeEdge: [1,6], backEdge: null,
    log: "Из B идём в G. tin[G]=3, low[G]=3.",
    activeCodeLine: [4,5,7,10,11]
  }
];

```

```

tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: [4,5], backEdge: null,
log: "E→F. tin[F]=7, low[F]=7.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: [5,3],
log: "Из F видим D (посещён). Обратное ребро (F,D).
activeCodeLine: [7,8,9]
},
{
currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: null,
log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо
activeCodeLine: [11,13,16,17]
},
{
currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: null,
log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо
activeCodeLine: [11,13,16,17]
},
{
currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: null,
log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) –
activeCodeLine: [13,14,15,16,17]
},
{
currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6
activeEdge: null, backEdge: null,
log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,

```



```

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">
        <div className="flex items-center gap-1 bg-slate-700 rounded"
          <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
            disabled={dfsIdx === 0}
            className="p-1.5 hover:bg-slate-700 rounded"
            <Undo className="h-3.5 w-3.5" />
          </button>
          <button onClick={() => setDfsAuto(!dfsAuto)}
            className={`px-2.5 py-1 rounded text-[10px] ${
              dfsAuto ? "bg-rose-600 text-white" : "bg-slate-700 text-white"
            }`}
            {dfsAuto ? <><Pause className="h-3 w-3" /> </> : </>}
          </button>
          <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length-1); }}
            disabled={dfsIdx === DFS_STEPS.length-1}
            className="p-1.5 hover:bg-slate-700 rounded"
            <SkipForward className="h-3.5 w-3.5" />
          </button>
          <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
            className="p-1.5 hover:bg-slate-700 rounded"
            <RefreshCw className="h-3.5 w-3.5" />
          </button>
        </div>
      </div>
    <div className="grid grid-cols-1 md:grid-cols-5 gap-4">
      /* Graph visualization */
      <div className="md:col-span-3 bg-slate-950 rounded"
        <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate-950_1px)]"
          </div>
        <svg className="w-full h-[280px] z-10 relative"
          {GRAPH_EDGES.map(e => {
            const u = GRAPH_NODES.find(n => n.id === e.u);
            const v = GRAPH_NODES.find(n => n.id === e.v);
            const bridge = step.bridges.includes(e.key);

```

```

        "transition-all duration-300" />
        <text x={n.x} y={n.y-20} textAnchor="left">
        { l:{step.low[n.id]|| "?"}}</text>
      </>
    ))
  </g>;
  )))
</svg>
<div className="flex items-center justify-between">
  <span>Step {dfsIdx+1}/{DFS_STEPS.length}</span>
  <div className="flex-1 mx-2 bg-slate-950 h-10 rounded">
    <div className="bg-indigo-600 h-full transition-all duration-300">
    </div>
  </div>
</div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
  {/* Code panel */}
  <div className="bg-slate-950 rounded-lg border border-slate-800">
    <div className="bg-slate-900 px-3 py-1.5 text-slate-400 font-mono">
      <pre>
        <div className="p-2 overflow-x-auto">
          <pre className="text-[10px] font-mono leading-relaxed">
            {CODE_LINES.map(({cl, i}) => {
              const hl = step.activeCodeLine.includes(cl.line)
              return <div key={i}
                className={`px-1 transition-all duration-300
                  ${hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-600"}
                `}>
                <span className="text-slate-600 mr-1">{cl.line} || ' '
              </div>;
            })}
          </pre>
        </div>
      </pre>
    </div>
  </div>
</div>

```

---

ФАЙЛ 37/87: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { useVizStepSync } from '../data/vizStepBus';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  currentNode: string | null;
  checkingEdge: { u: string; v: string } | null;
  distances: { [key: string]: number };
  visited: { [key: string]: boolean };
  previous: { [key: string]: string | null };
  log: string;
  activeCodeLine: number;
}

export default function DijkstraViz() {
  const nodes: Node[] = [
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
    { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
    { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
    { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
    { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
    { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
    { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
  ];

  const edges: Edge[] = [
    { u: 'A', v: 'B', w: 4 },
    { u: 'A', v: 'C', w: 2 },
    { u: 'C', v: 'B', w: 1 },
    { u: 'C', v: 'G', w: 3 },
  ];
```

```

const prev: Record<string, string | null> = { A: null };

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: ` Дейкстра готов к доставке. Начинаем из ПИ.`,
  activeCodeLine: 2,
});

for (let iter = 0; iter < nodes.length; iter++) {
  let minD = 999;
  let u = null;
  for (const n of nodes) {
    if (!visited[n.id] && dist[n.id] < minD) {
      minD = dist[n.id];
      u = n.id;
    }
  }

  if (!u) break;

  visited[u] = true;
  simulationSteps.push({
    currentNode: u, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ` Выбираем вершину ${u} с мин. непосещённой`,
    activeCodeLine: 5,
  });

  const neighbors = edges.filter(e => e.u === u);
  for (const edge of neighbors) {
    const v = edge.v;
    if (visited[v]) continue;

    simulationSteps.push({
      currentNode: u, checkingEdge: { u, v },
      distances: { ...dist }, visited: { ...visited },
      log: ` Смотрим соседа ${v}. Текущее расстояние`,
    });
  }
}

```

```

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border
    {/* Шапка симулятора */}
    <div className="flex flex-wrap items-center justify
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-indigo-600 text-white
          <span className="text-2xl"> </span>
        </div>
      </div>
      <h3 className="text-2xl font-bold text-white
        <p className="text-sm text-indigo-300">Полн
      </div>
    </div>

    <div className="flex items-center gap-2">
      <button
        onClick={() => { setIsPlaying(false); setCu
        className="flex items-center gap-1 bg-slate
        ion-colors"
      >
        Сброс
      </button>
      <button
        onClick={() => setIsPlaying(!isPlaying)}
        className={`flex items-center gap-1 px-4 py
          isPlaying ? 'bg-amber-600 hover:bg-amber-!
        }`}
      >
        {isPlaying ? '    Пауза' : '▶ Пуск'}
      </button>
      <button
        onClick={() => { setIsPlaying(false); if (c
        disabled={currentStepIndex >= steps.length

```

```

    let marker = 'url(#arrow-dh-normal)';
    if (isChecking) marker = 'url(#arrow-dh-a';
    else if (isOptimal) marker = 'url(#arrow-dh-n';

    return (
      <g key={idx}>
        <line
          x1={uNode.x} y1={uNode.y}
          x2={vNode.x} y2={vNode.y}
          stroke={isChecking ? '#f59e0b' : isOptimal ? '#f8b16d'}
          strokeWidth={isChecking ? 5 : isOptimal ? 3}
          markerEnd={marker}
          className="transition-all duration-300"
          strokeDasharray={isChecking ? '4,4' : '0'}
        />
        <circle
          cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)}
          r="12" fill="#1e293b"
          stroke={isChecking ? '#f59e0b' : '#f8b16d'}
        />
        <text
          x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2)}
          fill={isChecking ? '#f59e0b' : '#f8b16d'}
          fontSize="11" fontWeight="bold" textAnchor="center"
        >
          {edge.w}
        </text>
      </g>
    );
  }
}
}}}

{/* Вершины */}
{nodes.map((node) => {
  const isCurrent = currentStep.currentNode === node.id;
  const isVisited = currentStep.visited[node.id];
  const dist = currentStep.distances[node.id];

```

```

        isCurrentNode ? 'bg-emerald-900/60' :
        'text-slate-300'
      }`}>
      <div className="flex items-center gap-2">
        <span className={`px-2 py-0.5 rounded-sm text-slate-300`} `}>
          {u}
        </span>
        <span className="text-slate-400">→</span>
      </div>
      <div className="flex flex-wrap gap-2">
        {adjList[u].map((edge, idx) => {
          const isCheckingThis = currentStep === idx
          return (
            <span key={idx} className={`px-2 py-0.5 rounded-sm
              isCheckingThis ? 'bg-amber-900/60 text-slate-300' :
              'text-slate-400'`} `}>
              <span>{edge.v}</span><span className="font-size-0.8">
                <span className={isCheckingThis ? 'font-weight-normal' : 'font-weight-normal'}>
                  {edge.w}
                </span>
              </span>
            </div>
          )
        })}
      </div>
    </div>
  </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10 p-6">
    <div>
      <h4 className="text-lg font-bold text-rose-950">Таблица расстояний
      </h4>
      <div className="overflow-x-auto">

```





```

    });

    if (n in memoMap) {
        generatedSteps.push({
            id: stepId++,
            n,
            action: 'memo_hit',
            desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано`,
            codeLine: 2,
            memoState: { ...memoMap }
        });
        return memoMap[n];
    }

    if (n <= 2) {
        generatedSteps.push({
            id: stepId++,
            n,
            action: 'base_case',
            desc: `Базовый случай: n <= 2 (n=${n}). Возвр`,
            codeLine: 3,
            memoState: { ...memoMap }
        });
        return 1;
    }

    generatedSteps.push({
        id: stepId++,
        n,
        action: 'call',
        desc: `Вычисляем рекурсивно: fibМемо(${n - 1})`,
        codeLine: 4,
        memoState: { ...memoMap }
    });

    const res1 = simulateFib(n - 1);
    const res2 = simulateFib(n - 2);
    memoMap[n] = res1 + res2;

```

```

    return () => clearTimeout(timer);
  }, [isPlaying, currentStepIndex, steps, speed]);

const currentStep = steps[currentStepIndex] || null;
const memoState = currentStep?.memoState || {};

return (
  <div className="bg-slate-900 rounded-2xl border border-slate-800" >
    {/* Header & Controls */}
    <div className="flex flex-col lg:flex-row lg:items-center" >
      <div>
        <div className="flex items-center gap-3 mb-2" >
          <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
            <RefreshCw className="w-6 h-6" />
          </div>
          <h3 className="text-2xl font-extrabold text-white" >
            DP
          </h3>
          <p className="text-sm text-slate-400" >
            Смотри, как таблица DP кэширует результаты
          </p>
        </div>

        <div className="flex flex-wrap items-center gap-2" >
          <div className="flex items-center gap-2 bg-slate-800 p-2" >
            <span>N:</span>
            <select
              value={targetN}
              onChange={(e) => setTargetN(Number(e.target.value))}
              disabled={isPlaying}
              className="bg-transparent text-white font-mono" >
              <option value={4}>N = 4</option>
              <option value={5}>N = 5</option>
              <option value={6}>N = 6</option>
              <option value={7}>N = 7</option>
            </select>
          </div>
        </div>
      </div>
    </div>
  )

```

```

    { /* DP Table View */ }
    <div className="mb-8 bg-slate-950 p-4 md:p-6 rounded"
      <h4 className="text-xs font-bold uppercase tracking-wider"
        <div className="flex flex-wrap items-center gap-2"
          {Array.from({ length: targetN }, (_, i) => i)}
          const isCached = num in memoState || num <= 2
          const val = num <= 2 ? 1 : memoState[num]
          const isCurrent = currentStep?.n === num

          return (
            <div
              key={num}
              className={`flex flex-col items-center justify-center gap-2
                ${isCurrent ? 'bg-emerald-950 border-emerald-500' : ''}
                ${isCached ? 'bg-slate-900 border-emerald-500/50' : ''}
                ${!isCurrent && !isCached ? 'bg-slate-900/50 border-slate-800' : ''}
              `}
            >
              <span className="text-xs text-slate-400">Step {num}</span>
              <span className={`text-xl md:text-2xl font-medium`}
                {isCached ? val : '0'}>
              </span>
              {num <= 2 && <span className="text-[9px] font-mono">
                </div>
              )}
            </div>
          )
        </div>
      </div>

    {currentStep && (
      <div className="mt-6 pt-4 border-t border-slate-800"
        <div className="text-sm font-medium text-slate-300"
          <span className="inline-block w-2 h-2 rounded-full"
            <span>{currentStep.desc}</span>
          </div>
          <div className="text-xs font-mono bg-slate-900"

```

```

</h5>
<p className="text-sm text-slate-400">
  В классической рекурсии для  $N=\{targetN\}$  по
  вычисленное значение в таблице, мы момента.
</p>
<div className="p-4 bg-slate-900 rounded-xl
  text-slate-300">
  <span>Всего шагов симуляции: {steps.length}</span>
  <span className="text-emerald-400 font-bo
</div>
</div>
)}

{activeTab === 'code' && (
  <div className="bg-slate-950 rounded-2xl border
    <div className="bg-slate-900 px-6 py-3 border
      <span className="text-xs font-bold font-m
      <span className="text-xs text-emerald-400
    </div>
    <pre className="p-6 font-mono text-sm space
      {DP_CODE_LINES.map((line, idx) => {
        const lineNum = idx + 1;
        const isActive = currentStep?.codeLine
        return (
          <div
            key={lineNum}
            className={`flex items-center px-4
              isActive
                ? 'bg-emerald-600/20 border-l-4
                : 'text-slate-300 hover:bg-slate
            }`}
          >
            <span className="w-8 text-xs text-s
            <span>{line}</span>
          </div>
        );
      })}
    </pre>

```

```

    { id: 3, label: "D", x: 280, y: 250 },
    { id: 4, label: "E", x: 100, y: 250 }
  ];

const C1_EDGES = [
  { u: 0, v: 1 }, { u: 0, v: 2 },
  { u: 1, v: 3 }, { u: 2, v: 4 },
  { u: 1, v: 2 },
  { u: 3, v: 4 }
];

export function EulerSimulator() {
  const [c1Path, setC1Path] = useState<number[]>([]);
  const [c1VisitedEdges, setC1VisitedEdges] = useState<
  const [c1Msg, setC1Msg] = useState("Кликни на вершину");
  const [c1Done, setC1Done] = useState(false);

  const resetC1 = () => {
    setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
  };

  const clickC1 = (vId: number) => {
    if (c1Done && c1Path.length > 0) return;
    if (c1Path.length === 0) {
      setC1Path([vId]);
      setC1Msg("Старт! Кликай соседние вершины, чтобы п
      return;
    }
    const last = c1Path[c1Path.length - 1];
    const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
    if (!edge) {
      setC1Msg("Нет ребра между этими вершинами! Выбери
      return;
    }
    const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
    if (c1VisitedEdges.includes(key)) {
      setC1Msg("Это ребро уже пройдено! Эйлер не прощае
      setC1Done(true); return;
    }
  }
}

```

```

const key = `${Math.min(e.u,e.v)}-${Math.max(e.u,e.v)}`;
const used = clVisitedEdges.includes(key);
return <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
  stroke={used ? "#6366f1" : "#475569"}
  strokeWidth={used ? 4 : 2}
  className="transition-all duration-300" />
  )})
  {C1_NODES.map(n => {
    const isLast = clPath[clPath.length-1] === n.id;
    const visited = clPath.includes(n.id);
    return <g key={n.id} className="cursor-pointer"
      {isLast && <circle cx={n.x} cy={n.y} r={11}
        <circle cx={n.x} cy={n.y} r={11}
          fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
          stroke={isLast ? "#fff" : visited ? "#6366f1" : "#6366f1"}
          strokeWidth={2.5}
          className="transition-all duration-200" />
          <text x={n.x} y={n.y+4} textAnchor="middle">
            {n.label}</text>
          </g>
    )})
  </svg>
  <div className="absolute top-2 left-2 bg-slate-500 text-white p-2"
    <div>
      Ребёп: {clVisitedEdges.length}/{C1_EDGES.length}
    </div>
  </div>

  <div className={`p-3 rounded-xl border text-sm ${
    clDone && clVisitedEdges.length === C1_EDGES.length
      ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
      : clDone
      ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
      : "bg-slate-900/50 border-slate-700 text-slate-50"
    }`}>
    {clMsg}
  </div>
</div>
);

```

```
    'Потому что он написан только для положительных чисел',
    'Он жадный и считает, что каждый следующий шаг дает минимальный вес',
    'Отрицательные ребра создают гравитационный коллапс',
    'Дейкстра просто не любил отрицательные балансы на акциях',
  ],
  correctAnswer: 1,
  explanation: 'Именно! Дейкстра уверен: если ты приедешь в город, то для чего нужен алгоритм Беллмана-Форда.',
},
{
  id: 3,
  question: 'Какова ключевая фишка алгоритма Борувки',
  options: [
    'Он использует меньше памяти благодаря коммунити-меморизации',
    'Он умеет работать с отрицательными весами без циклов',
    'Он позволяет выполнять шаги слияния колхозов (коммуны)',
    'Он был разработан в Токио и поэтому самый быстрый',
  ],
  correctAnswer: 2,
  explanation: 'Абсолютно верно! В Борувке каждая коммуна должна делить вычисления на GPU или многопроцессорных кластерах',
},
{
  id: 4,
  question: 'В чем главная суть Динамического программирования',
  options: [
    'Писать код очень быстро, динамично нажимая на клавиши',
    'Запоминать (кешировать) результаты уже решенных подзадач',
    'Использовать динамическую типизацию как в Python',
    'Постоянно менять требования к проекту в процессе разработки',
  ],
  correctAnswer: 1,
  explanation: 'Точно! Главный принцип ДП – мемоизация (кеширование)',
},
{
  id: 5,
  question: 'Какая структура данных идеально помогает'
```

```

    });

    const handleRestart = () => {
      setCurrentQIndex(0);
      setSelectedOption(null);
      setIsAnswered(false);
      setScore(0);
      setShowResults(false);
    };

    if (showResults) {
      return (
        <div className="bg-slate-900/90 border border-slate-
          my-12">
          <div className="w-20 h-20 bg-gradient-to-tr from
            -6 shadow-xl shadow-indigo-500/30">
            <Award className="w-10 h-10 text-white" />
          </div>
          <h3 className="text-3xl font-bold text-white mb
          <p className="text-slate-400 text-sm mb-6">Ты пр

          <div className="bg-slate-950 border border-slate
            <p className="text-slate-400 text-sm uppercase
            <p className="text-5xl font-black text-indigo
            <p className="text-slate-300 text-sm">
              {score === quizQuestions.length
                ? ' Идеально! Ты настоящий сеньор-раскра
                : score >= 3
                ? ' Отличный результат! Главные концепции
                : ' Стоит повторить главы пособия и еще р
            </p>
          </div>

          <button
            onClick={handleRestart}
            className="inline-flex items-center gap-2 px-4
              d rounded-xl shadow-lg shadow-indigo-600/30
          >

```



```

        } else {
            optionStyle = "bg-slate-800/40 border-s
        }
    }
}

return (
    <div
        key={idx}
        onClick={() => handleSelectOption(idx)}
        className={"flex items-start gap-4 p-4"}
    >
        <div className="mt-0.5 shrink-0">
            {isAnswered && isCorrect ? (
                <CheckCircle2 className="w-6 h-6 text-
            ) : isAnswered && isSelected && !isCo
                <XCircle className="w-6 h-6 text-ro
            ) : (
                <div className={"w-6 h-6 rounded-fu
order-indigo-500 bg-indigo-500 text-white"
                    {String.fromCharCode(65 + idx)}
                </div>
            )}
        </div>
        <div className="flex-1 text-sm md:text-l
            {option}
        </div>
    </div>
);
}}
</div>
</div>

```

```

{isAnswered && (
    <div className="bg-slate-950 border border-slate
        <div className="flex items-center gap-2 mb-2
            <AlertCircle className="w-4 h-4" />
            <span>Объяснение:</span>
        </div>
    )}

```

```

export default function FloydViz() {
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  useVizStepSync(currentStepIndex, setCurrentStepIndex,
  const [isPlaying, setIsPlaying] = useState(false);

  const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
  const nodesSvg = [
    { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
    { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
    { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
    { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
    { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
    { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
  ];

  const initialMatrix = [
    [0, 4, 999, 2, 999, 999, 999],
    [999, 0, 3, 999, 999, 3, 999],
    [999, 999, 0, 999, 2, 999, 999],
    [999, 999, 999, 0, 4, 2, 999],
    [999, 999, 999, 999, 0, 999, 1],
    [999, 999, 999, 999, 999, 0, 5],
    [999, 1, 999, 999, 999, 999, 0]
  ];

  const adjList: { [key: number]: { v: number; w: numbe
    0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
    1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
    2: [{ v: 4, w: 2 }],
    3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
    4: [{ v: 6, w: 1 }],
    5: [{ v: 6, w: 5 }],
    6: [{ v: 1, w: 1 }],
  };

```

```

        k, i, j, matrix: dist.map(r => [...r]), u
        log: `  Улучшение пути [${i}]->[${j}] чере
        ∞'}.`,
        activeCodeLine: 7,
      });
    }
  }
}

simulationSteps.push({
  k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
  log: `  Все пары отработаны. Алгоритм Флойда завер
  activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length -
    timer = setTimeout(() => setCurrentStepIndex((p)
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl bor
    <div className="flex flex-wrap items-center justifi
      <div className="flex items-center gap-3">

```

```

const u = parseInt(uStr);
return adjList[u].map((edge) => {
  const uNode = nodesSvg.find((n) => n.id === u);
  const vNode = nodesSvg.find((n) => n.id === v);
  const isTarget = currentStep.i === u && currentStep.j === v;
  const isVia = (currentStep.i === u && currentStep.j !== v) || (currentStep.i !== u && currentStep.j === v);
  let strokeColor = '#475569'; let marker = 'none';
  if (isTarget) { strokeColor = '#10b981'; marker = 'circle'; }
  else if (isVia) { strokeColor = '#818cf8'; marker = 'none'; }
  return (
    <g key={`-${u}-${v}`}>
      <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} strokeWidth={4} markerEnd={marker} strokeDasharray={isTarget ? [5, 5] : [1, 0]} />
      <circle cx={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)} r={isTarget ? 4 : 2} stroke={strokeColor} strokeWidth={4} fill={isTarget ? '#10b981' : '#818cf8'} />
      <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2) + 10} text={`${u} -> ${v}`} font-size="10" font-weight="bold" fill="white" />
    </g>
  );
});
}}}
{nodesSvg.map((node) => {
  const isK = currentStep.k === node.id;
  const isI = currentStep.i === node.id;
  const isJ = currentStep.j === node.id;
  let fill = '#1e293b'; let stroke = '#64748f';
  if (isK) { fill = '#4f46e5'; stroke = '#a6a6a6'; }
  else if (isI) { fill = '#b45309'; stroke = '#a6a6a6'; }
  else if (isJ) { fill = '#047857'; stroke = '#a6a6a6'; }
  return (
    <g key={node.id} className="transition-colors">
      <circle cx={node.x} cy={node.y} r={isK ? 4 : 2} className="transition-colors duration-200" fill={fill} stroke={stroke} strokeWidth={4} />
      <text x={node.x} y={node.y + 5} fill="white" font-size="10" font-weight="bold" />
      <text x={node.x} y={node.y + 32} fill="white" font-size="10" font-weight="bold" />
    </g>
  );
});

```



```
const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
  { id: 'E', label: 'E', x: 150, y: 200 },
  { id: 'F', label: 'F', x: 250, y: 200 },
  { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
  'A': ['B', 'C'],
  'B': ['A', 'D', 'E'],
  'C': ['A', 'F', 'G'],
  'D': ['B'],
  'E': ['B'],
  'F': ['C'],
  'G': ['C'],
};
```

```
type Action = {
  type: 'visit' | 'process' | 'backtrack';
  node: string;
  desc: string;
  queueOrStack?: string[];
  visitedNodes?: string[];
};
```

```
function generateDFSSteps(start: string) {
  const steps: Action[] = [];
  const vis = new Set<string>();
```

```

        q], visitedNodes: [...Array.from(vis)] });
    q.shift();

    for (const u of adj[v]) {
        if (!vis.has(u)) {
            vis.add(u);
            q.push(u);
            steps.push({ type: 'visit', node: u, desc: `04
            tack: [...q], visitedNodes: [...Array.from(
            }
        }
    }

    steps.push({ type: 'backtrack', node: '', desc: `04
    om(vis)] });

    return steps;
}

export function GraphTraversalViz() {
    const [mode, setMode] = useState<"dfs" | "bfs">("df
    const [autoPlay, setAutoPlay] = useState(false);
    const [stepIdx, setStepIdx] = useState(0);

    const steps = useMemo(() => mode === 'dfs' ? genera
    useVizStepSync(stepIdx, setStepIdx, steps.length -

    useEffect(() => {
        setStepIdx(0);
        setAutoPlay(false);
    }, [mode]);

    useEffect(() => {
        if (autoPlay) {
            const t = setInterval(() => {
                setStepIdx(p => {
                    if (p >= steps.length - 1) {

```

```

        const v = nodes.find(n => n.id === vId)
        // Check if edge is part of current path
        const edgeTraversed = isVisited(v, path)

        return (
          <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
            className={`stroke-${
              500 : 'stroke-emerald-500'
            }`}
          />
        )
      )
    }

    /** Nodes */
    {nodes.map(n => {
      const visited = isVisited(n, path)
      const current = isCurrent(n, path)

      let fillColor = "fill-slate-500"
      let strokeColor = "stroke-slate-500"
      let textColor = "fill-slate-500"
      let radius = 18;

      if (visited) {
        fillColor = mode === 'dark' ? 'fill-amber-500' : 'fill-emerald-500'
        strokeColor = mode === 'dark' ? 'stroke-amber-500' : 'stroke-emerald-500'
        textColor = "fill-white"
      }

      if (current && step.type !== 'edge') {
        strokeColor = "stroke-amber-500"
        textColor = "fill-amber-500"
        radius = 22;
      }

      return (
        <g key={n.id} className={`node-${n.id}`}
          <circle cx={n.x} cy={n.y}
            className={`stroke-${strokeColor} fill-${fillColor} r=${radius}`}
          />
          <text x={n.x} y={n.y}
            className={`text-${textColor} font-size-12`}
          />
        />
      )
    })}
  )
}

```





```
function siftDown(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  const n = a.length;
  let idx = 0;
  while (true) {
    let largest = idx;
    const l = 2 * idx + 1;
    const r = 2 * idx + 2;
    if (l < n && a[l].priority > a[largest].priority) largest = l;
    if (r < n && a[r].priority > a[largest].priority) largest = r;
    if (largest !== idx) {
      [a[largest], a[idx]] = [a[idx], a[largest]];
      idx = largest;
    } else break;
  }
  return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
  return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
  const level = Math.floor(Math.log2(index + 1));
  const posInLevel = index - (Math.pow(2, level) - 1);
  const count = Math.pow(2, level);
  const x = ((posInLevel + 0.5) / count) * 100;
  const y = 10 + level * 24;
  return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
  { name: 'Иван (Инфаркт)', symptom: ' Болят сердце', priority: 100 },
  { name: 'Маша (Перелом)', symptom: ' Открытый перелом ноги', priority: 80 },
  { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', priority: 60 },
  { name: 'Семён (Температура)', symptom: ' Жар 39.5', priority: 40 }
];
```

```

        setTimeout(() => setShake(false), 400);
        return;
    }
    setBusy(true);

    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const finalName = name || preset.name.split(' ')[0];
    const finalEmoji = preset.emoji;
    const finalSymptom = preset.symptom;

    const newPatient: Patient = {
        id: `p${uid++}`,
        name: finalName,
        emoji: finalEmoji,
        symptom: finalSymptom,
        priority,
        animState: 'in',
    };

    const nextHeap = heapInsert(heap, newPatient).map(x => x);
    // Mark specifically this new patient as 'in' in the heap
    const target = nextHeap.find(x => x.priority === priority);
    const marked = nextHeap.map(x => x.id === (target ? target.id : null));

    setHeap(marked);
    addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

    setTimeout(() => {
        setHeap(nextHeap);
        setBusy(false);
    }, 500);
}, [busy, heap]);

const handleRandomInsert = () => {
    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const randomShift = Math.floor(Math.random() * 10);
    const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
    insertPatient(preset.name, prio);
};
```

```

    });

    // Patient gets healed in the bed
    setTimeout(() => {
      setHealing(prev => prev ? { ...prev, animStat: 'healed' } : null);
      addLog(` Успех! Пациент ${topPatient.name} поправился.`);
      setTimeout(() => {
        setHealing(null);
        setBusy(false);
      }, 600);
    }, 800);

    }, 350);
  }, 650);
}, [busy, heap]);

const reset = () => {
  setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 'idle' })));
  setLog([' База данных скорой помощи перезагружена.']);
  setHealing(null);
  setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 'idle' } ))));
};

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
  const res: React.ReactElement[] = [];
  const p = nodePos(idx);
  const l = 2 * idx + 1;
  const r = 2 * idx + 2;
  if (l < heap.length) {
    const lp = nodePos(l);
    res.push(
      <line key={`el${idx}`}
        x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
        x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
        stroke="#475569" strokeWidth="2"
      />
    );
  }
});

```

```

        className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-800 to-emerald-500
        opacity-40 disabled:cursor-not-allowed text-white font-mono text-sm"
        value={scale} transition-all cursor-pointer focus:outline-none"
    >
        <span> Привезти по Скорой (INSERT)</span>
    </button>

    {/* Добавить кастомного */}
    <form onSubmit={handleCustomInsert} className="flex flex-col items-center min-h-100"
    >
        <input
            type="text"
            required
            value={customName}
            onChange={e => setCustomName(e.target.value)}
            placeholder="Имя пациента"
            className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:text-slate-400"
        />
        <div className="flex flex-col items-center min-h-100"
        >
            <span className="text-[9px] font-mono text-slate-400"
            >
                <input
                    type="range"
                    min="10"
                    max="99"
                    value={customPriority}
                    onChange={e => setCustomPriority(Number(e.target.value))}
                    className="w-20 accent-emerald-500 cursor-pointer"
                />
            </div>
            <button
                type="submit"
                disabled={busy || !customName.trim() || !customPriority}
                className="bg-teal-600 hover:bg-teal-500 disabled:opacity-50 text-white font-mono text-sm px-4 rounded-xl shadow-lg active:scale-95 transition-all"
            >
                Везти
            </button>
        </form>
    
```

```

        const isRoot = idx === 0;
        return (
          <div
            key={patient.id}
            className={`
              absolute flex flex-col items-center
              -translate-x-1/2 -translate-y-1/2 t
              ${patient.animState === 'in' ? 'ani
              ${patient.animState === 'winner' ?
              ${patient.animState === 'out' ? 'an
              ${isRoot ? 'bg-rose-950/80 border-r
            `}
            style={{
              left: `${pos.x}%`,
              top: `${pos.y}%`,
              width: isRoot ? 60 : 52,
              height: isRoot ? 60 : 52,
            }}
          >
            <span className="text-2xl leading-non
            <span className="text-[10px] font-mono
              {patient.priority}%
            </span>

            {/* Tooltip */}
            <div className="absolute bottom-full
              <div className="bg-slate-950 border
0 shadow-xl">
              <div className="font-bold text-wh
              <div className="text-emerald-400"
              </div>
            </div>
          </div>
        );
      }
    </div>

    <div className="w-full h-3 bg-gradient-to-r f

```

```
      <div className="w-full h-3 bg-gradient-to-r f
    </div>

  </div>

  {/* ЛОГ ДЕЙСТВИЙ */}
  <div className="bg-slate-900 border border-slate-
    <div className="text-[10px] font-mono text-slate
      {log.map((entry, idx) => {
        <div
          key={idx}
          className={`text-xs font-mono flex items-ce
        >
          {idx === 0 ? <span className="text-emerald-5
          <span>{entry}</span>
        </div>
      }}}
    </div>
  </div>
);
};
```

---

ФАЙЛ 44/87: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { createContext, useContext, useEffect, u
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```

    return (
      <div className="w-full">
        <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
          {children}
        </svg>
        {caption && (
          <div className="text-[10px] text-slate-400 text-center">
            {caption}
          </div>
        )}
        <div className="text-[9px] text-slate-600 text-center">
          {paused ? "пауза – курсор на карточке" : "наведение"}
        </div>
      </div>
    );
  };
};

export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}>
      {label !== undefined && (
        <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
          {label}
        </text>
      )}
    </g>
  );
};

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
}> = ({ a, b }) => (
  <g>
    <line x1={a[0]} y1={a[1]} x2={b[0]} y2={b[1]}>
  </g>
);

```



```

        ? "Разрешаем пересадку через k: 3 + 4 = 7"
        : "7 < 9 → D[i][j] = 7"
    }
    >
    <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad
    <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done
    <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done

    <text x={130} y={112} textAnchor="middle" fontSize=
        9
    </text>
    <text x={72} y={54} textAnchor="middle" fontSize=
        3
    </text>
    <text x={188} y={54} textAnchor="middle" fontSize=
        4
    </text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
    </Svg>
    );
};

```

/\*\* Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][] = [
    [
      ["A", "B"], ["B", "C"]],
    [
      ["D", "E"]],
    [
      ["F", "G"], ["G", "H"], ["F", "H"]],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];

```

```

    <Edge a={a} b={b} color={step >= 1 ? C.active : C.idle}
    {step >= 2 && {
      <text
        x={({a[0] + b[0]) / 2}
        y={({a[1] + b[1]) / 2 - 4}
        textAnchor="middle"
        fontSize={10}
        fontWeight="700"
        fill={C.warn}
      >
        {w}
      </text>
    }}
  </g>
);
}})
{Object.entries(pos).map(([k, [x, y]]) => (
  <Node key={k} x={x} y={y} label={k} fill={C.idle} />
))}
</Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (
    <Svg
      caption={
        step === 0
          ? "Исходные координаты – огромные и с дырами"
          : step === 1
            ? "Сортируем уникальные значения"

```

```

const SplayFrames: React.FC<{
  frames: { pos: Record<string, Pt>; edges: [string, string][];
  captions: string[];
  ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
  const step = useLoop(frames.length, ms);
  const f = frames[step];
  const color = (id: string) =>
    id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f1c40f" : "#f1c40f";

  return (
    <Svg caption={captions[step]}>
      {f.edges.map(([a, b], i) => (
        <line
          key={i}
          x1={f.pos[a][0]}
          y1={f.pos[a][1]}
          x2={f.pos[b][0]}
          y2={f.pos[b][1]}
          stroke={C.edge}
          strokeWidth={1.8}
          style={{ transition: MOVE }}
        />
      ))}
      {Object.entries(f.pos).map(([id, [x, y]]) => (
        <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
          <circle r={13} fill={color(id)} stroke={C.idl} />
          <text textAnchor="middle" dominantBaseline="bottom">
            {id}
          </text>
        </g>
      ))}
    </Svg>
  );
};

```

/\*\* Zig – один поворот, когда родитель уже корень. \*/

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
  A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
  const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
  const step = useLoop(order.length + 1, 850);
  const visited = new Set(order.slice(0, step).flat());
  const wave = step > 0 && step <= order.length ? order
  return (
    <Svg caption={`Уровень ${Math.max(0, Math.min(step,
      {GRAPH_EDGES.map(([u, v], i) => {
        <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
      }}})
    {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
      <Node key={k} x={x} y={y} label={k}
        fill={wave.includes(k) ? C.active : visited.h
        stroke={wave.includes(k) ? "#a5b4fc" : undefin
    }}})
  </Svg>
  );
};

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join("

```

```

        x + 1}</text>
      })
    </g>
  );
}
<text x={224} y={62} textAnchor="middle" fontSize=
<text x={224} y={74} textAnchor="middle" fontSize=
</Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь"
      <Edge a={[60, 70]} b={[200, 70]} color={improved
      <text x={130} y={54} textAnchor="middle" fontSize=
      <Node x={60} y={70} r={20} fill={C.active} label=
      <Node x={200} y={70} r={20} fill={improved ? C.do
      <text x={60} y={106} textAnchor="middle" fontSize=
      <text x={200} y={106} textAnchor="middle" fontSiz
      <text x={130} y={22} textAnchor="middle" fontSize=
    </Svg>
  );
};

```

```

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D
  ];
  const cut = step === 1;
  return (

```

```

    });
    const edges = [
      { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
      { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
    ];
    const chosen = ["A-D", "A-B", "B-C", "D-E"];
    const step = useLoop(chosen.length + 1, 800);
    const taken = new Set(chosen.slice(0, step));
    return (
      <Svg caption={`Жадно берём самые лёгкие рёбра без ц
        {edges.map((e, i) => {
          const on = taken.has(`${e.u}-${e.v}`);
          return (
            <g key={i}>
              <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
              <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y
                textAnchor="middle" fontSize={9} fontWeigh
            </g>
          );
        })}
        {Object.entries(pos).map(([k, [x, y]]) => (<Node
      </Svg>
    );
  };

```

```

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
  };
  const edges: [string, string][] = [["A", "B"], ["B",
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 &&
  return (
    <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
      {edges.map(([u, v], i) => {
        <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
      })}
    )}
  );
}

```

```

return (
  <Svg caption={popping ? "pop: забираем ВЕРХНИЮ таре."
    <rect x={90} y={22} width={80} height={96} rx={6}
    {items.map((v, i) => (
      <g key={v} style={{ transition: "all .3s" }}>
        <rect x={94} y={100 - i * 26} width={72} height={26}
          fill={i === items.length - 1 ? (popping ? C.done : C.dim)} />
        <text x={130} y={111 - i * 26} textAnchor="middle" />
      </g>
    ))}
    <text x={130} y={16} textAnchor="middle" fontSize={12} />
  </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
  const step = useLoop(states.length, 800);
  const items = states[step];
  return (
    <Svg caption="pop_front слева, push_back справа – к концу"
      <defs>
        <marker id="mdArrow" markerWidth="6" markerHeight="6"
          <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
        </marker>
      </defs>
      <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
      <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={30 + i * 54} y={56} width={46} height={26}
            fill={i === items.length - 1 ? (popping ? C.done : C.dim)} />
          <text x={53 + i * 54} y={71} textAnchor="middle" />
        </g>
      ))}
      <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
    </Svg>
  );
};

```





```

const pref = a.reduce<number[]>((acc, v) => [...acc,
const step = useLoop(4, 1200);
const l = 1, r = 3;
const show = step >= 1;
return (
  <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -
    о префиксные суммы P">
    <text x={4} y={18} fontSize={8} fill={C.dim}>a</t
    {a.map((v, i) => (
      <g key={i}>
        <rect x={20 + i * 39} y={24} width={34} height
          fill={show && i >= l && i <= r ? C.active :
        <text x={37 + i * 39} y={36} textAnchor="midd
          >
        </g>
      )
    )
  )
  <text x={4} y={72} fontSize={8} fill={C.dim}>P</t
  {pref.map((v, i) => (
    <g key={i}>
      <rect x={20 + i * 33} y={78} width={29} height
        fill={show && i === l ? C.bad : show && i =
        stroke={C.idleStroke} style={{ transition:
      <text x={34 + i * 33} y={90} textAnchor="midd
        >
      </g>
    )
  )
  </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы дл
      {Array.from({ length: 8 }).map((_, i) => (
        <g key={i}>

```

```

    <text x={196} y={84} textAnchor="middle" dominantBaseline="middle" fill={C.dim}>
      ?"></text>
    <text x={130} y={124} textAnchor="middle" fontSize={16} fill={C.dim}>
    </Svg>
  );
};

```

```

export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);
  const i = Math.max(0, step - 1);
  const len = step > 0 ? pi[i] : 0;
  return (
    <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс = суффикс` : ""}>
      {s.split("").map((ch, idx) => (
        <g key={idx}>
          <rect x={18 + idx * 33} y={26} width={28} height={20}
            fill={step > 0 && (idx < len || (idx > i - len)) ? C.warn : C.idle}
            style={{ transition: "fill .25s" }} />
          <text x={32 + idx * 33} y={40} textAnchor="middle" font-size={16}>
            {ch}</text>
          <text x={32 + idx * 33} y={70} textAnchor="middle" font-size={16}>
            {pi[idx]}</text>
          <rect x={18 + idx * 33} y={70} width={28} height={20}
            fill={idx <= i && step > 0 ? C.warn : C.idle} />
        </g>
      ))}
      <text x={4} y={40} font-size={8} fill={C.dim}>s</text>
      <text x={4} y={70} font-size={8} fill={C.dim}>π</text>
      <text x={130} y={106} textAnchor="middle" font-size={16}>
        π[i] – длина самого длинного «префикс = суффикс»</text>
    </Svg>
  );
};

```

```

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);

```

```
import {
  ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBa
  ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
  | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
  | "suffix-link" | "toposort" | "relax" | "prefix-sum"
  | "segment-tree" | "bridge" | "articulation" | "mst"
  | "prefix-function" | "dp" | "floyd" | "components" |
  | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
  bfs: BfsDemo,
  dfs: DfsDemo,
  heap: HeapDemo,
  stack: StackDemo,
  queue: QueueDemo,
  dsu: DsuDemo,
  trie: TrieDemo,
  "trie-bfs": TrieBfsDemo,
  "suffix-link": SuffixLinkDemo,
  toposort: ToposortDemo,
  relax: RelaxDemo,
  "prefix-sum": PrefixSumDemo,
  "sparse-table": SparseTableDemo,
  "segment-tree": SegmentTreeDemo,
  bridge: BridgeDemo,
  articulation: ArticulationDemo,
  mst: MstDemo,
  amortized: AmortizedDemo,
  np: NpDemo,
  scc: SccDemo,
  "prefix-function": PrefixFunctionDemo,
  dp: DpDemo,
  floyd: FloydDemo,
  components: ComponentsDemo,
  graph: GraphBasicsDemo,
```

```

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
  useEffect(() => {
    if (!vizId) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") onClose();
    };
    window.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
      window.removeEventListener("keydown", onKey);
      document.body.style.overflow = "";
    };
  }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
  <div className="fixed inset-0 z-[110] flex items-en
    <div className="absolute inset-0 bg-slate-950/80
    <div className="relative w-full sm:max-w-5xl max-l
      ded-2xl shadow-2xl flex flex-col">
        <div className="flex items-center gap-3 px-4 py
          <div className="min-w-0 flex-1">
            <h3 className="font-bold text-white text-sm
              {viz.hint && <p className="text-[11px] text
            </div>
            <button
              onClick={onClose}
              className="p-2 rounded-lg bg-slate-800 hove
              aria-label="Заккрыть симулятор"
            >
              <X className="w-5 h-5" />
            </button>
          </div>
        </div>

```

```
export const TermPopover: React.FC<Props> = ({ anchor,
  const cardRef = useRef<HTMLDivElement>(null);
  const [pos, setPos] = useState<{ top: number; left: number}>({ top: 0, left: 0 });

  useEffect(() => {
    if (!anchor) {
      setPos(null);
      return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300;

    let left = anchor.rect.left + anchor.rect.width / 2;
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

    setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left });

    useEffect(() => {
      if (!anchor) return;
      const onKey = (e: KeyboardEvent) => {
        if (e.key === "Escape") onClose();
      };
      window.addEventListener("keydown", onKey);
      window.addEventListener("scroll", onClose, true);
      return () => {
        window.removeEventListener("keydown", onKey);
        window.removeEventListener("scroll", onClose, true);
      };
    }, [anchor, onClose]);

    if (!anchor) return null;
    const entry = GLOSSARY_BY_ID.get(anchor.termId);
```

```
        </p>
      )}

      {entry.complexity && {
        <div className="text-[11px] font-mono text-
          Сложность: {entry.complexity}
        </div>
      )}

      {viz && simulatorId && {
        <button
          onClick={() => onOpenSimulator(simulatorId)}
          className="w-full flex items-center justify-
            unded-lg py-2 transition-colors"
        >
          <Play className="w-3.5 h-3.5" /> Показать
        </button>
      )}
    </div>
  </div>,
  document.body
);
};
```

---

ФАЙЛ 51/87: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync } from '../data/vizStepBus';

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;
```

```

        return "stroke-slate-700 stroke-dasharray-4";
    }

    if (e.id === 'AB') {
        if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
        if (step > 4) return "stroke-emerald-500";
    }
    if (e.id === 'BC') {
        if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
        if (step > 5) return "stroke-emerald-500";
    }
    if (e.id === 'CA') {
        if (step === 6) return "stroke-amber-400 stroke-dasharray-4";
        if (step > 6) return "stroke-emerald-500";
    }

    if (step >= 4) return "stroke-slate-600"; // отсюда
    return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
    if (e.u === 'S') {
        if (step === 0 || step === 7) return "";
        if (step === 1) return "url(#arrow-amber)";
        if (step === 2) return "url(#arrow-pink)";
        return "url(#arrow-slate-dim)";
    }
    if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
    if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
    if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
    if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
    if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
    if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

    if (step >= 4) return "url(#arrow-slate-dim)";
    return "url(#arrow-slate)";
};

```

```

case 0: return (
    <div>
        <b>Исходный граф.</b> Имеется ребро
        Если мы запустим быстрый алгоритм Д
    </div>
);
case 1: return (
    <div>
        <b>Шаг 1: Точка отсчета.</b><br/>
        Добавляем фиктивную вершину S. Соед
ми</span>.
        Это наша база для расчета потенциал
    </div>
);
case 2: return (
    <div>
        <b>Шаг 2: Ищем потенциалы  $h(v)$ .</b>
        Запускаем медленный, но мощный алгор
ных вершин:
        <span className="text-pink-300 ml-1
    </div>
);
case 3: return (
    <div>
        <b>Шаг 3: Магическая формула.</b><b>
        Всем ребрам графа назначаем новый в
        <div className="text-center font-mo
        Это повышает или понижает вес ребра
    </div>
);
case 4: return (
    <div>
        <b>Шаг 3.1: Пересчет ребра  $A \rightarrow B$ .</b>
        Старый вес: <span className="text-r
        Вычисляем:  $-2 + 0 - (-2) =$  <span cl
    </div>
);
case 5: return (

```



```

start-reverse">
    <path d="M 0 0 L 10 5 L 10 10" />
    </marker>
    <marker id="arrow-slate-dim" viewBox="0 0 10 10" />
uto-start-reverse">
    <path d="M 0 0 L 10 5 L 10 10" />
    </marker>
    <marker id="arrow-amber" viewBox="0 0 10 10" />
start-reverse">
    <path d="M 0 0 L 10 5 L 10 10" />
    </marker>
    <marker id="arrow-pink" viewBox="0 0 10 10" />
tart-reverse">
    <path d="M 0 0 L 10 5 L 10 10" />
    </marker>
    <marker id="arrow-emerald" viewBox="0 0 10 10" />
o-start-reverse">
    <path d="M 0 0 L 10 5 L 10 10" />
    </marker>
</defs>

{/* Edges */}
{edges.map((e, i) => {
  const u = nodes.find(n => n.id === e.u);
  const v = nodes.find(n => n.id === e.v);

  const isS = e.u === 'S';
  if (isS && (step === 0 || step === 1)) {
    return null;
  }

  const midX = (u.x + v.x) / 2;
  const midY = (u.y + v.y) / 2;

  const edgeColorClass = getEdgeColorClass(e);
  const textHigh = isEdgeTextHigh(e);
  const textEmer = isEdgeTextEmer(e);

  return (
    <g key={i}>

```

```

</text>

        {pot !== null && {
          <g transform={`
fade-in zoom-in duration-500">
            <rect x="-1
k-500 stroke-1" />
            <text x="0"
font-bold fill-pink-400">
              {pot}
            </text>
          </g>
        }}
      </g>
    )
  }}}}
</svg>
</div>

```

```

<div className="w-full md:w-[320px] flex
  /* Log Box */
  <div className="bg-[#0f172a] p-5 ro
l justify-center leading-relaxed text-sm te
    {getDesc()}
  </div>

```

```

  /* Controls */
  <div className="flex bg-slate-900 b
    <button onClick={() => setStep(1
      className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
      <RotateCcw strokeWidth={3}
    </button>
    <button onClick={() => setStep(1
      className="p-3 bg-slate-800
-slate-400 transition-colors" title="Вар на
      <Undo strokeWidth={3} size=
    </button>

```

```

interface GNode {
  id: number;
  x: number;
  y: number;
  label: string;
}

interface GEdge {
  u: number;
  v: number;
}

const initialNodes: GNode[] = [
  { id: 0, x: 150, y: 100, label: "0" },
  { id: 1, x: 300, y: 100, label: "1" },
  { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
  { id: 3, x: 450, y: 160, label: "3" },
  { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 }, // Connection edge
  { u: 3, v: 4 },
  { u: 4, v: 3 },
];

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
}

```

```

    recordStep(
      "init",
      "Инициализация алгоритма Косарайю. Начинаем первый
      null,
      false,
      -1,
    );

    // Adj list
    const adj: number[][] = Array.from({ length: 5 }, (
    initialEdges.forEach((e) => adj[e.u].push(e.v)));

    // DFS 1: post-order list
    const dfs1 = (u: number) => {
      visited.add(u);
      recordStep(
        "dfs1",
        `DFS 1: зашли в вершину ${u}, помечаем посещение
        u,
        false,
        -1,
      );

      for (const to of adj[u]) {
        if (!visited.has(to)) {
          dfs1(to);
        }
      }

      order.push(u);
      recordStep(
        "dfs1",
        `DFS 1: вершина ${u} полностью обработана. Запи
        u,
        false,
        -1,
      );
    };
  };

```

```

        if (!visited.has(to)) {
            dfs2(to, currentScc);
        }
    }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
    const u = order[i];
    if (!visited.has(u)) {
        sccId++;
        recordStep(
            "dfs2",
            `DFS 2: берем следующую непосещенную из стека`,
            u,
            true,
            sccId,
        );
        dfs2(u, sccId);
    } else {
        recordStep(
            "dfs2",
            `DFS 2: вершина ${u} из стека уже покрашена в`,
            u,
            true,
            sccId,
        );
    }
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} компонент`,
    null,
    true,
    sccId,
);

```

```

    });

    return (
      <div className="bg-slate-900 rounded-xl border border-
        <div className="bg-slate-800 p-4 border-b border-
          <h3 className="font-bold text-white flex items-
            <Layers className="w-5 h-5 text-emerald-400"
              Визуализатор Алгоритма Косарайю (Поиск SCC)
            </h3>

          <div className="flex items-center gap-2 bg-slate-
            <button
              onClick={togglePlay}
              className="flex items-center gap-2 px-3 py-
                d-500 text-white"
            >
              {isPlaying ? (
                <Pause className="w-4 h-4 ml-1" />
              ) : (
                <Play className="w-4 h-4 ml-1" />
              )}
              {isPlaying ? "Пауза " : "Старт "}
            </button>

            <button
              onClick={reset}
              className="flex items-center justify-center
                ors"
            >
              <RotateCcw className="w-4 h-4" />
            </button>
          </div>
        </div>

        <div className="grid grid-cols-1 lg:grid-cols-12
          {/* Playfield SVG */}
          <div className="lg:col-span-8 bg-[#0B1120] rela
            <svg className="w-full h-full max-w-[650px]"

```

```

{initialEdges.map((edge, idx) => {
  const u = initialNodes.find((n) => n.id === edge.u);
  const v = initialNodes.find((n) => n.id === edge.v);

  // If transposed, draw reversed coordinates
  const x1 = step.transposed ? v.x : u.x;
  const y1 = step.transposed ? v.y : u.y;
  const x2 = step.transposed ? u.x : v.x;
  const y2 = step.transposed ? u.y : v.y;

  const isSccEdge =
    step.sccMap[edge.u] !== step.sccMap[edge.v] ||
    step.sccMap[edge.u] === step.sccMap[edge.v] &&
    step.sccMap[edge.u] !== step.sccMap[edge.v];
  let stroke = "#475569";
  let markerId = "arrow";

  if (step.transposed) {
    stroke = isSccEdge ? "#818cf8" : "#4f4682";
    markerId = "arrow-transposed";
  } else {
    if (
      step.currentNode === edge.u ||
      step.currentNode === edge.v
    ) {
      stroke = "#10b981";
      markerId = "arrow-active";
    }
  }
}

// Adjust slightly for bidirectional link
const isBidi =
  (edge.u === 3 && edge.v === 4) ||
  (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
  <line

```

```

        />
        <text
            x={node.x}
            y={node.y}
            dy=".3em"
            textAnchor="middle"
            fill="white"
            fontSize="13px"
            fontWeight="bold"
            className="select-none"
        >
            {node.label}
        </text>

        {sccId && (
            <text
                x={node.x}
                y={node.y - 28}
                textAnchor="middle"
                fill="#10b981"
                fontSize="10px"
                fontWeight="bold"
            >
                SCC #{sccId}
            </text>
        )}
    </g>
  );
  }}}
</svg>

{step.transposed && (
  <div className="absolute top-4 left-4 bg-in
    1 rounded">
    Граф транспонирован (Ребра обратные)
  </div>
  )}
</div>

```



```

    <div>
      <span className="text-[10px] text-slate-500">
        ЛОГ ОПЕРАЦИЙ:
      </span>
      <div className="space-y-1.5 max-h-[150px]">
        {steps
          .slice(0, currentStepIdx + 1)
          .reverse()
          .slice(0, 5)
          .map((s, idx) => {
            <div
              key={idx}
              className={`text-[11px] p-1.5 rounded`
            >
              {s.desc}
            </div>
          )}}
        </div>
      </div>
    </div>

    <div className="mt-4 pt-3 border-t border-slate-200">
      <span>
        Шаг {currentStepIdx + 1} из {steps.length}
      </span>
      <div className="flex gap-1.5">
        <button
          disabled={currentStepIdx === 0}
          onClick={() => setCurrentStepIdx((prev) => prev - 1)}
          className="bg-slate-900 border border-slate-700 text-white px-2 py-1 rounded"
        >
          Назад
        </button>
        <button
          disabled={currentStepIdx >= steps.length - 1}
          onClick={() => setCurrentStepIdx((prev) => prev + 1)}
          className="bg-slate-900 border border-slate-700 text-white px-2 py-1 rounded"
        >
          Далее
        </button>
      </div>
    </div>
  </div>
</div>

```

```

    title: string;
    description: string;
    type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
  { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
  { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
  { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
  { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
  { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
  { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
  { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
  { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
  { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
  { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
  { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
  { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
  { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
  { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
  { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
  { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
  { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
  { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
  { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
  { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
  { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'>
  const [vertices, setVertices] = useState<Vertex[]>(in
  const [edges, setEdges] = useState<Edge[]>(initialEdge

```

```

        title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
        description: 'Оно соединяет две другие бесцветны
        type: 'info'
    }, ...prev]));
},
// Step 4: Include D-E
() => {
    setVertices(prev => prev.map(v => v.id === 3 || v
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-E в осто́ве',
        description: 'Вершины D и E окрашены в синий цв
        type: 'success'
    }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Берем ребро B-C (вес 4)',
        description: 'Оно соединяет красную вершину B и
        type: 'info'
    }, ...prev]));
},
// Step 6: Include B-C
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро B-C в осто́ве',
        description: 'Вершина C перекрашена в красный ц
        type: 'success'
    }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{

```

```

        title: 'Шаг 6: Берем ребро E-F (вес 7)',
        description: 'Оно соединяет фиолетовую вершину I
        type: 'info'
    }, ...prev]));
},
// Step 12: Include E-F
() => {
    setVertices(prev => prev.map(v => v.id === 5 ? {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро E-F в осто́ве',
        description: 'Вершина F окрашена в фиолетовый ц
        type: 'success'
    }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Шаг 7: Берем ребро D-G (вес 8)',
        description: 'Оно соединяет фиолетовую вершину I
        type: 'info'
    }, ...prev]));
},
// Step 14: Include D-G
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-G в осто́ве',
        description: 'Вершина G окрашена в фиолетовый.'
        type: 'success'
    }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{

```

```

        description: 'Оба конца E и H уже фиолетовые. С
        type: 'warning'
    }, ...prev]);
},
// Step 20: Reject E-H
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Выбрасываем ребро E-H.',
        type: 'error'
    }, ...prev]));
},
// Step 21: Inspect H-I (12)
() => {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Шаг 11: Берем ребро H-I (вес 12)',
        description: 'Соединяет фиолетовую H и последнюю
        type: 'info'
    }, ...prev]));
},
// Step 22: Include H-I
() => {
    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Успех: MST построено Краскалом!',
        description: 'Все 9 вершин окрашены в единый фи
            + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
        type: 'success'
    }, ...prev]));
},
];

// --- PRIM STEPS ---
const primSteps = [
    // Step 1: Start at E

```

```

        type: 'info'
    }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 0 ? {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
        eap' } : e));
    setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
    setLogs(prev => [{
        title: 'Успех: Плесень поглотила вершину A',
        description: 'Новое ребро A-B(2) добавлено в кучу',
        type: 'success'
    }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
        description: 'Плесень стремительно бежит к вершине A',
        type: 'info'
    }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));
    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)
    setLogs(prev => [{
        title: 'Успех: Вершина B захвачена плесенью',
        description: 'В кучу добавлено B-C(4). Ребро B-E(6)
        type: 'success'
    }, ...prev]);
},
// Step 8: Pop B-C (4)
() => {

```

```
// Step 12: Pop E-F (7)
() => {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
  setLogs(prev => [{
    title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
    description: 'Плесень из центра Токио (E) тянет',
    type: 'info'
  }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
  setVertices(prev => prev.map(v => v.id === 5 ? {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
    eap' } : e));
  setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
  setLogs(prev => [{
    title: 'Успех: Вершина F захвачена',
    description: 'Ребро F-I(13) добавлено в кучу. Р
    type: 'success'
  }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
    description: 'Плесень расширяется вниз к G.',
    type: 'info'
  }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
  setVertices(prev => prev.map(v => v.id === 6 ? {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
    eap' } : e));
  setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
  setLogs(prev => [{
    title: 'Успех: Вершина G захвачена',
```

```

        setLogs(prev => [{
            title: 'Успех: Вершина H захвачена',
            description: 'В кучу добавлено H-I(12).',
            type: 'success'
        }, ...prev]));
    },
    // Step 20: Pop E-H (11) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{
            title: 'Шаг 11: Куча выдает E-H (вес 11)',
            description: 'Обе вершины E и H уже в плесени.',
            type: 'warning'
        }, ...prev]));
    },
    // Step 21: Reject E-H
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setHeapQueue(['H-I (12)', 'F-I (13)']));
        setLogs(prev => [{
            title: 'Отказ: Игнорируем E-H',
            description: 'Выбрасываем из кучи.',
            type: 'error'
        }, ...prev]));
    },
    // Step 22: Pop H-I (12)
    () => {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
            title: 'Шаг 12: Куча выдает H-I (вес 12)',
            description: 'Плесень тянется к последней чистой',
            type: 'info'
        }, ...prev]));
    },
    // Step 23: Include H-I
    () => {
        setVertices(prev => prev.map(v => v.id === 8 ? {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {

```



```

// Component 1: {A, B, C} (colored red)
// Component 2: {D, E, F, G, H, I} (colored gold)
setVertices(prev => prev.map(v =>
  [0, 1, 2].includes(v.id) ? { ...v, color: 'red' }
));
setEdges(prev => prev.map(e => {
  if (['0-1', '1-2'].includes(e.id)) {
    return { ...e, status: 'included', color: 'red' }
  }
  if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
    return { ...e, status: 'included', color: 'blue' }
  }
  return e;
}));
setLogs(prev => [{
  title: 'Слияние: Деревни объединились в колхозы',
  description: 'Все выбранные дороги построены радом с {D, E, F, G, H, I}.',
  type: 'success'
}, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between two kolkhozes)
() => {
  // Now both kolkhozes look at all outgoing roads
  // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}
  // A-D (5), B-E (6), C-F (9).
  // Cheapest is A-D (5).
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
    status: 'included', color: 'red'
  } : e));
  setLogs(prev => [{
    title: 'Вторая пятилетка: Колхозы выбирают мост',
    description: 'Теперь Красный и Синий колхозы объединились в один колхоз. Самый дешевый мост A-D с весом 5. Остальные мосты B-E(6) и C-F(9) остаются в планах.',
    type: 'warning'
  }, ...prev]);
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
  setVertices(prev => prev.map(v => ({ ...v, color: 'blue' })));
  setEdges(prev => prev.map(e => ({ ...e, status: 'included', color: 'blue' })));
  setLogs(prev => [{
    title: 'Завершение: Все деревни объединены в один колхоз',
    description: 'Все дороги построены. Колхозы объединены в один колхоз.',
    type: 'success'
  }, ...prev]);
}
}

```

```

    setHeapQueue([]);
    if (algo === 'kruskal') {
        setLogs([
            {
                title: 'Введение в раскраску (Краскал)',
                description: 'Представь, что все 9 вершин графа
                    уже покрашены, и тебе нужно выбрать
                    ребро (Heap) и захватываем соседей!',
                type: 'info',
            },
        ]);
    } else if (algo === 'prim') {
        setLogs([
            {
                title: 'Введение в Плесень (Прима)',
                description: 'Логика «Плесени»: выбираем стар
                    едь (Heap) и захватываем соседей!',
                type: 'info',
            },
        ]);
    } else {
        setLogs([
            {
                title: 'Введение в Коллективизацию (Борувка)',
                description: 'Логика «Борувки»: Каждая из 9 д
                    ет выбрать соседа, чтобы объединиться в колхоз.',
                type: 'info',
            },
        ]);
    }
};

```

```

const getColorClass = (color: string, id: number) => {
    switch (color) {
        case 'red': return 'bg-red-500 border-red-300 tex
        case 'blue': return 'bg-blue-500 border-blue-300
        case 'purple': return 'bg-purple-600 border-purple
        case 'mold': return 'bg-emerald-500 border-emerald
        default:
            if (activeAlgo === 'prim' && id === 4 && stepIn

```

```

    { /* Algorithm Tabs */ }
    <div className="flex flex-wrap items-center gap-1" />
    <div className="flex items-center bg-slate-400 p-2" />
    <button
      onClick={() => handleReset('kruskal')}
      className={
        "flex items-center gap-1.5 px-3 py-2"
      +
        (activeAlgo === 'kruskal'
          ? 'bg-indigo-600 text-white shadow-md'
          : 'text-slate-400 hover:text-slate-300')
      >
      <GitGraph className="w-3.5 h-3.5" />
      <span>Краскал (Билет 18)</span>
    </button>
    <button
      onClick={() => handleReset('prim')}
      className={
        "flex items-center gap-1.5 px-3 py-2"
      +
        (activeAlgo === 'prim'
          ? 'bg-emerald-600 text-white shadow-md'
          : 'text-slate-400 hover:text-slate-300')
      >
      <Layers className="w-3.5 h-3.5" />
      <span>Прима (Билет 19)</span>
    </button>
    <button
      onClick={() => handleReset('boruvka')}
      className={
        "flex items-center gap-1.5 px-3 py-2"
      +
        (activeAlgo === 'boruvka'
          ? 'bg-rose-600 text-white shadow-md'
          : 'text-slate-400 hover:text-slate-300')
      >

```

```
<div className="lg:col-span-2 bg-slate-950 rounded-2xl p-4 active min-h-[480px] overflow-hidden">
```

```
  { /* Legend */ }
```

```
  <div className="absolute top-4 left-4 bg-slate-950 p-4 text-center gap-3 text-xs">
```

```
    <div className="flex items-center gap-1.5">
```

```
      <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
    </div>
```

```
    {activeAlgo === 'kruskal' ? (
```

```
      <>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
        </div>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
        </div>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
        </div>
```

```
      </>
```

```
    ) : activeAlgo === 'prim' ? (
```

```
      <>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-1 bg-sky-600">
```

```
        </div>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
        </div>
```

```
      </>
```

```
    ) : (
```

```
      <>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
        </div>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
        </div>
```

```

        fill={isInspecting ? '#f59e0b' :
: '#94a3b8'}
        fontSize="4.5"
        fontFamily="monospace"
        fontWeight="bold"
        textAnchor="middle"
        dominantBaseline="middle"
        className="select-none filter drop
dy="-1.5"
    >
        {edge.weight}
    </text>
</g>
    );
  }}}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-f
  {vertices.map(vertex => {
    <div
      key={vertex.id}
      style={{ top: vertex.y + "%", left: v
      className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
    >
      <span>{vertex.label}</span>
      {vertex.id === 4 && activeAlgo === 'p
        <span className="text-[8px] block -
      }}
    </div>
  )}}
</div>

```

```

{stepIndex >= currentSteps.length && {
  <div className="absolute inset-x-6 bottom
shadow-xl z-20">

```

```

    <div className="flex-1 p-4 overflow-y-auto"
      {logs.map((log, idx) => {
        <div
          key={idx}
          className={
            "p-4 rounded-xl border transition-a
            {log.type === 'success'
              ? 'bg-emerald-950/40 border-emera
              : log.type === 'warning'
              ? 'bg-amber-950/40 border-amber-5
              : log.type === 'error'
              ? 'bg-rose-950/40 border-rose-500
              : 'bg-slate-900/60 border-slate-7
            }
          }
        >
          <div className="flex items-center gap
            {log.type === 'success' && <CheckCi
            {log.type === 'warning' && <HelpCir
            {log.type === 'error' && <XCircle c
            {log.type === 'info' && <Play class
            <h5 className="font-bold text-sm le
          </div>
          <p className="text-xs text-slate-300
        </div>
      })}
    </div>
  </div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-sla
  <div className="flex flex-col sm:flex-row items
    <div className="flex items-center gap-2.5">
      <BookOpen className="w-5 h-5 text-indigo-40
      <h4 className="text-xl font-bold text-white
    </div>

```

```

        <Award className="w-3.5 h-3.5 text-indigo-500">
    </p>
    <p className="text-2xl font-black text-indigo-500">
    <p className="text-slate-400 text-xs mt-1">
</div>
<div className="bg-slate-950 p-4 rounded-2xl">
    <p className="text-slate-400 text-xs font-medium">
ожении:</p>
    <p className="text-xs text-slate-300 leading-relaxed">
ди и красим вершины в один цвет, бракуя циклы.
    </div>
</div>
<div className="lg:col-span-2">
    <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
            <Code className="w-3.5 h-3.5 text-indigo-500">
            </p>
            <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

# 2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:
    if union(u, v):
        mst.append((u, v, w))`}
```

```
def prim(graph, start):
    visited = [False] * len(graph)
    heap = [(0, start, -1)] # (weight, node, parent)
    mst = []

    while heap:
        w, u, prev = heapq.heappop(heap)
        if visited[u]: continue
        visited[u] = True
        if prev != -1: mst.append((prev, u, w))

        for next_w, v in graph[u]:
            if not visited[v]:
                heapq.heappush(heap, (next_w, v, u))
    return mst`}
```

</pre>

</div>

</div>

</div>

}}

{activeCheatTab === 'boruvka' && {

<div className="grid grid-cols-1 lg:grid-cols-

<div className="lg:col-span-1 space-y-4">

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Clock className="w-3.5 h-3.5 text-ro

</p>

<p className="text-2xl font-black text-r

<p className="text-slate-400 text-xs mt

p>

</div>

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Award className="w-3.5 h-3.5 text-ro

</p>

<p className="text-2xl font-black text-r



```
    </div>
  );
};
```

---

ФАЙЛ 54/87: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const cards = [
  {
    img: dijkstraImg,
    alt: 'Добрый робот Дейкстра',
    title: '    Добрый (Дейкстра)',
    desc: 'Быстрый, жадный, но работает только с',
    highlight: 'положительными',
    highlightColor: 'text-emerald-400',
    rest: 'весами. Подсунь отрицательное ребро – сломает',
    complexity: 'O((V+E) log V)',
    border: 'border-blue-500/30 hover:border-blue-400/60',
    titleColor: 'text-blue-400',
    badge: 'text-blue-400/70 bg-blue-950/40',
  },
  {
    img: bellmanImg,
    alt: 'Фура по болоту Форд-Беллман',
    title: '    Фура по Болоту (Ф-Б)',
    desc: 'Медленный, тяжёлый – зато тащит',
    highlight: 'отрицательные',
    highlightColor: 'text-rose-400',
    rest: 'веса и детектит отрицательные циклы.',
    complexity: 'O(V · E)',
    border: 'border-rose-500/30 hover:border-rose-400/60',
```

}

---

ФАЙЛ 55/87: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";
```

```
interface Props {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Заголовок текущей темы — показываем в кнопке, что */
  currentTitle: string;
  index: number;
  total: number;
}
```

```
/**
```

```
 * Мобильное содержание: липкая панель снизу-сверху + в
```

```
 * На десктопе не рендерится (там обычный сайдбар).
```

```
 */
```

```
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
  const [open, setOpen] = useState(false);
```

```
  useEffect(() => {
    document.body.style.overflow = open ? "hidden" : "";
    return () => {
      document.body.style.overflow = "";
    };
  }, [open]);
```

```
  useEffect(() => {
    const onKey = (e: KeyboardEvent) => e.key === "Escape" ?
    window.addEventListener("keydown", onKey);
```

```
        <div className="flex-1 min-h-0">
          <Sidebar
            selectedChapterId={selectedChapterId}
            setSelectedChapterId={setSelectedChapterId}
            onNavigate={() => setOpen(false)}
          />
        </div>
      </div>
    </div>
  )}
</>
);
};
```

---

ФАЙЛ 56/87: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cpu } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator';
  setActiveTab: (tab: 'guide' | 'simulator') => void;
  /** Открыта ли боковая панель компилятора. */
  compilerOpen: boolean;
  onToggleCompiler: () => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab, setActiveTab, compilerOpen, onToggleCompiler }) => {
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
```

```

        <BookOpen className="w-4 h-4" /> Учебное по
</button>
<button
  id="tab-simulator-btn"
  onClick={() => setActiveTab('simulator')}
  className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
    activeTab === 'simulator'
      ? 'bg-indigo-600 text-white shadow-lg s
      : 'text-slate-400 hover:text-white'
  }`}
>
  <Cpu className="w-4 h-4" /> Тренажёры
</button>
<button
  id="tab-compiler-btn"
  onClick={onToggleCompiler}
  aria-pressed={compilerOpen}
  title="Боковая панель Python-компилятора (на
    страницей"
  className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
    compilerOpen
      ? 'bg-emerald-600 text-white shadow-lg
      : 'text-slate-400 hover:text-white'
  }`}
>
  <Terminal className="w-4 h-4" /> Python
</button>
<a
  id="download-pdf-btn"
  href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
target="_blank"
rel="noreferrer"
  className="flex items-center justify-center
over:bg-emerald-600/20 border border-emeral
title="Скачать полный PDF сборник всех стра

```

```

return (
  <div className="space-y-4">
    <h4 className="text-base font-bold text-white">K5
    <p className="text-xs text-slate-400">Просто смотри

  <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
    { /* K5 */ }
    <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
      <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
      <svg className="w-full h-full absolute inset-0">
        {(() => {
          const pts = [
            {id:0,x:160,y:40},
            {id:1,x:270,y:100},
            {id:2,x:230,y:230},
            {id:3,x:90,y:230},
            {id:4,x:50,y:100}
          ];
          const edges = [
            [0,1],[0,2],[0,3],[0,4],
            [1,2],[1,3],[1,4],
            [2,3],[2,4],
            [3,4]
          ];
          function findPt(id:number) { return pts.find(p => p.id === id); }
          function intersect(a:number,b:number,c:number,d:number) {
            if (a===c||a===d||b===c||b===d) return true;
            const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
            function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
              return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
            }
            return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
              && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
          }
          const crosses: number[] = [];
          for(let i=0;i<edges.length;i++) {
            for(let j=i+1;j<edges.length;j++) {

```

```

];
const edges = [
  [0,3],[0,4],[0,5],
  [1,3],[1,4],[1,5],
  [2,3],[2,4],[2,5]
];
function findPt(id:number) { return pts.f
function intersect(a:number,b:number,c:nu
  if (a===c||a===d||b===c||b===d) return
  const p1=findPt(a),p2=findPt(b),p3=find
  if (!p1||!p2||!p3||!p4) return false;
  function ccw(ax:number,ay:number,bx:num
    return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
  }
  return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.
    && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
}
const crosses:number[]=[];
for(let i=0;i<edges.length;i++) for(let j
  if (intersect(edges[i][0],edges[i][1],e
    if (!crosses.includes(i)) crosses.push
    if (!crosses.includes(j)) crosses.push
  }
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
  const p1=findPt(edges[i][0]),p2=findPt(
  const xing=crosses.includes(i);
  lines.push(<line key={`l${i}`} x1={p1.x
    stroke={xing?"#f43f5e":"#4f46e5"} stro
}
const labels = ["\u04141","\u04142","\u04
for(const p of pts) {
  circles.push(<g key={`c${p.id}`}>
    <circle cx={p.x} cy={p.y} r={8} fill=
    <text x={p.x} y={p.y+3} textAnchor="m
  </g>);
}

```

```

    Play,
    Plus,
    RotateCcw,
    SkipBack,
    SkipForward,
    Square,
    Terminal,
    Unlink,
    Variable,
    X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { emitVizStep, onVizDemo, vizHitsAtTrace, vizSte
import {
    baseVarName,
    buildDebugRunner,
    changedVars,
    DEBUG_STEP_CAP,
    orderVars,
    type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "../Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo

interface PythonCompilerProps {
    /** Страница, с которой синхронизируется редактор. */
    chapterId: string;
    chapterTitle: string;
    /** Перейти к странице пособия (посмотреть визуализац
    onOpenGuide: () => void;
    /** Скрыть панель. */
    onClose: () => void;
}

type PyodideRuntime = {

```

```

    tip: string;
    code: string;
}

const SNIPPETS: { group: string; items: Snippet[] }[] =
[
  {
    group: "Сортировка",
    items: [
      {
        label: "arr.sort() / sorted()",
        tip: "In-place сортировка по возрастанию и новая",
        code: 'arr = [5, 2, 8, 1]\narr.sort()
              ) # копия по убыванию\n',
      },
      {
        label: "sorted(key=...)",
        tip: "Сортировка по ключу: рёбра по весу, пары",
        code: 'edges = [(3, "A", "B"), (1, "B", "C"), (
              es)\n',
      },
    ],
  },
  {
    group: "Циклы",
    items: [
      {
        label: "for i in range(n)",
        tip: "Вложенные циклы по i и j – как обход матри",
        code: "n, m = 3, 4\nfor i in range(n):",
      },
      {
        label: "while ...",
        tip: "Классический while-счётчик – как n-1 итер",
        code: "i = 1\nwhile i < 5:\n    print(f\"итераци",
      },
      {
        label: "for i, x in enumerate(...)",
        tip: "Индекс и элемент одновременно: i – позици",

```



```

        items: [
            {
                label: "f-строка",
                tip: "Подставить значения переменных шага в строку",
                code: 'i, j, k = 1, 2, 0\nprint(f"dist[{i}][{j}]", dist[i][j])',
            },
        ],
    },
];

```

```

/** Переживает скрытие панели: код пользователя не теряется
let savedEditor: { code: string; template: string; chapterId: string };

```

```

export function PythonCompiler({ chapterId, chapterTitle }) {
    /** Активная вкладка демонстрации страницы (у sparse-страницы)
    const [demoId, setDemoId] = useState<string | null>(null);
    const sync = useMemo(() => getPageSync(chapterId, demoId));
    const syncVarBases = useMemo(() => sync.variables.map((v) => v.bases));
    const syncVarSet = useMemo(() => new Set(syncVarBases));
    /** Полный эталонный код – подсматривается «глазом», не редактируется
    const template = useMemo(() => buildSyncTemplate(chapterId, sync));
    /** Дефолт редактора: шапка + только инициализированные переменные
    const initTemplate = useMemo(() => buildInitTemplate(chapterId));
    /** Помеченные строки эталона по содержимому: каждое слово – это код
    const markedContents = useMemo(() => {
        const marks = sync.stepCodeLines;
        if (!marks || marks.length === 0) return undefined;
        const lines = template.split("\n");
        return new Set(marks.map((l) => (lines[l + 2] ?? "")));
    }, [sync, template]);
    /** Режим «подсмотреть эталон»: показывает полный код
    const [showRef, setShowRef] = useState(false);

    const [code, setCode] = useState(() => {
        if (!savedEditor) return initTemplate;
        if (savedEditor.code === savedEditor.template || savedEditor.code === "")
            return savedEditor.code;
        return savedEditor.code;
    });

```

```

useEffect(() => {
  const prev = prevTplRef.current;
  if (initTemplate === prev.base && template === prev.full) {
    prevTplRef.current = { base: initTemplate, full: template };
    setPlaying(false);
    setDebug(null); // трасса ссылается на строки старого кода
    setShowRef(false);
    if (code === prev.base || code === prev.full || code === prev.full) {
      setCode(initTemplate);
      setDesyncedFrom(null);
    } else {
      setDesyncedFrom(chapterId);
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [initTemplate, template]);

const resync = useCallback(() => {
  prevTplRef.current = { base: initTemplate, full: template };
  setCode(initTemplate);
  setDesyncedFrom(null);
  setStripTab("vars");
  setPlaying(false);
  setDebug(null);
  setShowRef(false);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
  try {
    await navigator.clipboard.writeText(showRef ? template : code);
    setCopied(true);
    setTimeout(() => setCopied(false), 1600);
  } catch {
    setRuntimeMessage("Не получилось скопировать – буфер переполнен");
  }
}, [code, showRef, template]);

```

```

        return { ...d, idx };
    });
},
[chapterId, markedContents, stepCapable, vizActiveNow]);

const stepBy = useCallback(
  (delta: number) => {
    setDebug((d) => {
      if (!d) return d;
      const idx = Math.max(0, Math.min(d.result.steps
      if (vizActiveNow(d.src)) emitVizStep(chapterId,
      return { ...d, idx };
    });
  },
[chapterId, markedContents, stepCapable, vizActiveNow]);

const stepTo = useCallback((idx: number) => goDebug(idx));

const stopDebug = useCallback(() => {
  setPlaying(false);
  setDebug(null);
}, []);

/** Интервал автопрохода шагов (мс). */
const AUTOPLAY_MS = 650;

// Автопроход: пока включен, шагает stepBy каждые AUTOPLAY_MS
useEffect(() => {
  if (!playing || !debug) return;
  if (debug.idx >= debug.result.steps.length - 1) {
    setPlaying(false);
    return;
  }
  const t = window.setTimeout(() => stepBy(1), AUTOPLAY_MS);
  return () => window.clearTimeout(t);
}, [playing, debug, stepBy]);

```

```

    } finally {
      setRunning(false);
    }
  }, [code, stdin, running, runtimeReady, chapterId, ma

// Стрелки ← → в режиме отладки листают ШАГИ (а не стр
useEffect(() => {
  if (!debug) return;
  const onKey = (e: KeyboardEvent) => {
    const target = e.target as HTMLInputElement | null;
    if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target))
    if (e.metaKey || e.ctrlKey || e.altKey) return;
    if (e.key === "ArrowRight") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying(false);
      stepBy(1);
    }
    if (e.key === "ArrowLeft") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying(false);
      stepBy(-1);
    }
    if (e.key === " ") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying((v) => !v);
    }
    if (e.key === "Escape") stopDebug();
  };
  // capture-фаза: чтобы глобальный обработчик навигац
  window.addEventListener("keydown", onKey, true);
  return () => window.removeEventListener("keydown", onKey);
}, [debug, stepBy, stopDebug]);

// держим текущую строку листинга в видимой области
const curDebug = debug ? { step: debug.result.steps[d]

```

```

aria-label="Python-компилятор"
className="fixed z-40 inset-x-0 bottom-0 h-[58dvh]
  px] flex flex-col bg-slate-900 border-t lg:bord
>
{ /* — Заголовок панели —————
<div className="shrink-0 flex items-center gap-2 p-2"
  <span className={`w-2 h-2 rounded-full shrink-0`}
  <Terminal className="w-4 h-4 text-emerald-400" style={{
  <span className="text-[13px] font-bold text-white" style={{
  <span className="hidden sm:inline text-[10px] text-white" style={{
  <div className="ml-auto flex items-center gap-1"
    <Tooltip side="bottom" content="Скопировать код"
      <button
        type="button"
        onClick={() => void copyCode()}
        className="p-1.5 rounded-lg text-slate-400"
        aria-label="Скопировать код"
      >
        {copied ? <Check className="w-4 h-4 text-emerald-400" /> : <Copy />}
      </button>
    </Tooltip>
    <Tooltip side="bottom" content="Вернуть комментарий"
      <button
        type="button"
        onClick={resync}
        className="p-1.5 rounded-lg text-slate-400"
        aria-label="Сбросить к шаблону страницы"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
    </Tooltip>
    <Tooltip side="bottom" content="Скрыть панель"
      <button
        type="button"
        onClick={onClose}
        className="p-1.5 rounded-lg text-slate-400"
        aria-label="Заккрыть компилятор"

```

```

    </div>
  </div>

  /* — Мини-вкладки: переменные страницы / шпаргалка
  <div className="shrink-0 border-b border-slate-800" >
    <div className="flex items-center gap-1 px-2 pt-2" >
      <button
        type="button"
        onClick={() => setStripTab("vars")}
        className={`inline-flex items-center gap-1.5
          stripTab === "vars" ? "bg-slate-800 text-white" : "text-slate-400"
        }` >
        >
        <Variable className="w-3.5 h-3.5" /> Переменные
      </button>
      <button
        type="button"
        onClick={() => setStripTab("hints")}
        className={`inline-flex items-center gap-1.5
          stripTab === "hints" ? "bg-slate-800 text-white" : "text-slate-400"
        }` >
        >
        <HelpCircle className="w-3.5 h-3.5" /> Подсказки
      </button>
      <div className="ml-auto pr-1" >
        <Tooltip
          side="bottom"
          content={
            hasViz
              ? "Редактор и визуализация смотрят на выбранную страницу"
              : "На выбранной странице нет интерактивных элементов"
          }
        >
        <span
          tabIndex={0}
          className={`cursor-help inline-flex items-center gap-1.5
            hasViz ? "text-emerald-400" : "text-slate-400"
          }` >
            >

```

```

    })
    {hasViz && {
      <div className="flex flex-wrap items-center">
        {sync.variables.map((v) => {
          <Tooltip
            key={v.name}
            side="bottom"
            content={
              <span>
                <span className="block font-bold">
                  <code className="font-mono text-emerald-300">{v.name}</code>
                </span>
                <span className="block">{v.role}</span>
                {v.range && <span className="block text-emerald-300">{v.range}</span>
              </span>
            }
          >
            <code
              tabIndex={0}
              className="cursor-help rounded-md font-bold text-emerald-300 hover:bg-emerald-300"
            >
              {v.name}
            </code>
          </Tooltip>
        })}
      {sync.stepNote && {
        <Tooltip side="bottom" content={`War r
          <span
            tabIndex={0}
            className="cursor-help inline-flex text-[10px] text-slate-400 hover:text-white"
          >
            <ArrowRightLeft className="w-3 h-3" />
            <span className="truncate max-w-50">{v.note}</span>
          </span>
        </Tooltip>
      }}
    }
  }
}

```

```

        ново (пробел)" : "Продолжить автопроход (пр
        <button
            type="button"
            onClick={() => {
                if (!curDebug?.step) return;
                if (playing) setPlaying(false);
                else {
                    if (debug.idx >= debug.result.steps
                        setPlaying(true);
                }
            }}
            className={`mx-0.5 p-1.5 rounded-md t
        : "text-emerald-400 bg-emerald-500/10 hove
            aria-label={playing ? "Пауза автопрох
        >
            {playing ? <Pause className="w-4 h-4"
        </button>
    </Tooltip>
    <Tooltip content="В начало трассы">
        <button type="button" onClick={() => {
            md text-slate-400 hover:text-white hover:bg
                <SkipBack className="w-4 h-4" />
            </button>
        </Tooltip>
        <Tooltip content="Шаг назад (←) – автопро
            <button type="button" onClick={() => {
            -md text-slate-400 hover:text-white hover:b
                <ChevronLeft className="w-4 h-4" />
            </button>
        </Tooltip>
        <Tooltip content="Шаг вперёд (→) – автопр
            <button type="button" onClick={() => {
            sult.steps.length - 1} className="p-1 round
            tion-colors">
                <ChevronRight className="w-4 h-4" />
            </button>
        </Tooltip>
        <Tooltip content="В конец трассы">

```



```

        : vizLink === "content"
        ? "Вы дописали тело сами, но по
эталоном."
        : vizLink === "partial"
        ? "В коде только строки иници
одом не ходит — допишите остальные помеченн
        : "В коде нет помеченных стро
их дословно или нажмите «Сбросить к шаблон
    }
  >
  <span
    tabIndex={0}
    className={`cursor-help inline-flex i
      stepCapable && vizLink !== "none"
      ? vizLink === "partial"
      ? "border-amber-500/40 bg-amber
      : "border-emerald-500/40 bg-eme
      : "border-slate-600 bg-slate-800
    }`}
  >
    {stepCapable && vizLink !== "none" ? .
    {stepCapable
      ? vizLink === "exact"
      ? "демо следует (эталон)"
      : vizLink === "content"
      ? "демо следует (ваши строки)"
      : vizLink === "partial"
      ? "демо: инициализация"
      : "демо отвязано"
      : "демо ручное"}
    </span>
  </Tooltip>
  <Tooltip content="Выход из отладчика (Esc
    <button type="button" onClick={stopDebu
00/30 transition-colors">
    <Square className="w-3.5 h-3.5" />
    </button>
  </Tooltip>

```

```

        {debugOrdered.length === 0 && !debug.result}
        <div className="mt-1 text-[11px] text-slate-500"
      /div>
    ))
    <div className="mt-1 flex flex-wrap gap-1"
      {debugOrdered.map(([name, value]) => {
        const changed = debugChanged[name];
        const synced = syncVarSet.has(name);
        return (
          <span
            key={name}
            className={`inline-flex items-baseline gap-1
              ${changed
                ? "border-amber-500/50 bg-amber-500/10"
                : synced
                ? "border-emerald-500/40 bg-emerald-500/10"
                : "border-slate-700 bg-slate-700/10"}
            }`
          >
            <span className={changed ? "text-amber-500" : "text-slate-500"}>
              {name}
            </span>
            <span className="text-slate-500">{value}</span>
            <span className={changed ? "text-amber-500" : "text-slate-500"}>
              {syncVarSet.get(name)}
            </span>
          </span>
        );
      })}
    </div>
  </div>
) : showRef ? (
  <div className="flex-1 flex min-h-[70px] flex-wrap"
    <div className="shrink-0 flex items-center justify-center gap-1"
      <Eye className="w-3.5 h-3.5 text-indigo-400" />
      <span className="text-[10.5px] font-bold text-slate-500">
        Эталонный код этого демо • только чтение
      </span>
      <div className="ml-auto flex items-center justify-center gap-1"

```

```

        className="flex-1 min-h-[90px] w-full resize-
        ine-none focus:ring-2 focus:ring-inset focus:
        placeholder="Напишите Python-код..."
    />
  })
  <div className="shrink-0 border-t border-slate-
    <label htmlFor="python-stdin" className="text
      stdin для input()
    </label>
    <input
      id="python-stdin"
      value={stdin}
      onChange={(event) => setStdin(event.target.v
      spellCheck={false}
      placeholder="строки ввода через ↵"
      className="w-full bg-transparent font-mono
    />
  </div>
</div>

{/* — Вывод —————
<div className="shrink-0 border-t border-slate-800
  <button
    type="button"
    onClick={() => setOutputOpen((o) => !o)}
    className="flex items-center justify-between
      colors"
  >
    <span className="inline-flex items-center gap-2
      <Terminal className="w-3.5 h-3.5 text-emera
    </span>
    <span className="inline-flex items-center gap-2
      {runtimeReady && <CheckCircle2 className="w-4
      <span className="text-slate-600">{outputOpen
    </span>
  </button>
  {outputOpen && {
    <pre className="flex-1 min-h-[70px] overflow-

```

```

        пливо для нашего НЛО!' }},
    { name: 'Бизнесмен',      emoji: ' ', color: 'from-slate-500',
      тартапы.' }},
  ];

let counter = 3;

export const QueueViz: React.FC = () => {
  const [queue, setQueue] = useState<Customer[]>([
    { id: 'q1', name: 'Студент Вася', emoji: ' ', color: 'from-slate-500',
      голода после матана!', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён',      emoji: ' ', color: 'from-slate-500',
      ишу ИИ за порцию борща!', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис',      emoji: ' ', color: 'from-slate-500',
      ез сметаны, плиз.' }, animState: 'idle' },
  ]));

  const [servedHistory, setServedHistory] = useState<string[]>([]);
  const [isServing, setIsServing] = useState(false);
  const [shakeEmpty, setShake] = useState(false);
  const [log, setLog] = useState<string[]>([]);

  const addLog = (msg: string) => setLog(prev => [...prev, msg]);

  // ENQUEUE: Встать в конец очереди (хвост / Tail)
  const enqueue = useCallback(() => {
    if (isServing) return;
    if (queue.length >= 6) {
      addLog('⚠ Хвост очереди уже на улице, повару тяжело!');
      setShake(true);
      setTimeout(() => setShake(false), 400);
      return;
    }
  }, [isServing, queue]);

  const preset = PRESETS[counter % PRESETS.length];
  counter++;

  const newGuest: Customer = {

```

```

        setQueue(prev => prev.slice(1));
        setIsServing(false);
        addLog(`  ${headGuest.name} поел и ушел сытым.`,
    }, 400);
    }, 900);
  }, [queue, isServing]);

```

```

const reset = () => {
  counter = 3;
  setQueue([
    { id: 'r1', name: 'Студент Вася', emoji: '🍴', color: 'red', text: 'Я  
т голода после матана!', animState: 'in' },
    { id: 'r2', name: 'Кодер Семён', emoji: '🍴', color: 'blue', text: 'Я  
апишу ИИ за порцию борща!', animState: 'in' },
    { id: 'r3', name: 'Кот Борис', emoji: '🐈', color: 'green', text: 'Я  
без сметаны, плиз.', animState: 'in' },
  ]);
  setServedHistory([]);
  setIsServing(false);
  setLog([' Столовая сброшена. Снова голодная толпа!']);
  setTimeout(() => {
    setQueue(prev => prev.map(c => ({ ...c, animState: 'out' })));
  }, 500);
};

```

```

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-indigo-950/40 border border-indigo-500 p-6">
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
        <p className="text-xs text-slate-300 mt-1 leading-4">
          В очереди за супом всё честно. Кто первый встал, тот и ест.
          Новые гости встают строго <b>в конец</b> очереди.
          Здесь нет обхода и блата — только строгий порядок.
        </p>
      </div>
    </div>
  </div>
)

```

```
-[300px] overflow-hidden ${shakeEmpty ? 'anim
<div className="absolute top-4 left-4 text-[1l
  Очередь голодных гостей (Буфер FIFO / BFS)
</div>
```

```
<div className="flex-1 flex flex-col justify-c
  <div className="flex items-end justify-betw
```

```
  {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
  <div className="flex flex-col items-cente
    <div className="relative">
      <div className="absolute -top-10 left
        <div className="w-1.5 h-6 bg-slate-
        <div className="w-2 h-4 bg-slate-40
      </div>
      <span className="text-5xl block anima
    </div>
    <div className="text-center">
      <span className="text-3xl block"> <
      <span className="text-[10px] font-bol
ase font-mono tracking-wider">
      ПОВАР
    </span>
  </div>
</div>
```

```
{/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}
<div className="flex flex-col items-cente
  <span className="text-2xl animate-pulse
  <span className="text-[8px] font-mono t
</div>
```

```
{/* СПИСОК ОЧЕРЕДИ */}
<div className="flex-1 flex items-end gap
  {queue.length === 0 ? {
    <div className="flex-1 flex flex-col
      <span className="text-4xl mb-1"> </
      <p className="text-xs font-bold fon
```

```

        {(isHead || isTail) && (
            <span className={`
                absolute -bottom-2 text-[
                    ${isHead ? 'bg-amber-500
                `}>
                {isHead && isTail ? 'h+t'
            </span>
        )}
    </div>
</div>
    );
    })
    })
</div>

</div>
</div>

    {/* Пол кухни */}
    <div className="w-full h-3 bg-gradient-to-r f
</div>

    {/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
    <div className="md:col-span-3 bg-slate-900 round
        <div className="absolute top-4 left-4 text-[1
            Сытые гости (Архив FIFO)
        </div>

        <div className="absolute top-4 right-4 flex i
            order-emerald-800/80 text-[9px] font-mono">
            <Sparkles className="w-3.5 h-3.5 animate-pu
            <span>СЫТЫ </span>
        </div>

        <div className="flex-1 flex flex-col justify-
            {servedHistory.length === 0 ? {
                <div className="flex-1 flex flex-col item
                    <span className="text-5xl mb-2"> </span>

```





```

        is_terminal: false,
        words: [],
        depth: newNodes[curr].depth + 1
    };
    }
    curr = newNodes[curr].children[c];
}
if (!newNodes[curr].words.includes(word)) {
    newNodes[curr].words.push(word);
    newNodes[curr].is_terminal = true;
}
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
    const childKeys = Object.values(newNodes[u].children);
    newNodes[u].y = 40 + depth * paddingY;

    if (childKeys.length === 0) {
        newNodes[u].x = 40 + currentX * paddingX;
        currentX++;
    } else {
        let leftX = -1;
        let rightX = -1;
        childKeys.forEach(v => {
            layoutDFS(v, depth + 1);
            if (leftX === -1) leftX = newNodes[v].x!;
            rightX = newNodes[v].x!;
        });
        newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX;
        if (leftX === -1) currentX++;
    }
};

layoutDFS(0, 0);

```

```
// Запуск BFS
const startBFS = () => {
  const rootChildren = Object.values(nodes[0].children);
  const queue = [...rootChildren];
  const updatedNodes = { ...nodes };
  rootChildren.forEach((childId) => {
    updatedNodes[childId].fail = 0;
  });

  setNodes(updatedNodes);
  setBfsQueue(queue);
  setSimulationStep(1);
  setCurrentNode(null);
  setSpeechText(
    'Начинаем обход в ширину (BFS)! Все вершины на графике  

    уже суффикса просто нет. Жми «Следующий шаг»!'
  );
};

// Один шаг BFS
const stepBFS = () => {
  if (bfsQueue.length === 0) {
    setSimulationStep(2);
    setCurrentNode(null);
    setSpeechText(
      'Ура! Очередь пуста! Мы успешно построили полный  

      граф. Теперь можно полюбоваться таблицей переходов!'
    );
    return;
  }

  const r = bfsQueue[0];
  const newQueue = bfsQueue.slice(1);
  const updatedNodes = { ...nodes };
  const rNode = updatedNodes[r];
  const childrenIds = Object.values(rNode.children);
```



```

        { /* Декоративные пузыри */ }
        <div className="absolute -top-1 right-4 text-slate-400">
        <div className="absolute top-8 right-1 text-slate-400">
        </div>
        <span className="mt-3 bg-rose-600/30 border border-slate-400 p-1">
        Эхо-Лосось Сеня
        </span>
    </div>

    { /* Пузырь с текстом (Баббл) */ }
    <div className="md:col-span-2 bg-slate-800 border border-slate-400 p-4
    tween min-h-[140px]">
        { /* Треугольник баббла */ }
        <div className="absolute -left-3 top-1/2 -translate-y-1/2
        slate-800 border-b-[12px] border-b-transparent">

        <div className="flex items-center space-x-2 text-slate-400">
            <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}` />
            <span>Прямое вещание из аквариума:</span>
        </div>

        <p className="text-slate-200 text-base leading-relaxed">
            {speechText}
        </p>

        <div className="mt-4 pt-3 border-t border-slate-400">
            <div className="text-xs text-slate-400 flex justify-between">
                <Info className="w-3.5 h-3.5 text-blue-400">
                    Статус: {simulationStep === 0 ? 'Бор готов к
                    оен!'}
                </div>

                <div className="flex gap-2">
                    {simulationStep === 0 && (
                        <button
                            onClick={startBFS}
                            className="bg-emerald-600 hover:bg-emerald-500
                            -1 shadow-lg shadow-emerald-900/40 transition-colors">

```

```

        if (n.x && n.x > maxX) maxX = n.x;
        if (n.y && n.y > maxY) maxY = n.y;
    });
    const svgWidth = Math.max(maxX + 80, 400);
    const svgHeight = Math.max(maxY + 60, 200);

    return (
      <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
        <defs>
          <marker id="arrowhead" markerWidth="6"
            <polygon points="0 0, 6 3, 0 6" fill=
          </marker>
          <marker id="fail-arrow" markerWidth="6"
            <polygon points="0 0, 6 3, 0 6" fill=
          </marker>
        </defs>

        {/* Draw Edges */}
        {Object.values(nodes).map(node => {
          if (node.id === 0) return null;
          const parent = nodes[node.parent];
          if (!parent.x || !parent.y || !node.x ||

          const isCurrent = currentNode === node.

          return (
            <g key={`edge-${node.id}`}>
              <line
                x1={parent.x}
                y1={parent.y + 16}
                x2={node.x}
                y2={node.y - 16}
                stroke={isCurrent ? '#60a5fa' : ''}
                strokeWidth="2"
                markerEnd="url(#arrowhead)"
              />
              <text
                x={({parent.x + node.x} / 2 - 10}

```

```

    { /* Draw Nodes */
    {Object.values(nodes).map(node => {
      if (!node.x || !node.y) return null;
      const isRoot = node.id === 0;
      const isCurrent = currentNode === node;
      const isInQueue = bfsQueue.includes(node);

      let fill = '#1e293b';
      let stroke = '#475569';

      if (isCurrent) {
        fill = '#2563eb';
        stroke = '#60a5fa';
      } else if (isInQueue) {
        fill = '#b45309';
        stroke = '#fbbf24';
      } else if (node.is_terminal) {
        fill = '#059669';
        stroke = '#34d399';
      }

      return (
        <g key={`node-${node.id}`}>
          <circle
            cx={node.x}
            cy={node.y}
            r="16"
            fill={fill}
            stroke={stroke}
            strokeWidth="2"
          />
          <text
            x={node.x}
            y={node.y + 4}
            fill="white"
            fontSize={isRoot ? "10" : "12"}
            fontWeight="bold"
            textAnchor="middle"
          />
        </g>
      );
    })}
  );
}

```

```

        className="text-slate-500 hover:text-slate-400"
        title="Удалить"
      >
        ✖
      </button>
    </span>
  )))
</div>
</div>

<form onSubmit={handleAddWord} className="flex flex-col gap-2" >
  <input
    type="text"
    value={newWord}
    onChange={(e) => setNewWord(e.target.value)}
    placeholder="НОВОЕ СЛОВО..."
    maxLength={8}
    className="bg-slate-800 border border-slate-700 border-radius-4px outline-none focus:border-blue-500"
  />
  <button
    type="submit"
    className="bg-slate-700 hover:bg-slate-600 text-white font-medium py-2 px-4 transition-colors"
  >
    <Plus className="w-3.5 h-3.5" /> Добавить
  </button>
</form>
</div>

{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
  <h5 className="text-white font-bold mb-3 flex items-center" >
    <span className="flex items-center gap-1.5" >
      <span> </span> Таблица переходов и fail-
    </span>
    <span className="text-xs text-slate-400 font-medium" >
      Всего вершин: {Object.keys(nodes).length}
    </span>
  </h5>
</div>

```

```

        ? 'bg-blue-400 animate-pulse'
        : isInQueue
        ? 'bg-amber-400'
        : 'bg-slate-600'
      )` }
    ></span>
    <span>#{node.id}</span>
    {node.id === 0 && <span className="
  </td>
  <td className="py-2.5 px-3">
    <span className="bg-slate-800 b
      {node.char}
    </span>
  </td>
  <td className="py-2.5 px-3 text-s
    {node.id === 0 ? '-' : `#${node
  </td>
  <td className="py-2.5 px-3">
    {node.id === 0 ? {
      <span className="text-slate-5
    } : {
      <span className="bg-indigo-95
        #{node.fail}
      </span>
    }}
  </td>
  <td className="py-2.5 px-3">
    {node.id === 0 || node.term_lin
      <span className="text-slate-5
    } : {
      <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Cк
      #{node.term_link}
    </span>
    }}
  </td>
  <td className="py-2.5 px-3 text-s
    {Object.keys(node.children).len

```



```

    desc: string;
    codeLine: number;
    treeState: Record<number, number>;           // node index
    highlightNodes: number[];                    // current nodes
    mergeChildren?: [number, number];           // pair being merged
    arrayHighlight?: [number, number];           // [L, R] range
    result?: number;                             // partial result
}

```

// ===== PURE LOGIC: build tree =====

```

function buildTreeFull(arr: number[]): Record<number, number> {
    const n = arr.length;
    const tree: Record<number, number> = {};
    function go(v: number, l: number, r: number) {
        if (l === r) { tree[v] = arr[l]; return; }
        const m = (l + r) >> 1;
        go(2 * v, l, m);
        go(2 * v + 1, m + 1, r);
        tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
    }
    go(1, 0, n - 1);
    return tree;
}

```

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)

```

function genBuildSteps(arr: number[]): Step[] {
    const n = arr.length;
    const steps: Step[] = [];
    let id = 0;
    const tree: Record<number, number> = {};

    const CODE = [
        "build(v, l, r) {",           // 1
        "    if (l === r) {",         // 2
        "        tree[v] = A[l]; return", // 3
        "    }",                     // 4
    ];
}

```

```

const steps: Step[] = [];
let id = 0;

function go(v: number, l: number, r: number, ql: number, qr: number) {
  if (ql > r || qr < l) {
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
      codeLine: 2, treeState: tree, highlightNodes: [v] });
    return 0;
  }
  if (ql <= l && r <= qr) {
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
      codeLine: 4, treeState: tree, highlightNodes: [v] });
    return tree[v];
  }
  const m = (l + r) >> 1;
  steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
    codeLine: 6, treeState: tree, highlightNodes: [v] });
  const left = go(2 * v, l, m, ql, qr);
  const right = go(2 * v + 1, m + 1, r, ql, qr);
  const res = left + right;
  steps.push({ id: id++, desc: `Возврат в узел ${v}:`,
    codeLine: 8, treeState: tree, highlightNodes: [v],
    rangeChildren: [2 * v, 2 * v + 1], arrayHighlight: [2 * v, 2 * v + 1] });
  return res;
}

const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]`,
  codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans });
return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

```

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
  const n = arr.length;
  // Build iterative tree: leaves at [n..2n-1]
  const tree: Record<number, number> = {};
  for (let i = 0; i < n; i++) tree[n + i] = arr[i];
  for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);
}

```

```
-----
interface VNode { v: number; l: number; r: number; val:
```

```
function buildVisualTree(treeState: Record<number, number>) {
  if (l === r) return { v, l, r, val: treeState[v] ?? null };
  const m = (l + r) >> 1;
  return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
    treeState, v + 1, m + 1, r) };
}
```

```
// ===== CODE SNIPPETS =====
```

```
const BUILD_CODE = [
  "build(v, l, r) {",
  "  if (l === r) { tree[v] = A[l]; return; }",
  "  // -----",
  "  // -----",
  "  mid = (l + r) >> 1;",
  "  build(2*v, l, mid);",
  "  build(2*v+1, mid+1, r);",
  "  tree[v] = tree[2*v] + tree[2*v+1];",
  "}"
];
```

```
const QUERY_TD_CODE = [
  "query(v, l, r, ql, qr) {",
  "  if (ql > r || qr < l) return 0;",
  "  // -----",
  "  if (ql <= l && r <= qr) return tree[v];",
  "  // -----",
  "  mid = (l + r) >> 1;",
  "  // спускаемся в обоих детей",
  "  return query(left) + query(right);",
  "}"
];
```

```
const QUERY_BU_CODE = [
  "queryBU(ql, qr) {",
  "  res = 0;",
  "  l = ql + n; r = qr + n + 1;",
  "  // -----",
  "  while (l < r) {",
```

```

const p = Player({ steps, color });
if (!p) return null;
const { step, idx, setIdx, playing, setPlaying, speed } = p;
const n = arr.length;

const vTree = useMemo(() => buildVisualTree(step.tree), [step]);

const renderNode = useCallback((nd: VNode): React.ReactNode => {
  const isHigh = step.highlightNodes.includes(nd.v);
  const isChild = step.mergeChildren?.includes(nd.v);
  const has = nd.val !== null;

  let cls = 'bg-slate-800 border-slate-700 text-slate-200';
  if (isHigh) cls = `${clr.ring} text-white ring-2 shadow-2xl`;
  else if (isChild) cls = `${clr.childRing} text-amber-400 ring-1`;
  else if (has) cls = `${clr.filled} text-slate-200`;

  return (
    <div key={nd.v} className="flex flex-col items-center justify-center">
      <div className={`flex flex-col items-center justify-center ${cls}`}>
        <span className="text-[7px] opacity-60 font-mono">{nd.val}</span>
        <span className="text-[9px] font-bold">{nd.l}</span>
        <span className="text-xs font-extrabold font-mono">{nd.r}</span>
      </div>
      {(nd.left || nd.right) && (
        <div className="flex flex-col items-center mt-2">
          <div className="w-px h-2 bg-slate-700" />
          <div className="flex justify-around w-full">
            {nd.left && renderNode(nd.left)}
            {nd.right && renderNode(nd.right)}
          </div>
        </div>
      )}
    </div>
  );
}, [step, clr]);

```



```

const randomize = () => {
  const size = arr.length;
  const a = Array.from({ length: size }, () => Math.f
  setArr(a);
  setCustomInput(a.join(', '));
};

const buildSteps = useMemo(() => genBuildSteps(arr),
const queryTDSteps = useMemo(() => genQueryTopDownSte
const queryBUSteps = useMemo(() => genQueryBottomUpSte

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
  <div className="bg-slate-900 rounded-2xl border bor
    /* ---- TOP BAR: array editor + query range ----
    <div className="mb-6 space-y-4">
      <div className="flex flex-col lg:flex-row gap-4
        /* Array input */
        <form onSubmit={handleApply} className="flex-
          <div className="flex items-center justify-b
            <label className="text-[10px] font-bold up
            label>
              <button type="button" onClick={randomize}
            "><RefreshCw className="w-3.5 h-3.5" /></bu
          </div>
          <div className="flex gap-2">
            <input
              value={customInput}
              onChange={e => setCustomInput(e.target.
              className="flex-1 bg-slate-900 border b
            one focus:border-indigo-500"
              placeholder="5, 8, 3, 12, 7, 2"
            />
            <button type="submit" className="bg-indig
            ition-colors">Применить</button>
          </div>
          <div className="flex flex-wrap gap-1.5 pt-1

```

```

        <div className="text-[10px] text-slate-500" style={style}>
          Запрос:  $\text{sum}(A[\{qL\}..\{qR\}]) = \{\text{arr.slice}(qL, qR)\}$ 
        </div>
      </div>
    </div>
  </div>

  {/* ----- TABS ----- */}
  <div className="flex flex-wrap items-center gap-1" style={style}>
    <button onClick={() => setView('build')} className="flex items-center justify-center gap-1" style={style}
      on-colors ${view === 'build' ? 'border-indigo-500 text-slate-200'}`>
      <Hammer className="w-3.5 h-3.5" /> Построить
    </button>
    <button onClick={() => setView('queries')} className="flex items-center justify-center gap-1" style={style}
      on-colors ${view === 'queries' ? 'border-emerald-500 text-slate-200'}`>
      <Search className="w-3.5 h-3.5" /> Запрос: св
    </button>
    <button onClick={() => setView('theory')} className="flex items-center justify-center gap-1" style={style}
      on-colors ${view === 'theory' ? 'border-blue-500 text-slate-200'}`>
      <Info className="w-3.5 h-3.5" /> Почему / Сра
    </button>
  </div>

  {/* ----- TAB CONTENT ----- */}
  {view === 'build' && (
    <SimPanel
      key={`build-${arr.join('-')}`}
      title="Построение дерева (рекурсия)"
      icon=""
      steps={buildSteps}
      code={BUILD_CODE}
      color="indigo"
      arr={arr}
    />
  )}
  )}

```

```

        <div className="text-slate-300">mid = |{0
        <div className="text-indigo-300">→ Левый:
        <div className="text-emerald-300">→ Правы
h - 1) >> 1)} эл.)</div>
        <div className="text-slate-500 mt-2">Всег
    } внутренних.</div>
  </div>
</div>

```

```

{ /* Comparison cards */ }
<div className="grid grid-cols-1 xl:grid-cols
  <div className="bg-slate-950 p-5 rounded-xl
    <div className="absolute -top-3 left-4 bg
      e border border-emerald-500 shadow-md">
      | Запрос Сверху-вниз (Рекурсия)
    </div>
    <p className="text-xs text-slate-300 mb-3
      Начинаем с корня. Если отрезок узла полн
      наче спускаемся в обоих детей.
    </p>
    <div className="space-y-1.5 text-xs">
      <div className="flex justify-between bo
        span className="text-rose-400 font-mono">0{
          <div className="flex justify-between bo
        ><span className="text-rose-400 font-mono">
          <div className="flex justify-between"><
        d-400 font-bold">✓ легко</span></div>
      </div>
    </div>
  </div>
  <div className="bg-slate-950 p-5 rounded-xl
    <div className="absolute -top-3 left-4 bg
      rder border border-amber-500 shadow-md">
      | Запрос Снизу-вверх (Итерация)
    </div>
    <p className="text-xs text-slate-300 mb-3
      Стартуем с двух указателей (l и r) на л
      ём его левого соседа. Поднимаемся пока l {"
    </p>

```



```

selectedChapterId: string;
setSelectedChapterId: (id: string) => void;
/** Мобильный режим: список живёт в выдвижной панели */
onNavigate?: () => void;
}

export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId }) => {
  const [query, setQuery] = useState("");

  const groups = useMemo(() => {
    const q = query.trim().toLowerCase();
    const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
    const map = new Map<string, typeof chapters>();
    for (const c of filtered) {
      const key = c.category?.trim() || "Прочие темы";
      if (!map.has(key)) map.set(key, []);
      (map.get(key) as typeof chapters).push(c);
    }
    return Array.from(map.entries());
  }, [query]);

  return (
    <aside className="w-full bg-slate-900/80 lg:bg-transparent lg:p-4">
      <div className="p-4 pb-3 shrink-0">
        <h3 className="text-xs font-bold text-slate-400">
          <Compass className="w-4 h-4 text-indigo-400" />
        </h3>
        <div className="relative">
          <Search className="w-4 h-4 text-slate-500 absolute top-0 left-0">
            <input
              value={query}
              onChange={(e) => setQuery(e.target.value)}
              placeholder="Поиск по темам..."
              className="w-full bg-slate-800/70 border border-slate-500 focus:outline-none focus:border-indigo-500"
            />
            {query && (
              <button

```

```

        />
      }}
      <span className="font-semibold text-indigo-400">
        <ChevronRight
          className={`w-4 h-4 shrink-0 mt-0
            isSelected ? "text-indigo-400" : "text-slate-400"
          }` />
      </button>
    );
  }}}
</div>
</div>
)))
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-400 to-slate-300
  ] text-slate-300 space-y-1.5 hidden lg:block">
  <div className="font-bold text-indigo-400 flex items-center">
    <Sparkles className="w-3.5 h-3.5" /> Как пользоваться?
  </div>
  <p className="leading-relaxed">
    Подчёркнутые термины в тексте раскрываются по нажатию на стрелку.
  </p>
  <p className="text-slate-400 italic">Стрелки ← и → переключают видимость подсказок.
  </p>
</div>
</aside>
);
};

```

ФАЙЛ 63/87: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from 'react';
```

```
// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
  { id: '1', text: 'Кандидат: "Привет, я искусственный"',
  { id: '2', text: 'Кандидат: "Привет, я испек вкусный"',
  { id: '3', text: 'Кандидат: "Привет, я испанский ин
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.1);
const [heapMessage, setHeapMessage] = useState<string>('');

// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: '🥞' },
    { name: 'Кастрюля от супа', icon: '🍲' },
    { name: 'Чашка от кофе', icon: '☕' },
    { name: 'Миска с остатками салата', icon: '🥗' },
    { name: 'Тарелка от тако', icon: '🌮' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setStack(prev => [...prev, newPlate]);
  setPopMessage(`Положили поверх стопки: ${newPlate.name} ${newPlate.icon}`);
};

const popPlate = () => {
  if (stack.length === 0) {
    setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
    return;
  }
  const topPlate = stack[stack.length - 1];
  setStack(prev => prev.slice(0, prev.length - 1));
  setPopMessage(`Вымыта верхняя тарелка (LIFO): ${topPlate.name} ${topPlate.icon}`);
};
```

```

const resetQueue = () => {
  setQueue([
    { id: '1', name: 'Студент Вася', icon: ' ' },
    { id: '2', name: 'Голодный кодер', icon: ' ' },
    { id: '3', name: 'Кот Борис', icon: ' ' },
  ]);
  setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
  e.preventDefault();
  if (!newCustomText.trim()) return;
  const item: Hypothesis = {
    id: Date.now().toString(),
    text: `Кандидат: "${newCustomText}"`,
    probability: Number(newCustomProb)
  };
  setHeap(prev => [...prev, item]);
  setNewCustomText('');
  setHeapMessage(` Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
  if (heap.length === 0) {
    setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
    return;
  }
  // Find highest probability
  let maxIdx = 0;
  for (let i = 1; i < heap.length; i++) {
    if (heap[i].probability > heap[maxIdx].probability) {
      maxIdx = i;
    }
  }
  const best = heap[maxIdx];
  setHeap(prev => prev.filter(( , idx) => idx !== maxIdx));
};

```

```

        <Layers className={`w-5 h-5 ${activeTab ===
        <div className="text-left">
          <div className="font-bold text-sm text-wh
          <div className="text-xs opacity-80">LIFO
        </div>
      </div>
    <span className="text-xs bg-blue-500/20 text-l
      DFS
    </span>
  </button>

  <button
    onClick={() => setActiveTab('queue')}
    className={`flex items-center justify-between
      activeTab === 'queue'
        ? 'bg-indigo-600/20 border-indigo-500 tex
        : 'bg-slate-800/60 border-slate-700/60 te
    }`}
  >
    <div className="flex items-center gap-3">
      <AlignLeft className={`w-5 h-5 ${activeTab :
      <div className="text-left">
        <div className="font-bold text-sm text-wh
        <div className="text-xs opacity-80">FIFO
      </div>
    </div>
    <span className="text-xs bg-indigo-500/20 tex
      BFS
    </span>
  </button>

  <button
    onClick={() => setActiveTab('heap')}
    className={`flex items-center justify-between
      activeTab === 'heap'
        ? 'bg-emerald-600/20 border-emerald-500 t
        : 'bg-slate-800/60 border-slate-700/60 te
    }`}

```

```

        className="flex-1 md:flex-none flex items-center justify-center
border-blue-500/40 px-4 py-2.5 rounded-xl"
      >
        Помыть верхнюю
      </button>
      <button
        onClick={resetStack}
        title="Сбросить"
        className="p-2.5 bg-slate-800 hover:bg-slate-700 text-white
transition-colors cursor-pointer"
      >
        <RotateCcw className="w-5 h-5" />
      </button>
    </div>
  </div>

  {popMessage && (
    <div className="bg-blue-950/60 border border-blue-900/60 p-4
animate-fade-in">
      <span className="inline-block w-2 h-2 rounded-full bg-white" />
      {popMessage}
    </div>
  )}

  {/* Visualization */}
  <div className="bg-slate-950/60 p-6 rounded-2xl" >
    >
    {stack.length === 0 ? (
      <div className="text-center py-12 text-slate-400">
        <div className="text-5xl mb-3">    </div>
        <p className="text-base font-bold">Стек пуст</p>
        <p className="text-xs mt-1">Добавьте элемент</p>
      </div>
    ) : (
      <div className="w-full max-w-sm flex flex-direction-column align-items-center gap-1">
        <div className="absolute top-0 text-xs font-mono font-mono text-right">
          full flex items-center gap-1">
            <span>ВЕРШИНА СТЕКА (TOP)</span>

```

```

{/* QUEUE TAB */}
{activeTab === 'queue' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Симуляция очереди за супом</span>
          <span className="text-xs font-mono bg-i
        /span>
        </h3>
        <p className="text-slate-400 text-xs mt-1
          Кто первым пришел, тот первым получил с
        </p>
      </div>
      <div className="flex flex-wrap items-center
        <button
          onClick={addPerson}
          className="flex-1 md:flex-none flex ite
        -2.5 rounded-xl text-sm font-bold shadow-lg
        >
          <Plus className="w-4 h-4" /> Встать в о
        </button>
        <button
          onClick={servePerson}
          className="flex-1 md:flex-none flex ite
        er border-indigo-500/40 px-4 py-2.5 rounded
        >
          Налить суп первому
        </button>
        <button
          onClick={resetQueue}
          title="Сбросить"
          className="p-2.5 bg-slate-800 hover:bg-
        tion-colors cursor-pointer"
        >
          <RotateCcw className="w-5 h-5" />

```

```

        className={`flex flex-col items-center justify-center
        isFirst
        ? 'bg-gradient-to-b from-indigo-500 to-slate-900'
        : 'bg-slate-900/90 border-slate-800'
        }`}
      >
        {isFirst && (
          <span className="absolute -top-10 left-0 right-0
se tracking-wider shadow">
            Первый (Head)
          </span>
        )}
        <span className="text-4xl mb-2">{
          <span className={`font-bold text-slate-500
            {person.name}
          </span>
          <span className={`text-[10px] font-mono text-slate-500
            isFirst ? 'bg-indigo-500/30 text-white' : 'text-slate-500'
          }`}>
            {isFirst ? 'Получает сун' : `В
          </span>
        </div>
      </div>
    );
  })))

  <div className="flex flex-col items-center justify-center
-600 min-w-[120px] h-[120px]">
    <Plus className="w-6 h-6 mb-1" />
    <span className="text-xs font-mono uppercase">
    </div>
  </div>
)}
<div className="mt-6 text-xs text-slate-500">
  ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
</div>
</div>
</div>

```



```

s-12 gap-4 items-end">
<div className="md:col-span-6">
  <label className="block text-xs font-bold">
    <input
      type="text"
      value={newCustomText}
      onChange={e => setNewCustomText(e.target.value)}
      placeholder="например: 'Привет, я лучший!'"
      className="w-full bg-slate-900 border border-emerald-500 transition-colors"
    />
  </div>
<div className="md:col-span-3">
  <label className="block text-xs font-bold">
    Вероятность: <span className="text-emerald-500">{newCustomProb}</span>
  </label>
  <input
    type="range"
    min="0.01"
    max="0.99"
    step="0.01"
    value={newCustomProb}
    onChange={e => setNewCustomProb(Number(e.target.value))}
    className="w-full accent-emerald-500 cursor-pointer"
  />
</div>
<div className="md:col-span-3">
  <button
    type="submit"
    disabled={!newCustomText.trim()}
    className="w-full flex items-center justify-center gap-2 bg-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
  >
    <Plus className="w-4 h-4" /> Добавить в список
  </button>
</div>
</form>

```

```

<div className="flex items-center"
  <span className={`flex items-center
    isTop ? 'bg-emerald-500 text-
  }`}>
    #{index + 1}
  </span>
  <div>
    <div className={`font-bold te
      {item.text}
    </div>
    {isTop && {
      <span className="text-[10px
-0.5 rounded inline-block mt-1">
        Вершина Кучи (Root) • Буд
      </span>
    }}
  </div>
</div>

<div className="flex items-center"
  {/* Custom progress bar */}
  <div className="hidden sm:block"
    <div
      className={`h-full rounded-
      style={{ width: `${percent}%
    ></div>
  </div>
  <span className={`font-mono fon
    isTop ? 'text-emerald-400 tex
  }`}>
    {percent}%
  </span>
</div>
</div>
  );
  }}}
</div>
})

```

```

const titleById = new Map(chapters.map((c) => [c.id, c.title]));
const all = Object.entries(VIZ_REGISTRY).map(([id, entry], index) => ({
  id,
  entry,
  chapterTitle: titleById.get(id),
}));
const q = query.trim().toLowerCase();
if (!q) return all;
return all.filter(
  (i) =>
    i.entry.title.toLowerCase().includes(q) ||
    (i.entry.hint ?? "").toLowerCase().includes(q) ||
    (i.chapterTitle ?? "").toLowerCase().includes(q)
);
}, [query]);

return (
  <div>
    <div className="mb-6">
      <h2 className="text-xl sm:text-2xl font-extrabold">Визуализация
      <p className="text-sm text-slate-400 leading-relaxed">
        Все интерактивные демонстрации в одном месте.
        Здесь их можно открыть напрямую.
      </p>
    </div>

    <div className="relative mb-6 max-w-md">
      <Search className="w-4 h-4 text-slate-500 absolute left-4 top-4">
        <input
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          placeholder="Поиск тренажёра..."
          className="w-full bg-slate-900 border border-slate-800
            focus:outline-none focus:border-indigo-500"
        />
      {query && (
        <button
          onClick={() => setQuery("")}

```

```
        }}
      </div>
    </div>
  )))
</div>
</div>
);
};
```

---

ФАЙЛ 65/87: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, RotateCw, Undo, } from "lucide-react";
```

```
/**
```

```
 * Песочница поворотов.
```

```
 *
```

```
 * Здесь ничего не подсказывается: дано настоящее дерево.
```

```
 * поднять отмеченный узел в корень. Клик по узлу = один шаг.
```

```
 * Порядок поворотов выбираешь сам; правильный ли он пока неясно.
```

```
 * только после того, как узел окажется наверху (или по крайней мере в корень).
```

```
 */
```

```
export interface TNode {
  key: number;
  left: TNode | null;
  right: TNode | null;
}
```

```
const leaf = (key: number): TNode => ({ key, left: null, right: null });
```

```
const clone = (t: TNode | null): TNode | null =>
  t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null;
```

```

        return { key: 8, left: n4, right: leaf(9) };
    },
};

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode[] {
    const path: TNode[] = [];
    let cur = root;
    while (cur) {
        path.push(cur);
        if (key === cur.key) return path;
        cur = key < cur.key ? cur.left : cur.right;
    }
    return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
    const path = findPath(root, key);
    if (path.length < 2) return root;

    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const gg = path.length >= 3 ? path[path.length - 3] : null;

    if (p.left === x) {
        p.left = x.right;
        x.right = p;
    } else {
        p.right = x.left;
        x.left = p;
    }

    if (!gg) return x;
    if (gg.left === p) gg.left = x;
    else gg.right = x;
}

```

```

    if (n.right) edges.push([n.key, n.right.key]);
    walk(n.right, depth + 1);
  };
  walk(root, 0);

  const cols = Math.max(1, order);
  const rows = Math.max(1, Math.max(...nodes.map((n) =>
  const colW = 46;
  const rowH = 62;
  return {
    nodes: nodes.map((n) => ({ ...n, x: n.x * colW + co
    edges,
    width: cols * colW,
    height: rows * rowH + 20,
  });
}

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
  const [presetIdx, setPresetIdx] = useState(0);
  const preset = PRESETS[presetIdx];

  const [tree, setTree] = useState<TNode>(() => preset.l
  const [history, setHistory] = useState<TNode[]>([]);
  const [moves, setMoves] = useState<{ node: number; co
  const [showAnswer, setShowAnswer] = useState(false);

  const target = preset.target;
  const solved = tree.key === target;

  const reset = useCallback(
    (idx = presetIdx) => {
      setPresetIdx(idx);
      setTree(PRESETS[idx].build());
      setHistory([]);
      setMoves([]);
      setShowAnswer(false);
    },
  );
}

```

```

    </p>
  </div>
  <button
    onClick={() => reset((presetIdx + 1) % PRESETS.length)}
    className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-medium transition-colors shrink-0"
  >
    <Dice className="w-3.5 h-3.5" /> Другое дерево
  </button>
</div>

<div className="flex gap-2 mb-3 overflow-x-auto" >
  {PRESETS.map((p, i) => {
    <button
      key={p.name}
      onClick={() => reset(i)}
      className={`px-3 py-1.5 rounded-lg text-[11px] font-medium transition-colors
        ${i === presetIdx ? "bg-indigo-600 text-white" : "bg-white text-slate-700"}
      `}
    >
      {p.name}
    </button>
  )}
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800 p-4" >
  <svg
    viewBox={`0 0 ${width} ${height}`}
    className="h-auto block mx-auto"
    style={{ minWidth: Math.min(width * 1.5, 420) }}
    role="img"
  >
    {edges.map(([a, b], i) => {
      const na = nodes.find((n) => n.key === a);
      const nb = nodes.find((n) => n.key === b);
      if (!na || !nb) return null;
      const onPath = path.has(a) && path.has(b);
      return (

```

```

        </g>
      );
    }}}}
  </svg>
</div>

<div className="flex flex-wrap items-center gap-2"
  <button
    onClick={undo}
    disabled={history.length === 0}
    className="flex items-center gap-1.5 px-3 py-1
      -white disabled:opacity-40 text-xs font-bol
  >
    <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
  </button>
  <button
    onClick={() => reset()}
    className="flex items-center gap-1.5 px-3 py-1
      ext-xs font-bold transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" /> Заново
  </button>
  <button
    onClick={() => setShowAnswer((s) => !s)}
    className={`flex items-center gap-1.5 px-3 py-1
      showAnswer
        ? "bg-amber-600/20 border-amber-500 text-
        : "bg-slate-800 border-slate-700 text-sla
    }`
  >
    {showAnswer ? <EyeOff className="w-3.5 h-3.5"
    {showAnswer ? "Скрыть разбор" : "Сдаться / по
  </button>

  <span className="text-[11px] text-slate-500 ml-
    поворотов: {moves.length} • минимум: {minimal
  </span>
</div>

```



```
        </div>
      })
    </div>
  );
};
```

---

ФАЙЛ 66/87: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";
```

```
/**
```

```
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag
 *
```

```
 * Что сделано ради понятности:
```

```
 * * у узлов настоящие числа (20 / 40 / 60), а не абстрактные
```

```
 * * сразу видно, что дерево остаётся деревом поиска;
```

```
 * * под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ
```

```
 * * это и есть доказательство, что поворот ничего не ломает
```

```
 * * кадр «прицел» ничего не двигает, только подсвечивает
```

```
 * * на кадре «результат» остаются призраки – пунктирные
```

```
 * * местах и стрелки «откуда → куда»;
```

```
 * * по умолчанию анимация НЕ крутится сама: пользователь
```

```
 * * идёт в своём темпе. Автопрокрутка включается кнопкой
```

```
 */
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
```

```
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
```

```
  pos: Pos;
```

```
  edges: [NodeId, NodeId][];
```

```
  kind: "start" | "aim" | "result";
```

```
  title: string;
```

```

keys: { x: 20, p: 40 },
inorder: ["A", "20", "B", "40", "C"],
ranges: "A < 20 · B между 20 и 40 · C > 40",
frames: [
  {
    ...ZIG_BEFORE,
    kind: "start",
    title: "Было: 20 висит под корнем 40",
    text: "Нам нужен ключ 20, а он на глубине 1. Деду",
    ms: RESULT_MS,
  },
  {
    ...ZIG_BEFORE,
    kind: "aim",
    title: "Прицел: крутим ребро 40 – 20",
    text: "Ничего пока не двигается. Смотри на подсве",
    focus: ["p", "x"],
    ms: AIM_MS,
  },
  {
    pos: { x: [160, 50], A: [80, 130], p: [245, 130],
    edges: [ ["x", "A"], ["x", "p"], ["p", "B"], ["p",
    kind: "result",
    title: "Стало: 20 – корень",
    text: "40 стало правым сыном двадцатки. Поддереве",
    eва от 40 – это одно и то же место.",
    moved: ["x", "p", "B"],
    ms: RESULT_MS,
  },
],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
  pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
  edges: [ ["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

    },
    {
        ...ZZ_S1,
        kind: "aim",
        title: "Прицел 2: ребро 40 – 20",
        text: "А теперь обычный одиночный поворот – тот же",
        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [100, 45], A: [45, 118], p: [188, 118],
        edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
        kind: "result",
        title: "Стало: 20 в корне, бамбук стал кустом",
        text: "Сравни с первым кадром: А и В были на глуб
            дерево, а не только достаёт ключ.",
        moved: ["x", "p", "A", "B"],
        ms: RESULT_MS,
    },
    ],
};

```

/\* ----- Zig-Zag -----

```

const ZG_S0 = {
    pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
    edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZG_S1 = {
    pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
    edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIGZAG: Case = {
    key: "zig-zag",
    label: "Zig-Zag",
    when: "x и p – дети с РАЗНЫХ сторон (змейка)",
    rotations: "2 поворота, первым – нижний",

```

```

    edges: [{"x", "p"}, {"x", "g"}, {"p", "A"}, {"p",
kind: "result",
title: "Стало: 20 и 60 – два сына сорока",
text: "Дерево стало идеально симметричным: все че
    очку». ",
moved: [{"x", "p", "g", "C"}],
ms: RESULT_MS,
  },
],
};

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```

const COLOR: Record<string, { fill: string; stroke: str
  x: { fill: "#6366f1", stroke: "#a5b4fc" },
  p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
  g: { fill: "#f59e0b", stroke: "#fcd34d" },
  sub: { fill: "#1e293b", stroke: "#475569" },
};

```

```

const ROLE_RU: Partial<Record<NodeId, string>> = {
  x: "x – ищем его",
  p: "p – родитель",
  g: "g – дед",
};

```

```
const isSubtree = (id: NodeId) => id === "A" || id ===
```

```

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
  const { pos, edges, focus, moved } = frame;
  const dim = (id: NodeId) => (focus ? !focus.includes(
  const ghosts = frame.kind === "result" ? (moved ?? [])

```

```

return (
  <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
    <defs>
      <marker id="splay-arrow" viewBox="0 0 10 10" re
        <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />

```

```

        key={` ${a} - ${b} - ${i} `}
        x1={pa[0]}
        y1={pa[1]}
        x2={pb[0]}
        y2={pb[1]}
        stroke={hot ? "#fbbf24" : "#475569"}
        strokeWidth={hot ? 5 : 2}
        opacity={focus && !hot ? 0.25 : 1}
        style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)" }}
      />
    );
  }
}

```

```

{Object.keys(pos) as NodeId[]}.map((id) => {
  const p = pos[id];
  if (!p) return null;
  const sub = isSubtree(id);
  const c = COLOR[sub ? "sub" : id];
  const r = sub ? 15 : 20;
  const hot = Boolean(focus && focus.includes(id));
  return (
    <g>
      key={id}
      style={{
        transform: `translate(${p[0]}px, ${p[1]}px)`,
        transition: MOVE,
        opacity: dim(id) ? 0.28 : 1,
      }}
    >
      {hot && <circle r={r + 7} fill="none" stroke={c.stroke} />
      <circle r={r} fill={c.fill} stroke={c.stroke} />
      <text>
        textAnchor="middle"
        dominantBaseline="central"
        fontSize={sub ? 12 : 15}
        fontWeight="700"
        fill={sub ? "#94a3b8" : "#fff"}
      </text>
    </g>
  );
}

```

```

return (
  <div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800">
    <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
      {CASES.map((c, i) => (
        <button
          key={c.key}
          onClick={() => setCaseIdx(i)}
          className={`px-3 sm:px-4 py-2 rounded-lg text-sm transition-colors
            i === caseIdx ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300"
          }`}
        >
          {c.label}
        </button>
      ))}
    </div>

    <div className="mb-3 text-xs sm:text-[13px] text-slate-400">
      <span className="font-bold text-indigo-300">Корреляция</span>
      <span className="text-slate-500">({active.rotation}°)</span>
    </div>

    {/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение */}
    <div className="mb-3 rounded-xl border border-slate-800">
      <div className="flex items-center gap-2 mb-1 flex-wrap">
        <span
          className={`text-[10px] font-bold uppercase transition-colors
            current.kind === "aim"
              ? "bg-amber-500/20 text-amber-300"
              : current.kind === "start"
                ? "bg-slate-700 text-slate-300"
                : "bg-indigo-500/20 text-indigo-300"
          }`}
        >
          {current.kind === "aim" ? "прицел" : current.kind === "start" ? "начало" : "конец"}
        </span>
        <span className="text-sm font-bold text-white">Шаг {idx + 1} из {total}. {current.title}</span>
      </div>
    </div>
  </div>
)

```

```
</div>
```

```
<div className="flex flex-wrap items-center gap-2">
  <button
    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f - 1 + total) % total);
    }}
    disabled={idx === 0}
    className="px-3 py-2 rounded-lg bg-slate-800 text-white
      d: hover:text-slate-300 text-xs font-bold transition-colors"
  >
    ← Назад
  </button>
  <button
    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f + 1) % total);
    }}
    className="px-4 py-2 rounded-lg bg-indigo-600 text-white
      w-indigo-900/40"
  >
    {last ? "⏮ Сначала" : "Дальше →"}
  </button>
  <button
    onClick={() => setPlaying((p) => !p)}
    className="flex items-center gap-1.5 px-3 py-2 text-white
      text-xs font-bold transition-colors"
  >
    {playing ? <Pause className="w-3.5 h-3.5" /> : <Play />}
    {playing ? "Стоп" : "Авто"}
  </button>
  <button
    onClick={() => setSlow((s) => !s)}
    title="Автопрокрутка ещё медленнее"
    className={`flex items-center gap-1.5 px-3 py-2 text-white
      ${slow ? "bg-emerald-600/20 border-emerald-500" : "bg-slate-800/20 border-slate-500"}
      rounded-lg transition-colors`
  >
```

```
import React, { useState } from "react";
import {
  RotateCcw,
  HelpCircle,
  BookOpen,
} from "lucide-react";

interface SplayNode {
  key: number;
  left: SplayNode | null;
  right: SplayNode | null;
  x?: number;
  y?: number;
}

// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number) => {
  if (!root) return { key, left: null, right: null };
  if (key < root.key) {
    root.left = insertBST(root.left, key);
  } else if (key > root.key) {
    root.right = insertBST(root.right, key);
  }
  return root;
};

// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
  const y = x.left;
  if (!y) return x;
  x.left = y.right;
  y.right = x;
  return y;
};

// Left Rotation (Zag / Left Rotate)
```



```
// Splay operation that tracks steps!
const splayStepByStep = {
  root: SplayNode | null,
  key: number,
}: { newRoot: SplayNode | null; steps: string[] } =>
  const steps: string[] = [];
  if (!root) return { newRoot: null, steps: ["Дерево"]

  // We implement the classic top-down or bottom-up splay
  // For visualization logs, we do a bottom-up explanation
  let path: { node: SplayNode; dir: "left" | "right" }[] = [];
  let current: SplayNode | null = root;

  // Find the element or the element where search failed
  let lastNode: SplayNode = root;
  while (current) {
    lastNode = current;
    if (key === current.key) {
      path.push({ node: current, dir: null });
      break;
    } else if (key < current.key) {
      path.push({ node: current, dir: "left" });
      current = current.left;
    } else {
      path.push({ node: current, dir: "right" });
      current = current.right;
    }
  }

  const targetKey = current ? key : lastNode.key;
  const isFound = !!current;

  if (!isFound) {
    steps.push(`Поиск ${key}: элемент отсутствует в дереве`);
    steps.push(
      `Поиск споткнулся на узле [${lastNode.key}]. Инициализация`
    );
  }
}
```

```

        if (!node.right) return node;

        if (k < node.right.key) {
            // Zag-Zig (Right-Left)
            node.right.left = splayAction(node.right.left, target);
            if (node.right.left) {
                node.right = rightRotate(node.right);
                steps.push(`Компонент Zig-Zag: правый поворот`);
            }
        } else if (k > node.right.key) {
            // Zag-Zag (Right-Right)
            node.right.right = splayAction(node.right.right, target);
            node = leftRotate(node);
            steps.push(
                `Вращение Zag-Zag (Левый поворот родителя д...`
            );
        }

        if (!node.right) return node;
        steps.push(
            `Финальный поворот Zag (Левое вращение корня)`
        );
        return leftRotate(node);
    }
};

const finalRoot = splayAction(cloneTree(root), target);
return { newRoot: finalRoot, steps };
};

const handleSplay = (key: number) => {
    setHighlightedNode(key);
    const { newRoot, steps } = splayStepByStep(tree, key);
    if (newRoot) {
        setTree(newRoot);
    }
    setLog(steps);
};

```

```

    levelWidth: number,
  ): void => {
    if (!node) return;
    node.x = x;
    node.y = y;
    if (node.left) {
      computeCoordinates(node.left, x - levelWidth, y +
    }
    if (node.right) {
      computeCoordinates(node.right, x + levelWidth, y +
    }
  }
};

```

```

const drawTree = cloneTree(tree);
computeCoordinates(drawTree, 300, 40, 140);

```

```

// Render lines and nodes
const renderLines = (node: SplayNode | null): React.R
  if (!node) return [];
  let lines: React.ReactNode[] = [];
  if (node.left && node.left.x && node.left.y) {
    lines.push(
      <line
        key={`l-${node.key}-${node.left.key}`}
        x1={node.x}
        y1={node.y}
        x2={node.left.x}
        y2={node.left.y}
        stroke="#475569"
        strokeWidth="2"
      />,
    );
    lines = lines.concat(renderLines(node.left));
  }
  if (node.right && node.right.x && node.right.y) {
    lines.push(
      <line
        key={`l-${node.key}-${node.right.key}`}

```

```

        textAnchor="middle"
        fill="white"
        fontSize="12px"
        fontWeight="bold"
        className="select-none pointer-events-none"
      >
        {node.key}
      </text>
    </g>,
  );
  circles = circles.concat(renderNodes(node.left));
  circles = circles.concat(renderNodes(node.right));
  return circles;
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    {/* Header */}
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <BookOpen className="w-5 h-5 text-indigo-400"
        Интерактивное Splay-дерево
      </h3>
      <p className="text-xs text-slate-400">
        Кликайте на вершины дерева, чтобы «вытащить»
        Splay.
      </p>
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-12
      {/* Playfield */}
      <div className="lg:col-span-7 bg-[#0f172a] p-4
        <svg
          className="w-full h-full max-w-[600px] max-h-[320px]
          viewBox="0 0 600 320"
        >
          {renderLines(drawTree)}
          {renderNodes(drawTree)}
        </svg>
      </div>
    </div>
  );
}

```

```

        Вставить
      </button>
    </form>

    <button
      onClick={resetTree}
      className="flex items-center gap-1.5 px-3"
    >
      <RotateCcw className="w-3.5 h-3.5" /> Сброс
    </button>
  </div>
</div>

{/* Logs */}
<div className="lg:col-span-5 bg-slate-950 border-1
  to overflow-y-auto custom-scrollbar">
  <h4 className="text-xs font-bold text-slate-400">
    Ход вращения (Splay-логи)
  </h4>
  <div className="flex-1 space-y-2 mb-4 overflow-hidden">
    {log.map((step, idx) => (
      <div
        key={idx}
        className={`text-xs p-2.5 rounded transition
          ${idx === log.length - 1
            ? "border-l-2 border-indigo-500 bg-slate-900"
            : "text-slate-400 bg-slate-900/50"}
        `}
      >
        {step}
      </div>
    ))}
  </div>

  <div className="border-t border-slate-800 pt-2">
    <p>
      <span className="font-semibold text-slate-400">
        у самого корня. Делаем 1 вращение.

```

```
        сверху.
    </p>
</div>
<div className="mt-3 text-[10px] text-indigo-500">
    Поиск настраивает локальный кэш под реали
</div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l
<div>
    <p className="font-bold text-slate-300 mb-2">
        2. Эффект качелей (Swing)
    </p>
    <p className="text-slate-400 leading-relaxed">
        Если агент запрашивает по очереди{" "}
        <code className="text-rose-400">[10]</code>
        <code className="text-rose-400">[60]</code>
        углы), дерево будет превращаться в палку
        сторону. Первая операция Splay(60) стои
        <strong className="text-white">O(n)</strong>
        глубину n. Вторая Splay(10) заставляет
        Постоянный свинг убивает производительн
        сложности <strong className="text-white">O(n)</strong>
    </p>
</div>
<div className="mt-3 text-[10px] text-rose-500">
    Кэшировать свингующие пары на концах – фа
</div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l
<div>
    <p className="font-bold text-slate-300 mb-2">
        3. Амортизация O(log n)
    </p>
    <p className="text-slate-400 leading-relaxed">
        Парадокс: один запрос за{" "}
        <code className="text-emerald-400">O(n)</code>
```

```
-----
  { name: 'Тарелка из-под пиццы', emoji: ' ', color: 'f
  { name: 'Блюдец от торта', emoji: ' ', color: 'from-p
  { name: 'Тарелка от тако', emoji: ' ', color: 'from-y
  { name: 'Тарелка из-под суши', emoji: ' ', color: 'fr
  { name: 'Сковорода от глазуньи', emoji: ' ', color: '
];
```

```
let counter = 0;
```

```
export const StackViz: React.FC = () => {
  const [dirtyStack, setDirtyStack] = useState<Plate[]>([
    { id: 'p1', name: 'Тарелка из-под спагетти', emoji:
      ate: 'idle' },
    { id: 'p2', name: 'Тарелка из-под пиццы', emoji:
      State: 'idle' },
    { id: 'p3', name: 'Блюдец от торта', emoji: '
      tate: 'idle' },
  ]);
```

```
const [cleanRack, setCleanRack] = useState<Plate[]>([
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([
```

```
const addLog = (msg: string) => setLog(prev => [msg,
```

```
// PUSH: Положить грязную тарелку сверху стопки
```

```
const pushPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length >= 7) {
    addLog('⚠ Стопка слишком высокая, тарелки могут р
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }
}
```

```
const preset = PLATE PRESETS[counter % PLATE PRESETS
```

```

    const cleanPlate: Plate = {
      ...topPlate,
      isDirty: false,
      animState: 'clean',
    };

    setCleanRack(prev => [cleanPlate, ...prev].slice(0, 1));
    setDirtyStack(prev => prev.slice(0, prev.length - 1));
    setShowSoap(false);
    setIsWashing(false);
    addLog(`    Готово! Чистая тарелка ${topPlate.emoji}`, 850);
  }, [dirtyStack, isWashing]);

const reset = () => {
  counter = 0;
  setDirtyStack([
    { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'in' },
    { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'in' },
    { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', state: 'in' },
  ]);
  setCleanRack([]);
  setIsWashing(false);
  setShowSoap(false);
  setLog(['    Стол сброшен. Посуда снова грязная!']);
};

const topIndex = dirtyStack.length - 1;

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-blue-950/40 border border-blue-950/40 p-6">
      <div className="text-3xl">    </div>
      <div>

```



```
</div>
```

```
{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
```

```
<div className="grid grid-cols-1 md:grid-cols-12">
```

```
  {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
```

```
  <div className="md:col-span-7 bg-slate-900 rounded-3xl h-[380px]">
```

```
    <div className="absolute top-4 left-4 text-[12px] text-white">
      Грязная стопка (Стек вызовов / LIFO)
```

```
    </div>
```

```
  {/* Визуализация раковины */}
```

```
  <div className="absolute top-4 right-4 flex items-center justify-center gap-2" style={{background: 'linear-gradient(to right, slate-800/80, slate-800/80) text-[10px] font-mono'}}>
```

```
    <span className="w-1.5 h-1.5 rounded-full bg-white" style={{display: inline-block; width: 10px; height: 10px; border-radius: 50%; background-color: white; border: 1px solid black; margin-right: 5px; vertical-align: middle;"/>
```

```
    <span>РАКОВИНА </span>
```

```
  </div>
```

```
  {/* Мыльные пузыри при мытье */}
```

```
  {showSoap && {
```

```
    <div className="absolute inset-0 pointer-events-none">
```

```
      <div className="absolute w-6 h-6 rounded-full bg-white border-2 border-black" style={{width: 24px, height: 24px, border: 2px solid black; border-radius: 50%; background-color: white; margin: 10px auto; position: relative;"/>

```

```
      <div className="absolute w-4 h-4 rounded-full bg-white border-2 border-black" style={{width: 16px, height: 16px, border: 2px solid black; border-radius: 50%; background-color: white; margin: 10px auto; position: relative;"/>

```

```
      <div className="absolute w-5 h-5 rounded-full bg-white border-2 border-black" style={{width: 20px, height: 20px, border: 2px solid black; border-radius: 50%; background-color: white; margin: 10px auto; position: relative;"/>

```

```
      <div className="absolute text-4xl anim-crash" style={{position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); font-size: 2em; color: red; font-weight: bold; opacity: 0.5; pointer-events: none;"/>

```

```
  })
```

```
{dirtyStack.length === 0 ? {
```

```
  <div className="flex-1 flex flex-col items-center justify-center gap-4" style={{height: 100px; display: flex; align-items: center; justify-content: center; background-color: #f0f0f0; border: 1px solid #ccc; border-radius: 10px; text-align: center; font-size: 1.2em; font-weight: bold; color: #333; margin: 10px auto; width: 80%; box-shadow: 0 4px 6px #ccc;"/>

```

```
    <span className="text-6xl mb-3 animate-pulse" style={{font-size: 3em; color: #333; margin-bottom: 10px; opacity: 0.5; pointer-events: none;"/>

```

```
    <p className="text-white font-black text-2xl" style={{color: white; font-weight: bold; font-size: 1.5em; margin: 0; opacity: 0.5; pointer-events: none;"/>

```

```
    <p className="text-slate-500 text-xs mt-1" style={{color: #666; font-size: 0.8em; margin: 0; opacity: 0.5; pointer-events: none;"/>


```
  </div>
```


```

```

        }}
      </div>
    );
  }}}
</div>
}}

{/* Столешница раковины */}
<div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800" >
</div>

{/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */}
<div className="md:col-span-5 bg-slate-900 rounded-3xl p-4" >
  <div className="absolute top-4 left-4 text-[10px] font-mono" >
    Сушилка для чистой посуды
  </div>

  <div className="absolute top-4 right-4 flex items-center justify-center gap-2" >
    <div className="order-emerald-800/80 text-[9px] font-mono" >
      <Sparkles className="w-3.5 h-3.5 animate-pulse" />
      <span>ЧИСТО </span>
    </div>

    <div className="flex-1 flex flex-col justify-between p-2" >
      {cleanRack.length === 0 ? (
        <div className="flex-1 flex flex-col item-center gap-2" >
          <span className="text-5xl mb-2" > </span>
          <p className="text-xs font-bold font-mono" >Нет тарелок
          <p className="text-[10px] mt-1" >Здесь будет тарелка
        </div>
      ) : (
        <div className="flex flex-col gap-2 w-full" >
          {cleanRack.map(plate => (
            <div
              key={plate.id}
              className="w-full h-8 rounded-full border-1
            items-center justify-center gap-2 opacity-0.5"
          )}
        </div>
      )}
    </div>
  </div>
</div>

```

```
-----
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'react-native';

function generateKmpSteps(pattern: string, text: string) {
  if (!pattern) pattern = " ";
  const s = pattern + "#" + text;
  const n = s.length;
  const pi = new Array(n).fill(0);
  const steps: any[] = [];

  steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

  for (let i = 1; i < n; i++) {
    let j = pi[i - 1];

    steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });

    while (j > 0 && s[i] !== s[j]) {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадения нет` });
      j = pi[j - 1];
    }

    if (s[i] === s[j]) {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
      j++;
    } else {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадения нет` });
    }

    pi[i] = j;
    steps.push({ i, j, pi: [...pi], desc: `Записываем` });

    if (j === pattern.length) {
      steps.push({ i, j, pi: [...pi], found: true });
    }
  }
}
```

```

        steps.push({ i, l, r, z: [...z], desc: `Фиксирываем` });

        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
            steps.push({ i, l, r, z: [...z], desc: `Обновляем` });
        }

        if (z[i] === pattern.length) {
            steps.push({ i, l, r, z: [...z], found: true });
        }
    }
    return { s, steps, patternLength: pattern.length };
}

```

```

export function StringAlgorithmsViz({ defaultMode = "kmp" }) {
    const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
    const [pattern, setPattern] = useState("aba");
    const [text, setText] = useState("abacaba");

```

```

    // Ensure inputs are valid dynamically
    const safePattern = pattern || " ";
    const safeText = text || " ";

```

```

    const { s, steps, patternLength } = useMemo(() => {
        if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
        else return generateZSteps(safePattern, safeText);
    }, [mode, safePattern, safeText]);

```

```

    const [autoPlay, setAutoPlay] = useState(false);
    const [stepIdx, setStepIdx] = useState(0);
    useVizStepSync(stepIdx, setStepIdx, steps.length - 1);
    const timer = useRef<NodeJS.Timeout | null>(null);

```

```

    // Reset when inputs or mode change
    useEffect(() => {
        setStepIdx(0);
    }, [mode, pattern, text]);

```

```

        <div className="flex items-center gap-2"
          <div className="bg-slate-800 p-1 flex items-center"
            <span className="text-[10px] text-white text-xs font-bold font-mono px-2 py-1"
              <input value={pattern} onChange={onChangePattern} type="text" />
            </div>
            <div className="bg-slate-800 p-1 flex items-center"
              <span className="text-[10px] text-white text-xs font-bold font-mono px-2 py-1"
                <input value={text} onChange={onChangeText} type="text" />
              </div>
            </div>
          </div>
        </div>

<div className="bg-slate-950 rounded-xl border-[2px] flex items-center justify-start"
  <div className="flex gap-1.5 px-4 h-full"
    {s.split("").map((char, index) => {
      const isPattern = index < pattern.length;
      const isSeparator = index === pattern.length;

      let borderColor = "border-slate-900/40";
      let bgColor = "bg-slate-900";
      let textColor = isSeparator ? "text-white" : "text-slate-400";

      // KMP Logic
      if (mode === "kmp") {
        if (step.highlight?.include(char)) {
          borderColor = "border-amber-500";
          bgColor = step.match === char ? "bg-amber-500" : "bg-slate-900/40";
        }
      }

      // Z Logic
      if (mode === "z") {
        if (index >= step.l && index <= step.r) {
          borderColor = "border-amber-500";
          bgColor = "bg-amber-500";
          textColor = "text-white";
        } else {
          borderColor = "border-slate-900/40";
          bgColor = "bg-slate-900";
          textColor = "text-slate-400";
        }
      }

      return (
        <div className={`${textColor} font-mono font-bold text-[10px] flex items-center justify-center gap-0.5`}>
          {char}
        </div>
      );
    })}
  </div>
</div>

```

```

z-20">
        <span className=
    </div>
    })
    {mode == "kmp" && step
        <div className="abs
            <span className=
        </div>
    })
    {mode == "kmp" && step
        <div className="abs

20">
        <span className=
    </div>
    })

    {/* Z fingers */}
    {mode == "z" && step.i
        <div className="abs

e z-20">
        <span className=
    </div>
    })
    {mode == "z" && step.z
        <div className="abs
            <span className=
        </div>
    })
    {mode == "z" && step.z
        <div className="abs
            <span className=
        </div>
    })

    {/* Found flash effect
    {step.found && mode ==
        <div className="abs

ulse z-0 bg-emerald-500/20 shadow-[0 0 15px

```

```

        <div className="text-[10px] text-slate-400" data-bbox="274 100 1000 145">
            {steps.length}</div>
        <div className="text-sm text-slate-400" data-bbox="407 148 1000 193">
            {step.desc}
        </div>
    </div>
</div>

{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl" data-bbox="274 315 1000 335">
    <span className="text-2xl mt-1" data-bbox="341 338 1000 358"> </span>
    <p className="text-xs text-slate-400" data-bbox="341 361 1000 381">
        Обрати внимание на хитрость с решёткой.
        Мы пишем <b>Шаблон + # + Текст</b>.
        Когда значени {mode === 'kmp' ? "проблема" : "не найдено"}
        о было найдено!
        Пройди шаги вручную, посмотри, как работает алгоритм
        для пропуска работы (в Z).
    </p>
</div>

{/* Code Sneak Peek */}
<div className="bg-slate-900 border border-slate-800" data-bbox="274 623 1000 643">
    <div className="bg-slate-800 px-4 py-2 border border-slate-400 uppercase" data-bbox="323 646 1000 686">
        <span>Код C++ ({mode === 'kmp' ? 'КМП' : 'ЗКМП'})</span>
    </div>
    <pre className="p-4 text-xs font-mono text-slate-400" data-bbox="323 741 1000 761">
{mode === 'kmp' ? `vector<int> pi(s.length());` : `vector<int> pi(n);`}
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] > r) l = i, r = i + z[i];
}
return z;
    </pre>

```

```

return (
  <span
    className={`relative inline-flex ${className} ?? ""
    onMouseEnter={() => setOpen(true)}
    onMouseLeave={() => setOpen(false)}
    onFocus={() => setOpen(true)}
    onBlur={() => setOpen(false)}
  >
    {children}
    <span
      role="tooltip"
      className={`pointer-events-none absolute left-1/2
        bg-slate-900 px-3 py-2 text-left text-[11px]
        l duration-150 ${
          side === "top" ? "bottom-full mb-2" : "top-full
        } ${open ? "visible translate-y-0 opacity-100"
      >
        {content}
        <span
          className={`absolute left-1/2 h-2 w-2 -transl
            side === "top" ? "-bottom-[5px] border-b bo
          }`}
        />
      </span>
    </span>
  );
};

```

ФАЙЛ 71/87: `src/components/TopologicalSortViz.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import {
  Play,

```



```

    currentNode: number | null;
    topoList: number[];
    dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
  const [steps, setSteps] = useState<TStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const listSteps: TStep[] = [];
    const visited = new Set<number>();
    const fullyProcessed = new Set<number>();
    const topoList: number[] = [];
    const dfsStack: number[] = [];

    const recordStep = (desc: string, currentNode: number) => {
      listSteps.push({
        desc,
        visited: new Set(visited),
        fullyProcessed: new Set(fullyProcessed),
        currentNode,
        topoList: [...topoList],
        dfsStack: [...dfsStack],
      });
    };

    recordStep(
      "Начало топологической сортировки. Используем кла",
      null,
    );

    // Adj list
    const adj: number[][] = Array.from({ length: 5 }, () => []);
    dagEdges.forEach((e) => adj[e.u].push(e.v));
  }, []);
};

```

```

    }
  }

  recordStep(
    "Топологическая сортировка завершена! Полученный
    null,
  );
  setSteps(listSteps);
  setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
  desc: "Заныск...",
  visited: new Set(),
  fullyProcessed: new Set(),
  currentNode: null,
  topoList: [],
  dfsStack: [],
};

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1)
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1600);
  } else {
    setIsPlaying(false);
  }
  return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

```

```

<div className="grid grid-cols-1 lg:grid-cols-12" >
  { /* Playfield SVG */}
  <div className="lg:col-span-8 bg-[#0B1120] relative" >
    <svg className="w-full h-full max-w-[650px]" >
      <defs>
        <marker
          id="dag-arrow"
          viewBox="0 0 10 10"
          refX="24"
          refY="5"
          markerWidth="6"
          markerHeight="6"
          orient="auto-start-reverse"
        >
          <path d="M 0 1 L 10 5 L 0 9 z" fill="#4" />
        </marker>
        <marker
          id="dag-arrow-active"
          viewBox="0 0 10 10"
          refX="24"
          refY="5"
          markerWidth="6"
          markerHeight="6"
          orient="auto-start-reverse"
        >
          <path d="M 0 1 L 10 5 L 0 9 z" fill="#8" />
        </marker>
      </defs>

      { /* Edges */}
      {dagEdges.map((edge, idx) => {
        const u = dagNodes.find((n) => n.id === edge.u)
        const v = dagNodes.find((n) => n.id === edge.v)

        const isActive =
          (step.currentNode === edge.u || step.currentNode === edge.v) ||
          step.visited.has(edge.v);
      })}
    </svg>
  </div>
</div>

```

```

        textColor = "text-indigo-200";
    }

    return (
      <g key={node.id}>
        <rect
          x={node.x - 55}
          y={node.y - 22}
          width="110"
          height="44"
          rx="8"
          fill={fill}
          stroke={stroke}
          strokeWidth="2"
          className="transition-all duration-100"
        />
        <text
          x={node.x}
          y={node.y - 4}
          textAnchor="middle"
          fill="white"
          fontSize="11px"
          fontWeight="bold"
          className="pointer-events-none"
        >
          ({node.label}) {node.name.split(" ").join(" ")}
        </text>
        <text
          x={node.x}
          y={node.y + 12}
          textAnchor="middle"
          fill="#94a3b8"
          fontSize="9px"
          className="pointer-events-none"
        >
          {node.name.split(" ").slice(1).join(" ")}
        </text>
      </g>
    );
  }
}

```

```

        <span className="text-[10px] text-slate-500">
          ВЕРШИНЫ В DFS:
        </span>
        <div className="flex items-center gap-2">
          <span className="w-3.5 h-3.5 rounded bg-slate-400">
            <span className="text-slate-400 text-[10px]">
              Серые (зашли)
            </span>
          </span>
        </div>
        <div className="flex items-center gap-2">
          <span className="w-3.5 h-3.5 rounded bg-slate-400">
            <span className="text-slate-400 text-[10px]">
              Черные (готово)
            </span>
          </span>
        </div>
      </div>
    </div>

    <div className="mt-4 pt-3 border-t border-slate-200">
      <span>
        Шаг {currentStepIdx + 1} из {steps.length}
      </span>
      <div className="flex gap-1">
        <button
          disabled={currentStepIdx === 0}
          onClick={() => setCurrentStepIdx((prev) => prev - 1)}
          className="bg-slate-900 border border-slate-200 text-white"
        >
          Назад
        </button>
        <button
          disabled={currentStepIdx >= steps.length - 1}
          onClick={() => setCurrentStepIdx((prev) => prev + 1)}
          className="bg-slate-900 border border-slate-200 text-white"
        >
          Вперед
      </div>
    </div>
  </div>
)

```

```

-----
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithms";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimation";
import ChapterImage from "./ChapterImage";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";

/** Разбор поворота и песочница всегда идут парой: посмотри */
/**
 * Анимация и песочница идут друг под другом, а не в две
 * колонке дерево сжималось до нечитаемого размера, а по
 * компактности. Между ними – подпись, объясняющая перемену
 */
const SplayRotationsPair: React.FC = () => (
  <div className="space-y-4">
    <SplayRotationsViz />
    <p className="flex items-start gap-2 text-[12px] leading-3 py-2">
      <span aria-hidden="true"> </span>
      <span>
        Разобрался – проверь себя: ниже то же самое дерево
        не решишь.
      </span>
    </p>
    <SplayRotationSandbox />
  </div>
);

export interface VizEntry {
  /** Заголовок панели/модалки. */
  title: string;
  /** Короткое пояснение, что здесь можно потыкать. */
  hint?: string;
  Component: React.FC;
}

```

```

    hint: "Наведите на число – увидите, из чего оно сложилось",
    Component: PrefixSumViz,
  },
  "dynamic-programming": {
    title: "Динамическое программирование",
    hint: "Таблица подзадач заполняется на глазах.",
    Component: DPVisualizer,
  },
  "splay-tree": {
    title: "Splay-дерево",
    hint: "Покадровый разбор трёх поворотов, песочница",
    Component: () => (
      <div className="space-y-10">
        <SplayRotationsPair />
        <SplayTreeViz />
      </div>
    ),
  },
  "splay-rotations": {
    title: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
    hint: "Сверху – покадровый разбор поворота, снизу – результат",
    Component: () => <SplayRotationsPair />,
  },
  "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: GraphDFS },
  "top-sort": { title: "Топологическая сортировка", Component: TopSort },
  "scc-kosaraju": { title: "Компоненты сильной связности", Component: SCC },
  "graph-articulation": { title: "Точки сочленения", Component: Articulation },
  "bridges-code": { title: "Мосты: DFS + tin/low", Component: Bridges },
  "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: EulerPath },
  "planarity-euler-formula": { title: "Планарность: K5 и K4", Component: Planarity },
  dijkstra: {
    title: "Дейкстра",
    hint: "Жадно забираем ближайшую вершину и релаксируем",
    Component: () => (
      <>
        <ChapterImage vizType="dijkstra" />
        <DijkstraViz />
      </>
    ),
  },

```

```
-----  
    title: "Бытовой тренажёр: стек, очередь, куча",  
    hint: "Тарелки, очередь в столовой и мешок гипотез",  
    Component: Simulator,  
  },  
};
```

```
export const getViz = (id?: string): VizEntry | undefined
```

```
-----  
ФАЙЛ 73/87: src/components/WaterfallAnimationWidget.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО
```

```
-----  
import React, { useState } from 'react';  
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb }
```

```
// Структура заранее подготовленного красивого бора для  
// Словарь: ["HE", "SHE", "HIS", "HERS"]
```

```
interface WNode {  
  id: number;  
  label: string; // путь от корня  
  char: string;  
  x: number; // Координаты для SVG водопада  
  y: number;  
  depth: number;  
  fail: number;  
  term_link: number;  
  is_terminal: boolean;  
  words: string[];  
}
```

```
const WATERFALL_NODES: { [key: number]: WNode } = {  
  0: { id: 0, label: 'ROOT', char: 'Ø', x: 400, y: 60, depth: 0,  
    // Ветка H -> E -> R -> S  
    1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1,  
      2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2,  
        3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3,
```



```

const [foundMatches, setFoundMatches] = useState<{ in
const [activeSearchCodeLine, setActiveSearchCodeLine]

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и
const [trainStep, setTrainStep] = useState<number>(0)

const handleTextChange = (e: React.ChangeEvent<HTMLInp
  const clean = e.target.value.toUpperCase().replace(
  setTextToScan(clean);
  resetSearchSimulation();
};

const resetSearchSimulation = () => {
  setCurrentIndex(0);
  setCurrentNode(0);
  setIsSearchDone(false);
  setSearchLog('Симуляция сброшена. Лосось вернулся н
  setJumpPath(null);
  setFoundMatches([]);
  setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
  if (currentIndex >= textToScan.length) {
    setIsSearchDone(true);
    setSearchLog('Мы проплыли весь текст! Лосось дово
    setActiveSearchCodeLine(4);
    return;
  }

  const char = textToScan[currentIndex];
  let curr = currentNode;
  let logMsg = `[Символ '${char}']`;

  let jumpType: 'normal' | 'fail' | null = null;
  void jumpType;
  let originalCurr = curr;

```

```

        }
        temp = WATERFALL_NODES[temp].term_link;
    }

    if (matchedWordsForLog.length > 0) {
        logMsg += `    БИНГО! Найдено слово: ${matchedWords}
        setActiveSearchCodeLine(4);
    }

    setCurrentNode(curr);
    setCurrentIndex(currentIndex + 1);
    setSearchLog(logMsg);
    if (currentMatches.length > 0) {
        setFoundMatches((prev) => [...prev, ...currentMat
    }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
    {
        targetNodeId: 0,
        title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
        desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
            рние вершины сразу инициализируются с fail = RO
        codeLine: 1,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'ø',
        checkingBranchesFrom: null,
    },
    {
        targetNodeId: 1,
        title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
        desc: 'Мы обрабатываем первую вершину из очереди -
            просто не существует. fail[H] автоматически нап
        codeLine: 5,
        scoutPos: 0,
        inspectedParentFail: 0,
    },

```

```

    title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТИ
    desc: '    ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
        ). Рыба плавает в ROOT и ищет ветку "H". У корня
        суффикс "H"!',
    codeLine: 4,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'H',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 3,
    title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
    desc: 'Переходим на Depth 3 к #3 (HER). Родитель
        е подходят. fail[HER] = ROOT.',
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'R',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 6,
    title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
    desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
        но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
    codeLine: 4,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'S',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 9,
    title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
    desc: '    МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
        ба плавает в #1 (H) и ищет ветку "E". У вершины
    codeLine: 4,

```

```

    {/* Переключатель режимов */}
    <div className="bg-slate-900 p-1.5 rounded-xl b
      <button
        onClick={() => setActiveMode('training')}
        className={`flex items-center space-x-1.5 p
          activeMode === 'training'
            ? 'bg-indigo-600 text-white shadow-lg s
            : 'text-slate-400 hover:text-white'
          `}
      >
      <GitBranch className="w-4 h-4" />
      <span>Режим 1: Полное обучение (BFS от корн
    </button>
    <button
      onClick={() => setActiveMode('search')}
      className={`flex items-center space-x-1.5 p
        activeMode === 'search'
          ? 'bg-blue-600 text-white shadow-lg sha
          : 'text-slate-400 hover:text-white'
        `}
      >
      <Play className="w-4 h-4" />
      <span>Режим 2: Поиск в тексте</span>
    </button>
  </div>
</div>

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? {
  /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
  <div className="grid grid-cols-1 lg:grid-cols-3
    {/* Управление шагами обучения */}
    <div className="bg-slate-900/90 p-5 rounded-x
      <div>
        <h5 className="text-indigo-300 font-bold
          <span> </span> Полный обход в ширину (В
        </h5>
        <p className="text-xs text-slate-300 mb-3

```

```

        </button>
      </div>
    </div>

    /* Блок с кодом построения fail-ссылки и опи
    <div className="lg:col-span-2 bg-slate-900/90
      >
      <div>
        <h5 className="text-white font-bold mb-3
          <span className="flex items-center gap-
            <span>{</span> Код BFS построения fail
          </span>
          <span className="text-xs bg-indigo-950
            Вершина: #{targetTrainingNode.id} ({t
          </span>
        </h5>
        <div className="bg-slate-950 p-4 rounded-
          <div className={`p-1.5 rounded flex ite
            -l-4 border-indigo-500 text-indigo-300 font
              <span>1. let u = trie[r][char]; // Те
                {currentTrainInfo.codeLine === 1 && <
            тут</span>}
          </div>
          <div className={`p-1.5 rounded flex ite
            -l-4 border-indigo-400 text-indigo-200 font
              <span>2. let v = fail[r]; // Подъем к
            ainInfo.inspectedParentFail].label}}</span>
                {currentTrainInfo.codeLine === 2 && <
            к родителю</span>}
          </div>
          <div className={`p-1.5 rounded flex ite
            l-4 border-amber-500 text-amber-300 font-bo
              <span>3. while (v !== root && trie[v]
                {currentTrainInfo.codeLine === 3 && <
            т, возврат в корень</span>}
          </div>
          <div className={`p-1.5 rounded flex ite
            r-l-4 border-emerald-500 text-emerald-300 f

```

```

        value={textToScan}
        onChange={handleTextChange}
        maxLength={15}
        placeholder="ВВЕДИТЕ ТЕКСТ..."
        className="w-full bg-slate-800 border border-blue-500
cus:outline-none focus:border-blue-500"
      />
    </div>

    <div>
      <div className="mb-4 flex flex-wrap gap-1">
        <span className="text-xs text-slate-400">
          {textToScan.split('').map((char, idx) =>
            <span
              key={idx}
              className={`w-7 h-8 flex items-center justify-center
                ${idx === currentIndex ? 'bg-blue-600 text-white shadow' :
                idx < currentIndex ? 'bg-emerald-950/60 text-emerald-500' :
                'bg-slate-800 text-slate-500'}
            >
              {char}
            </span>
          )}
        </span>
      </div>
      {currentIndex >= textToScan.length && (
        <span className="bg-emerald-600 text-white text-sm font-medium">
          ГOTOBO ✓
        </span>
      )}
    </div>

    <button
      onClick={stepSearchSimulation}
      disabled={isSearchDone || textToScan.length === 0}
      className={`w-full py-2.5 rounded-lg font-medium text-sm
        ${isSearchDone || textToScan.length === 0 ? 'bg-slate-800 text-slate-500' :

```

```

        {activeSearchCodeLine === 4 && <span>
овпадение!</span>}
      </div>
    </div>
  </div>

  <div>
    <h5 className="text-slate-300 font-bold m-0">
      <AlertCircle className="w-3.5 h-3.5 text-slate-300"/>
    </h5>
    <div className="bg-slate-800 p-3.5 rounded-2xl items-center">
      {searchLog}
    </div>
  </div>
</div>
})

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent shadow-2xl">
  {/* Водопадные фоновые эффекты */}
  <div className="absolute inset-0 opacity-10 bg-gradient-to-b from-blue-900 to-transparent pointer-events-none"></div>
  <div className="absolute top-0 left-1/2 -translate-x-1/2">
    <div className="w-8 bg-blue-400 h-full animate-pulse"></div>
    <div className="w-12 bg-blue-300 h-full animate-pulse"></div>
    <div className="w-6 bg-blue-500 h-full animate-pulse"></div>
  </div>

  <div className="flex items-center justify-between p-4">
    <h5 className="text-white font-bold text-base">
      <span> </span> Ветвящийся синий водопад (Борис)
    </h5>
    <div className="flex flex-wrap gap-4 text-xs">
      <span className="flex items-center gap-1 text-white">
        <span className="w-3 h-0.5 bg-blue-500 inline-block"></span>

```

```

    /* Рендер прямых переходов (потоков воды)
    {Object.entries(TRIE_TRANSITIONS).map(([fromNode, toNode]) => {
      const fromNode = WATERFALL_NODES[Number(fromNode)];
      return Object.entries(children).map(([char, toNode]) => {
        const toNode = WATERFALL_NODES[toNode];

        // Подсветка в режиме поиска
        let isHighlighted = activeMode === 'search' ||
        уpe === 'normal';

        // Подсветка в режиме обучения (когда и
        let isTrainingExamined = activeMode === 'training' ||
        let isMatchBranch = isTrainingExamined & isMatchBranch;

        return (
          <g key={`edge-${fromNode.id}-${toNode.id}`}>
            /* Линия течения воды */
            <line
              x1={fromNode.x}
              y1={fromNode.y}
              x2={toNode.x}
              y2={toNode.y}
              stroke={isHighlighted ? '#3b82f6' : '#ccc'}
              strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
              className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
            />
            /* Символ перехода */
            <circle cx={{fromNode.x + toNode.x} / 2} cy={{fromNode.y + toNode.y} / 2}
              r={10} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5b') : 'white'}
              stroke={isHighlighted ? '#3b82f6' : '#ccc'}
            />
            <text
              x={{(fromNode.x + toNode.x) / 2}}
              y={{(fromNode.y + toNode.y) / 2 + 15}}
              fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5b') : 'white'}
              fontSize="12"
              fontWeight="bold"
              textAnchor="middle"
            />
          </g>
        );
      });
    });
  );
}

```



```

    return (
      <g key={`fail-${node.id}`}>
        <path
          d={`M ${node.x} ${node.y} Q ${cx} $
          fill="none"
          stroke={isHighlighted ? '#f43f5e' :
          strokeWidth={isHighlighted || isJus
          strokeDasharray="6,6"
          className={isHighlighted || isJustE
          opacity={isHighlighted || isJustEst
        />
        {isJustEstablished && (
          <text x={cx} y={cy - 10} fill="#10b
        >
          fail[{node.label}] = {targetNode.
        </text>
      )}
    </g>
  );
}}

```

```

{/* Рендер вершин (бассейнов водопада) */}
Object.values(WATERFALL_NODES).map((node) => {
  const isCurrent = activeFishPosNode.id === node.id;
  const isTargetChild = activeMode === 'target';
  const hasWords = node.words.length > 0;

```

```

  return (
    <g key={`node-${node.id}`} className="node">
      {/* Внешнее свечение для активной вершины */}
      {isCurrent && (
        <circle cx={node.x} cy={node.y} r="10" fill="none" stroke="#f43f5e" stroke-width="2" />
      )}
      {isTargetChild && (
        <circle cx={node.x} cy={node.y} r="10" fill="none" stroke="#10b9d1" stroke-width="2" />
      )}
    </g>
  );
}

```

```

        *
      </text>
    </g>
  })
</g>
);
}}}
```

*{/\* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ \*/}*

```

<g>
  {/* Круг-подложка под рыбу */}
  <circle
    cx={activeFishPosNode.x + 15}
    cy={activeFishPosNode.y - 25}
    r="18"
    fill={activeMode === 'training' ? '#8b5'
    stroke="#ffffff"
    strokeWidth="2"
    style={{ transition: 'cx 0.8s cubic-bezi
    className="shadow-lg"
  />
  {/* Сама рыба */}
  <text
    x={activeFishPosNode.x + 15}
    y={activeFishPosNode.y - 19}
    fontSize="18"
    textAnchor="middle"
    style={{ transition: 'x 0.8s cubic-bezi
    className="animate-float select-none"
  >
    {activeMode === 'training' ? '  ♂  ' :
  </text>
</g>

</svg>
</div>
</div>
```

ФАЙЛ 74/87: src/components/visualizers/PrefixSumViz.tsx

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРӨК: 360 | РАЗМЕР: 10.5 КБ

```
import React, { useContext, useEffect, useMemo, useState } from 'react';
import { emitVizDemo, useVizStepSync, VizChapterContext } from 'viz';
```

```
/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * * в 1D наводим на P[i] – подсвечивается кусок массива
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения
 */
```

```
const A1 = [3, 1, 4, 1, 5, 9, 2, 6];
```

```
const A2 = [
  [1, 2, 3, 4],
  [5, 6, 7, 8],
  [9, 1, 2, 3],
  [4, 5, 6, 7],
];
```

```
const cellBase = `
flex items-center justify-center rounded-md font-mono
w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center`;
```

```
export const PrefixSumViz: React.FC = () => {
  const [mode, setMode] = useState<"1d" | "2d">("1d");
  const chapterId = useContext(VizChapterContext);
```

```
// Вкладка 2D = своё демо (свой скелет): компилятор подсчитывает
useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, mode === "2d" ? A2 : A1);
}, [chapterId, mode]);
```

```

* Управляемый режим (отладчик «По шагам»): Р заполняет
* за шаг. null = обычный интерактив (наведение/клик)
*/

```

```

const [driven, setDriven] = useState<number | null>(null);
useVizStepSync(driven ?? 0, setDriven, A1.length);

```

```

const covered =
  hover?.kind === "pref"
    ? { from: 0, to: hover.i - 1 }
    : hover?.kind === "a"
      ? { from: hover.i, to: hover.i }
      : null;

```

```

const sum = pref[range.r + 1] - pref[range.l];

```

```

return (
  <div className="space-y-5">
    <div className="overflow-x-auto pb-1">
      <div className="inline-block min-w-full">
        /* индексы */
        <div className="flex gap-1.5 mb-1 pl-[52px]">
          {A1.map((_, i) => (
            <div key={i} className={` ${cellBase} !h-5`}>
              {i}
            </div>
          ))}
        </div>

```

```

        /* массив a */
        <div className="flex gap-1.5 items-center mb-1">
          <div className="w-[46px] shrink-0 text-right">
            {A1.map((v, i) => {
              const inCover = covered && i >= covered.f;
              const inRange = i >= range.l && i <= range.r;
              const isDrivenAdd = driven !== null && driven === i;
              return (
                <div
                  key={i}

```

```

        ? "bg-slate-800/50 text-slate-700"
        : active
        ? "bg-emerald-500 text-white"
        : isR
        ? "bg-emerald-700 text-white"
        : isL
        ? "bg-rose-700 text-white"
        : "bg-slate-900 text-slate-300"
    }`}
    title={masked ? `P[${i}]` — ещё не посчитано` : v}
  >
  {masked ? "." : v}
</div>
  );
  }}}
</div>
</div>
</div>

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border border-slate-800" >
  {driven !== null && !hover ? (
    <p className="text-slate-300">
      <b className="text-emerald-400">По шагам от  $P[0]$  до  $P[A1.length]$  (последний:  $P[\text{driven}] = \{\text{pref}[\text{driven}]\}$ ) —  $P[\{\text{Math.min}(\text{driven} + 1, A1.length)\}] = P[\text{driven}]$ 
      <span className="font-mono text-amber-400">
         $P[\text{driven}] = \{\text{pref}[\text{driven}]\}$ 
      </span>
    </p>
  ) : hover?.kind === "pref" ? (
    hover.i === 0 ? (
      <p className="text-slate-300">
        <b className="text-emerald-400"> $P[0] = 0$  — длина отрезка, начинающегося с нуля.
      </p>
    ) : (
      <p className="text-slate-300">
        <b className="text-emerald-400">
           $P[\{\text{hover.i}\}] = \{\text{pref}[\text{hover.i}]\}$ 

```

```

        </div>
    </div>
  );
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
  const n = A2.length;
  const m = A2[0].length;

  const P = useMemo(() => {
    const p = Array.from({ length: n + 1 }, () => Array<number>().length);
    for (let i = 1; i <= n; i++) {
      for (let j = 1; j <= m; j++) {
        p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i][j - 1];
      }
    }
    return p;
  }, [n, m]);

  const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
  const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

  const D = P[rect.r1][rect.c1];
  const B = P[rect.r1][rect.c2 + 1];
  const Cc = P[rect.r2 + 1][rect.c1];
  const Aa = P[rect.r2 + 1][rect.c2 + 1];
  const total = Aa - B - Cc + D;

  const cornerOf = (i: number, j: number) => {
    if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
    if (i === rect.r1 && j === rect.c2 + 1) return "B";
    if (i === rect.r2 + 1 && j === rect.c1) return "C";
    if (i === rect.r1 && j === rect.c1) return "D";
    return null;
  };

  return (

```

```

const corner = cornerOf(i, j);
const isHover = hover?.i === i && hover?.j === j;
const cornerColor =
  corner === "A"
    ? "bg-emerald-600 text-white"
    : corner === "D"
    ? "bg-sky-600 text-white"
    : corner === "C"
    ? "bg-rose-600 text-white"
    : "bg-slate-900 border border-slate-700";
return (
  <div
    key={j}
    onMouseEnter={() => setHover({ i, j })}
    onMouseLeave={() => setHover(null)}
    className={` ${cellBase} cursor-pointer ${
      isHover ? "bg-amber-500 text-white" : ""
    }`}
  >
    {v}
    {corner && !isHover && (
      <span className="absolute -top-1 -right-1" style={{
        width: 10px, height: 10px, background-color: cornerColor,
        border: 1px solid black, border-radius: 50%
      }} />
    )}
  </div>
);
}}}
</div>
)}}
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border border-slate-700" style={{
  {hover ? {
    <p className="text-slate-300">

```

```

    Lock,
    ChevronDown,
  } from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () =>
  for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j)]);
    }
  }
);

// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
  [45, 12, 88, 34, 11, 76, 23, 90],
  [67, 19, 44, 55, 33, 21, 65, 87],
  [14, 51, 99, 13, 22, 64, 43, 76],
  [89, 32, 54, 71, 15, 88, 29, 60],
  [25, 41, 16, 92, 9, 17, 56, 31],
  [59, 18, 77, 24, 61, 82, 35, 12],
  [73, 8, 38, 85, 47, 95, 19, 58],
  [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][][] = Array.from({length: 8}, () =>
  Array.from({length: 8}, () =>
    Array.from({length: 4}, () =>
      Array(4).fill(0)
    )
  )
);

for (let r = 0; r < 8; r++) {

```



```

    <button
      onClick={() => setTab("1d")}
      className={`px-3 sm:px-4 py-2 rounded-lg text-
        00 text-white" : "bg-slate-800 text-slate-4
    >
      1. Сворачивание 1D
    </button>
    <button
      onClick={() => setTab("2d_build")}
      className={`px-3 sm:px-4 py-2 rounded-lg text-
        digo-600 text-white" : "bg-slate-800 text-s
    >
      2. Сворачивание 2D (Построение)
    </button>
    <button
      onClick={() => setTab("2d_query")}
      className={`px-3 sm:px-4 py-2 rounded-lg text-
        se-600 text-white" : "bg-slate-800 text-sla
    >
      3. Запрос 2D (Формула)
    </button>
  </div>

  <AnimatePresence mode="wait">
    {tab === "1d" ? (
      <Viz1D key="1d" />
    ) : tab === "2d_build" ? (
      <Viz2DBuild key="2d_build" />
    ) : (
      <Viz2DQuery key="2d_query" />
    )}
  </AnimatePresence>
</div>
);
};

const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);

```

```

        <ChevronRight size={20} />
      </button>
      <button
        onClick={() => setStep(0)}
        className="p-2 bg-slate-800 hover:bg-slate-700"
      >
        <RotateCcw size={20} />
      </button>
    </div>
  </div>

  <div className="bg-slate-900 rounded-xl border border-slate-800" >
    <div className="text-[11px] uppercase tracking-wider" >
      <div className="flex gap-1 sm:gap-1.5 overflow-x-auto" >
        {arr1D.map((v, idx) => {
          const inCover = !!covered && idx >= covered - 1;
          return (
            <div key={idx} className="flex flex-col items-center justify-center" >
              <div
                className={`flex items-center justify-center w-[44px] h-[clamp(26px,8vw,44px)] text-[clamp(9px,1.2em)] ${
                  inCover ? "bg-indigo-500 text-white" : "bg-slate-800 text-slate-300"
                }`}
              >
                {v}
              </div>
              <span className="text-[9px] text-slate-400" >
                {v}
              </span>
            </div>
          );
        })}
      </div>
      <div className="mt-2 text-xs min-h-[36px]" >
        {hover && covered ? (
          <p className="text-slate-300" >
            <b className="text-sky-400" >
              ST[{hover.i}][{hover.j}] = {st1D[hover.i][hover.j]}
            </b>{ " " }
            — это минимум на отрезке{ " " }
          </p>
        ) : (
          <p className="text-slate-400" >
            ST[{hover.i}][{hover.j}] = {st1D[hover.i][hover.j]}
          </p>
        )}
      </div>
    </div>
  </div>

```

</div>

```

    { /* Matrix */
    {Array.from({ length: N }).map((_, i) => {
      <React.Fragment key={i}>
        <div className="text-slate-500 font-mono" style={fontStyle}>
          {i}
        </div>
        {Array.from({ length: LOG }).map((_, j) => {
          const isValid = i + (1 << j) <= N;
          const isVisible =
            isValid &&
            (j === 0 ||
              computeSteps.findIndex((s) => s.i === i + (1 << j) && s.j === j + 1));

          // Active computing state
          let isActive = false;
          let isSrc1 = false;
          let isSrc2 = false;

          if (step > 0 && step <= maxStep) {
            const currentStep = computeSteps[step];
            if (currentStep.i === i && currentStep.j === j + 1 && currentStep.isActive) {
              isActive = true;
            }
            if (currentStep.j === j + 1 && currentStep.i === i + (1 << (currentStep.j - j))) {
              isSrc1 = true;
            }
            if (
              currentStep.j === j + 1 &&
              currentStep.i === i - (1 << (currentStep.i - i))
            ) {
              isSrc2 = true; // wait, no. Src2 is not the source of this cell
            }
            // Let's re-eval exact source:
            // We are asking if THIS cell [i][j] is the source of the next cell
            if (currentStep.j - 1 === j) {
              if (currentStep.i === i) isSrc1 = true;
              if (currentStep.i + (1 << (currentStep.j - j)) === i) isSrc2 = true;
            }
          }

          return (
            <div style={fontStyle} className={
              `text-slate-500 font-mono ${
                !isValid ? 'opacity-50' : ''
              } ${
                !isVisible ? 'opacity-50' : ''
              }`
            }>
              {i + (1 << j)}
            </div>
          );
        })}
      </React.Fragment>
    })}
  </div>
)

```

```

        <motion.svg
          initial={{ opacity: 0 }}
          animate={{ opacity: 1 }}
          className="absolute pointer-events-none"
          style={{
            width: 100,
            height: isSrc2 ? 40 + (1 << 2) : 40,
            right: -50,
            top: 24,
            overflow: "visible",
          }}
        >
          <path
            d={`M 0 0 C 30 0, 30 ${isSrc2 ? 40 : 40} 0`}
            fill="none"
            stroke={isSrc1 ? "#10b981" : "#10b981"}
            strokeWidth="3"
          />
        </motion.svg>
      </div>
    </div>
  );
}
</React.Fragment>
)
</div>
</div>
</div>

```

```

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-800 p-4">
  {step > 0 && step <= maxStep ? (
    (() => {
      const c = computeSteps[step - 1];
      const half = 1 << (c.j - 1);
      return (
        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">

```



```

highlightClass = 'bg-ros
md:scale-[1.08]';
    } else if (isTop && !isLeft
        highlightClass = 'bg-blur
] md:scale-[1.08]';
    } else if (!isTop && isLeft
        highlightClass = 'bg-eme
9,0.3)] md:scale-[1.08]';
    } else {
        highlightClass = 'bg-amb
3)] md:scale-[1.08]';
    }
    zIndex = 'z-10';
  }
}

return {
  <div
    key={idx}
    onMouseEnter={() => { if(is
    onMouseLeave={() => { if(is
    onClick={() => {
      if(isCurr) {
        if(locked && locked.
        else setLocked({r, c
      }
    }}
    className={`flex items-cent
sCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp
clamp(20px,5.2vw,30px)] text-[clamp(8px,2vw
>
    {ST2D[r][c][step][step]}
  </div>
);
  }}}
</div>
</div>
</div>

```

```

        &nbsp;&nbsp;<span className="text-amber-500">c + pL</span>][k-<span className="text-slate-500">// Правый Низ</span><b>
        {'}'}));
    </div>

    {activeCell ? (
      <div className="bg-slate-950 p-5 rounded-2xl mt-2 transition-all">
        <p className="text-slate-300 mb-3">
          <span>Пример для ячейки:</span>
          {locked && <span className="text-rose-500 border border-rose-500/20"><Lock size={
        </p>
        <p className="text-indigo-300 border">
          <span className="font-bold text-white">
            }}[k]][k]</span>
          <span className="text-slate-400">
        </p>

        <div className="space-y-2 pt-3 pl-2">
          {(() => {
            const prevL = 1 << (k - 1);
            return (
              <>
                <div className="flex items-center">
                  <span className="flex-shrink-0 h-8 w-8 bg-slate-800 shadow-[0_0_8px_#f43f5e]"></div> ST[{activeCell.r}[activeCell.c][k-1][k-1]}</span>
                </div>
                <div className="flex items-center">
                  <span className="flex-shrink-0 h-8 w-8 bg-slate-800 shadow-[0_0_8px_#3b82f6]"></div> ST[{activeCell.r}[activeCell.c + prevL][k-1][k-1]}</span>
                </div>
                <div className="flex items-center">

```





```

        </div>
      </label>
      <label className="flex items-center" >
        <span className="text-slate-500" >
          <div className="flex gap-2">
            <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
            <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
          </div>
        </label>
      </div>
    </div>
  </div>

```

```

<div className="flex-1 bg-slate-900 p-6 rounded" >
  <h3 className="text-xl font-bold text-emerald-400" >
    <p className="text-sm font-sans text-slate-400" >
      Мы знаем, что максимальная степень дв
    rounded text-white">{Lx}</span>. Аналогично
    te">{Ly}</span>.
  </p>
  <p className="text-sm font-sans text-slate-400" >
    Чтобы покрыть весь прямоугольник, мы
    Name="font-mono bg-[#0a0f1e] px-1 rounded b
    роса. (Цветные прямоугольники слева).
  </p>

```

```

    <div className="bg-[#0a0f1e] rounded-xl p-6" >
      text-slate-300 shadow-inner overflow-x-auto" >
        <div className="absolute right-3 top-3" >
          <span className="text-slate-500"> // 1.
          <span className="text-purple-400">int<
        ext-amber-300">1</span>); <span className="
          <span className="text-purple-400">int<
        ext-amber-300">1</span>); <span className="
          <span className="text-purple-400">int<
        "text-slate-500"> // Выс: {Lx}</span><br/>

```

```

        let isTL = r === r1 && c === c1;
        let isTR = r === r1 && c === c2;
        let isBL = r === r2 - Lx + 1 && c === c1;
        let isBR = r === r2 - Lx + 1 && c === c2;

        let color = "bg-slate-800 text-white";
        let txt = ST2D[r][c][kx][ky].text;

        let badge = null;
        if (isTL) { color = "bg-rose-400 text-white";
0 border-2"; badge = "TL"; }
        else if (isTR) { color = "bg-blue-300 text-white";
ue-300 border-2"; badge = "TR"; }
        else if (isBL) { color = "bg-emerald-300 text-white";
-emerald-300 border-2"; badge = "BL"; }
        else if (isBR) { color = "bg-amber-300 text-white";
mber-300 border-2"; badge = "BR"; }

        return (
            <div key={idx} className={
tion-all duration-300 relative shrink-0 ${color}
            {txt}
            {badge && <div className="text-slate-700 text-white px-1 font-bold rounded
            </div>
        )
    }
  }
</div>
</div>
</div>

<div className="mt-auto bg-slate-950 font-sans text-white text-sm
)]">
    <div className="text-center font-bold text-white bg-[#0a0f1e] py-3 px-2 rounded-lg border
    <span className="text-slate-400 font-sans text-sm">
    <span className="text-rose-400 font-sans text-sm">
    <span className="text-blue-400 font-sans text-sm">

```

```
      : 'px-3 py-1 rounded text-sm font-bold'

const aside = document.querySelector('aside')
const headerMain = document.querySelector('h1')
const questionsContainer = document.getElementById('questions')
const wtfContainer = document.getElementById('wtf')

if (mode === 'wtf') {
  document.body.classList.add('wtf-mode')
  if (aside) aside.style.display = 'none'
  if (headerMain) headerMain.style.display = 'none'
  if (questionsContainer) questionsContainer.style.display = 'none'
  if (wtfContainer) wtfContainer.classList.remove('hidden')

  // Remove the URL hash to avoid scrolling
  history.pushState("", document.title, window.location.pathname + window.search)
  window.scrollTo(0, 0);
} else {
  document.body.classList.remove('wtf-mode')
  if (aside) aside.style.display = '';
  if (headerMain) headerMain.style.display = '';
  if (questionsContainer) questionsContainer.style.display = '';
  if (wtfContainer) wtfContainer.classList.add('hidden')
}

function setTheme(theme) {
  const body = document.body;
  body.classList.remove('theme-dark', 'theme-light');
  body.classList.add('theme-' + theme);

  // Just some visual feedback on buttons
  document.querySelectorAll('[id^="theme-"]').forEach(btn => {
    btn.classList.remove('outline', 'outline-1');
  });

  if (theme === 'dark') document.getElementById('dark-button').classList.add('active')
  if (theme === 'light') document.getElementById('light-button').classList.add('active')
```

```

        setTimeout(() => drawerOverlay.
    }
    document.body.style.overflow = "";
}
}

if (mobileMenuBtn && mobileDrawer) {
    mobileMenuBtn.addEventListener("click",
}
if (closeDrawerBtn && mobileDrawer) {
    closeDrawerBtn.addEventListener("click"
}
if (drawerOverlay) {
    drawerOverlay.addEventListener("click",
}

// Close on link click
document.querySelectorAll("#mobile-drawer a
    link.addEventListener("click", () => {
        if (mobileDrawer && !mobileDrawer.c
            toggleDrawer();
    })
});

const observerOptions = {
    root: null,
    rootMargin: '-10% 0px -70% 0px',
    threshold: 0
};

const observer = new IntersectionObserver((
    entries.forEach(entry => {
        if (entry.isIntersecting && !document
            // Highlight active TOC link lo
            const id = entry.target.id;
            document.querySelectorAll('aside
                if (link.getAttribute('href

```

```

    };
</script>
<style>
    @import url('https://fonts.googleapis.com/css2?
    @reference './src/index.css';
    .math-font { font-family: 'Fira Code', monospace; }
    .uno-card {
        width: 70px; height: 105px; border-radius: 10px;
        box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
        align-items: center; justify-content: center; text-align: center;
        color: white; overflow: hidden; flex-shrink: 0;
        text-shadow: 2px 2px 0 #000, -1px -1px 0 #000, 1px -1px 0 #000;
        font-family: 'Arial Black', Impact, sans-serif;
    }
    .uno-card::before, .uno-card::after {
        content: attr(data-val); position: absolute;
        text-shadow: 1px 1px 0 #000, -1px -1px 0 #000, 1px -1px 0 #000;
    }
    .uno-card::before { top: 2px; left: 4px; }
    .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
    .uno-card.mini { width: 45px !important; height: 60px !important; }
    .uno-card.mini::before, .uno-card.mini::after {
        font-size: 0.8em;
    }

    /* Global formula scroll settings */
    .formula-scroll {
        overflow-x: auto;
        overflow-y: hidden;
        -webkit-overflow-scrolling: touch;
        scrollbar-width: none; /* Firefox */
        -ms-overflow-style: none; /* IE and Edge */
    }
    .formula-scroll::-webkit-scrollbar {
        display: none;
    }

    mjx-container {
        max-width: 100% !important;
    }

```

```

    .card-blue { background-color: #004dcf; } .card
    .card-green { background-color: #339e35; } .card
    .card-yellow { background-color: #ffc400; } .ca
    .card-black { background-color: #222; } .card-b
    .card-black::before, .card-black::after { text-
    .info-block { @apply bg-slate-800/80 border-l-4
    .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
    ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
    king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
    r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 for

/* THEME OVERRIDES */
body.theme-light {
    background-color: #f8fafc !important;
    color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-light
    important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-col
body.theme-light #top-controls { background-col

body.theme-notebook {
    background-color: #fcf8eb !important; /* li
    color: #3b2818 !important; /* brown ink */
    background-image: repeating-linear-gradient
    background-attachment: local !important;
    font-family: "Comic Sans MS", "Caveat", "Se

```

```

</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

  <div class="sticky top-0 z-50 w-full bg-slate-950/90
    center items-center gap-4 sm:gap-8 transition-colors">
    <div class="flex items-center gap-3">
      <span class="text-sm font-bold uppercase tracking-wider">Режим</span>
      <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
        <button onclick="setMode('normal')" id="btn-normal"
          transition-colors">Список тем</button>
        <button onclick="setMode('wtf')" id="btn-wtf"
          transition-colors">WTF</button>
      </div>
    </div>
    <div class="flex items-center gap-3">
      <span class="text-sm font-bold uppercase tracking-wider">Тема</span>
      <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
        <button onclick="setTheme('dark')" id="btn-dark"
          transition-colors">Темная</button>
        <button onclick="setTheme('light')" id="btn-light"
          transition-colors">Светлая</button>
        <button onclick="setTheme('notebook')" id="btn-notebook"
          transition-colors">Notebook</button>
      </div>
    </div>
  </div>

  <!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
  <svg width="0" height="0" class="absolute">
    <defs>
      <marker id="arrow-yellow" markerWidth="6" markerHeight="6"
        viewBox="0 0 6 6" fill="yellow">
        <path d="M0,0 L6,3 L0,6 Z" fill="yellow">
      </marker>
      <marker id="arrow-orange" markerWidth="6" markerHeight="6"
        viewBox="0 0 6 6" fill="orange">
        <path d="M0,0 L6,3 L0,6 Z" fill="orange">
      </marker>
      <marker id="arrow-pink" markerWidth="6" markerHeight="6"
        viewBox="0 0 6 6" fill="pink">
        <path d="M0,0 L6,3 L0,6 Z" fill="pink">
      </marker>
    </defs>
  </svg>

```

```

        <path d="M 45 50 Q 60 50 60 30 L 75 30"
        <circle cx="75" cy="30" r="4" fill="#333"
        <!-- Выход -->
        <path d="M 5 45 L -5 45 L -5 55 L 5 55"
        <circle cx="25" cy="50" r="15" fill="#f
    </g>
    <g id="lego-green">
        <rect x="0" y="10" width="20" height="15"
        <rect x="3" y="5" width="14" height="5"
    </g>
    <g id="lego-white">
        <rect x="0" y="0" width="40" height="20"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="0" y1="2" x2="40" y2="2" stro
        <text x="20" y="14" fill="#475569" font
    </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
    0/50 z-50 hover:bg-blue-500 transition-transform
        <svg xmlns="http://www.w3.org/2000/svg" class="
            <path stroke-linecap="round" stroke-linejoin="
        </svg>
    </button>

<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed inset-0
    background-color: #333; opacity: 0.5; z-index: 1000;

<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-wrap
    justify-between" style="background-color: #f9f9f9; padding: 10px;">

    <!-- Левое меню (Оглавление) -->
    <aside id="mobile-drawer" class="fixed inset-y-0 left-0
        translate-x-full transition-transform duration-300
        w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
        <div class="h-full flex flex-col p-6 overflow-y-auto"

```



```

        <a href="#section-2-2" class="block">
        <a href="#section-2-3" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-teal-500 pl-3 font-bold text-teal-300">
        <a href="#question-3" class="block">
            3. Тригонометрическая форма
        </a>
    </summary>
    <div class="pl-6 space-y-2 text-teal-300">
        <a href="#section-3-1" class="block">
        <a href="#section-3-2" class="block">
        <a href="#section-3-3" class="block">
        <a href="#section-3-4" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">
        <a href="#question-4" class="block">
            4. Степени и корни
        </a>
    </summary>
    <div class="pl-6 space-y-2 text-amber-300">
        <a href="#q4-def" class="block">
        <a href="#q4-moivre" class="block">
        <a href="#q4-root" class="block">
        <a href="#q4-count" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">
        <a href="#question-5" class="block">
```

```

        <div class="pl-6 space-y-2 text
          <a href="#q7-1" class="bloc
          <a href="#q7-2" class="bloc
          <a href="#q7-3" class="bloc
          <a href="#q7-4" class="bloc
          <a href="#q7-5" class="bloc
        </div>
      </details>

      <details class="group">
        <summary class="list-none curso
          <a href="#question-8" class:
er-violet-500 pl-3 font-bold text-violet-300">
            8. Сравнимость и Поля В
          </a>
        </summary>
        <div class="pl-6 space-y-2 text
          <a href="#q8-1" class="bloc
          <a href="#q8-2" class="bloc
          <a href="#q8-3" class="bloc
          <a href="#q8-4" class="bloc
          <a href="#q8-5" class="bloc
          <a href="#q8-6" class="bloc
          <a href="#q8-7" class="bloc
        </div>
      </details>

      <details class="group">
        <summary class="list-none curso
          <a href="#question-9" class:
er-green-500 pl-3 font-bold text-green-400">
            9. Кольцо многочленов
          </a>
        </summary>
        <div class="pl-6 space-y-2 text
          <a href="#q9-1" class="bloc
          <a href="#q9-2" class="bloc
          <a href="#q9-3" class="bloc
```

```
    <details class="group">
      <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-400">
        12. Взаимно простые мно
      </a>
    </summary>
    <div class="pl-6 space-y-2 text-blue-500">
      <a href="#q12-1" class="block">
      <a href="#q12-2" class="block">
      <a href="#q12-3" class="block">
        еление)</a>
    </div>
  </details>
```

```
    <details class="group">
      <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-400">
        13. Корни многочленов.
      </a>
    </summary>
    <div class="pl-6 space-y-2 text-emerald-500">
      <a href="#q13-1" class="block">
      <a href="#q13-2" class="block">
      <a href="#q13-3" class="block">
    </div>
  </details>
```

```
    <details class="group">
      <summary class="list-none cursor-pointer text-fuchsia-500 pl-3 font-bold text-fuchsia-400">
        14. Кратность корня. Ко
      </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2 text-fuchsia-500">
```

```

<div class="mt-4 mb-2 text-xs font-l
  <a href="#question-17" class="block
-500 pl-3 text-slate-400">
    <span class="font-bold text-tea
  </a>
  <a href="#question-18" class="block
-500 pl-3 text-slate-400">
    <span class="font-bold text-blur
  </a>
  <a href="#question-19" class="block
ald-500 pl-3 text-slate-400">
    <span class="font-bold text-eme
  </a>
  <a href="#question-20" class="block
-500 pl-3 text-slate-400">
    <span class="font-bold text-ros
  </a>
  <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
    <span class="font-bold text-ind
  </a>
  <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
    <span class="font-bold text-fuc
  </a>

  <a href="#proof-algorithms" class="
-red-500 pl-3 mt-4 text-red-500 font-bold m

    ▲ Ликбез: Алгоритмы выживания
  </a>

</nav>
<script>
  document.querySelectorAll('details.
    detail.addEventListener('toggle
      if (detail.open) {
        document.querySelectorA
          if (other !== detail

```

```

        const corresponding
        if (correspondingLi
            const details =
            if (details &&
                details.open
                document.qu
                if (othe
                othe
            }
        }));
    }
}
});
}, {
    root: null,
    rootMargin: '-10% 0px -70%
    threshold: [0, 0.5]
});

sections.forEach(section => {
    observer.observe(section);
});
});

function fallbackModal(contentRaw,
    const modal = document.createEl
    modal.style.position = 'fixed';
    modal.style.top = '0';
    modal.style.left = '0';
    modal.style.width = '100vw';
    modal.style.height = '100vh';
    modal.style.backgroundColor = '
    modal.style.zIndex = '99999';
    modal.style.display = 'flex';
    modal.style.flexDirection = 'co
    modal.style.alignItems = 'cente
    modal.style.justifyContent = 'c

```

```

copyBtn.style.backgroundColor =
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
    if (navigator.clipboard) {
        navigator.clipboard.writeText(textarea.value);
        copyBtn.innerText = 'Скопировано';
    } else {
        textarea.select();
        document.execCommand('copy');
        copyBtn.innerText = 'Скопировано';
    }
    setTimeout(() => copyBtn.innerText = 'Копировать', 1000);
};

```

```

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#333';
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';
closeBtn.style.cursor = 'pointer';
closeBtn.onclick = () => modal.remove();

```

```

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
    box.appendChild(textarea);
    btnContainer.appendChild(copyBtn);
}
btnContainer.appendChild(closeBtn);
box.appendChild(btnContainer);
modal.appendChild(box);
document.body.appendChild(modal);
}

```



```

        } else if (theme === 'terminal') {
            body.classList.add('bg-black');
            document.getElementById('theme-selector').style.innerHTML = `
                body.bg-black .bg-slate-950 {
                    background-color: transparent;
                }
                body.bg-black section h1,
                body.bg-black button {
                    color: #4ade80 !important;
                    border-color: #22c55e !important;
                    font-family: monospace;
                }
                body.bg-black svg path,
                body.bg-black .text-slate-950 {
                    stroke: #22c55e !important;
                    fill: transparent !important;
                }
            `;
        }

        document.head.appendChild(style);
    }
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 border-1px border-slate-950 p-4">
    <!-- Кнопки Конвертации / Экспорта
    <div class="flex flex-col gap-2">
        <div class="text-xs text-slate-950 font-medium">
            <button onclick="downloadWord()"
            font-bold flex items-center justify-center gap-2">
                <span> </span> Word
            </button>
            <a href="algebra.pdf" target="_blank"
            -sm font-bold flex items-center justify-center

```



## ВЫСШАЯ АЛГЕБРА

</h1>

<p class="text-slate-400 italic text-lg">

<!-- Print Table of Contents -->

<div class="hidden print:block mt-8 text-center: always;">

<h2 class="font-bold text-xl mb-4">

<ul class="text-base space-y-2">

<li><a href="#question-1">1. Ал

<li><a href="#question-2">2. Ко

<li><a href="#question-3">3. Тр

<li><a href="#question-4">4. Ст

<li><a href="#question-5">5. Гр

<li><a href="#question-6">6. Де

<li><a href="#question-7">7. Ал

<li><a href="#question-8">8. Кл

<li><a href="#question-9">9. Ко

<li><a href="#question-10">10. ,

<li><a href="#question-11">11. <

<li><a href="#question-12">12. l

<li><a href="#question-13">13. l

<li><a href="#question-14">14. l

<li><a href="#question-15">15. <

<li><a href="#question-17">17. l

<li><a href="#question-18">18. >

<li><a href="#question-19">19. l

<li><a href="#question-20">20. l

<li><a href="#question-21">21. l

<li><a href="#question-22">22. l

</ul>

</div>

</header>

<!-- ===== WTF CONTAINER =====>

<div id="wtf-container" class="hidden">

<h2 class="text-3xl font-black text-cen

```

-----
        <a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
    </a>

    <!-- Row 3: НОД и НОК / Алгоритм Ев
    <a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
    </a>
    <a href="#question-12" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
    </a>
    <!-- Empty 3rd row cell -->
    <div class="hidden md:block"></div>

    <!-- Row 4: Факторизация / Структур
    <a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
    </a>
    <a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
    </a>
    <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
    </a>

    <!-- Row 5: Корни / Уравнения -->

```

```

        </a>
        <a href="#question-18" onclick="setf
-l-4 border-blue-500 hover:bg-slate-700 tra
        <div class="text-xs text-blue-50
        <div class="font-bold text-slate-700"
        </a>
        <a href="#question-19" onclick="setf
-l-4 border-emerald-500 hover:bg-slate-700
        <div class="text-xs text-emerald-500
        <div class="font-bold text-slate-700"
        </a>

        <a href="#question-20" onclick="setf
-l-4 border-rose-500 hover:bg-slate-700 tra
        <div class="text-xs text-rose-500
        <div class="font-bold text-slate-700"
        </a>
        <a href="#question-21" onclick="setf
-l-4 border-indigo-500 hover:bg-slate-700 t
        <div class="text-xs text-indigo-500
        <div class="font-bold text-slate-700"
        </a>
        <a href="#question-22" onclick="setf
-l-4 border-fuchsia-500 hover:bg-slate-700
        <div class="text-xs text-fuchsia-500
        <div class="font-bold text-slate-700"
        </a>

        <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ
        <div class="col-span-1 md:col-span-3
border-b border-red-500/30 pb-2">Кластер 4:

        <a href="#proof-algorithms" onclick="setf
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
        <div class="text-xs text-red-500
        <div class="font-bold text-slate-700"
        </a>
    </div>

```

```

<div class="bg-slate-800/80 p-6" style="background-color: #1a2b3c; color: white;">
  <h3 class="text-xl text-fuchsia" style="color: #ff00ff; margin-bottom: 10px;">
    <p class="text-sm text-slate-400" style="color: #a0b0c0; margin-bottom: 10px;">
      <ul class="list-disc pl-5" style="list-style-type: none;">
        <li><span class="text-white" style="color: white;">
          i>
          <li><span class="text-white" style="color: white;">
            т.д.).</li>
            <li><span class="text-white" style="color: white;">
              что так и задумано: "Ой, противоречие, теорема".</li>
            </ul>
          </div>

```

```

<!-- Единственность -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a2b3c; color: white;">
  <h3 class="text-xl text-emerald" style="color: #00ff00; margin-bottom: 10px;">
    <p class="text-sm text-slate-400" style="color: #a0b0c0; margin-bottom: 10px;">
      <ul class="list-disc pl-5" style="list-style-type: none;">
        <li><span class="text-white" style="color: white;">
          ,  $r_2$ ).</li>
          <li><span class="text-white" style="color: white;">
          <li><span class="text-white" style="color: white;">
            счет ограничений (размер остатка) докажи, что не существует.</li>
          </ul>
        </div>

```

```

<!-- Индукция -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a2b3c; color: white;">
  <h3 class="text-xl text-yellow" style="color: #ffff00; margin-bottom: 10px;">
    <p class="text-sm text-slate-400" style="color: #a0b0c0; margin-bottom: 10px;">
      <ul class="list-disc pl-5" style="list-style-type: none;">
        <li><span class="text-white" style="color: white;">
          валось.</li>
          <li><span class="text-white" style="color: white;">
            Просто перепиши формулу с буквой  $k$ .</li>
            <li><span class="text-white" style="color: white;">
              усок  $k$ , замени его на жареного карася из пункта 1.</li>
          </ul>

```

```
        <p class="text-sm text-slate-400">
        <p class="text-xs font-medium text-slate-400">
ициентов (они делятся на $p$), кроме старшего,
зложить.</p>
```

```
</div>
```

```
    <div class="bg-slate-800 p-4">
      <h4 class="text-red-400 font-medium">
      <p class="text-sm text-slate-400">
      <p class="text-xs font-medium text-slate-400">
. Чаще всего после этого Фейс-контроль $p$ .
```

```
</div>
```

```
</div>
```

```
    <h2 class="text-2xl font-bold text-slate-400">
(Правила экстрасенса)</h2>
    <p class="text-sm text-slate-400">
отношения), а ты их не помнишь. Юзай базовые
нейность) для делимости.</p>
```

```
    <ul class="space-y-4 pb-10">
      <li class="bg-slate-800/50 p-4">
        <span class="text-2xl">
        <div>
          <h4 class="text-cyan-400 font-medium">
          <p class="text-sm text-slate-400">
          <code class="text-x"
        </div>
      </li>
```

```
      <li class="bg-slate-800/50 p-4">
        <span class="text-2xl">
        <div>
          <h4 class="text-cyan-400 font-medium">
          <p class="text-sm text-slate-400">
х двух, потом последних. <br>
          <code class="text-x
```

Универсальное пособие • полный исходный код

---

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 79/87: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 228 | РАЗМЕР: 9.8 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили защиты внутрь, интерактив
 * его можно открыть с флешки, отправить в мессенджер и т.д.
 * Живые React-визуализации в статику не переносятся — вместо них
 * подставляется заметка со ссылкой на онлайн-версию, а для элементов
 * которые в приложении всплывают по наведению, собираю их отдельно
 * в конце документа.
 */

/** Собирает CSS страницы: сначала пробуем правила, при отсутствии — fallback
 * async function collectCss(): Promise<string> {
 *   const parts: string[] = [];
 *   for (const sheet of Array.from(document.styleSheets)) {
 *     try {
 *       const rules = (sheet as CSSStyleSheet).cssRules;
 *       parts.push(Array.from(rules).map((r) => r.cssText));
 *     } catch {
 *       // ignore
 *     }
 *   }
 *   return parts.join("\n");
 * }
```

```

}
`;

const escapeHtml = (s: string) =>
  s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;");

/**
 * Размечает термины и превращает их в якорные ссылки на
 * Возвращает готовый HTML главы и список найденных терминов
 */
function prepareBody(html: string): { body: string; termIds: string[] } {
  const host = document.createElement("div");
  host.innerHTML = html;

  try {
    annotateTerms(host);
  } catch {
    /* если браузер не осилил регулярку – выгружаем текст как есть */
  }

  const ids: string[] = [];
  host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
    const id = el.dataset.termId;
    if (!id) return;
    if (!ids.includes(id)) ids.push(id);
    const link = document.createElement("a");
    link.className = "term-hint";
    link.href = `#term-${id}`;
    link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
    link.textContent = el.textContent ?? "";
    el.replaceWith(link);
  });

  // <details> в файле лучше раскрыть: иначе код и разбегается
  host.querySelectorAll("details").forEach((d) => d.setOpen());

  return { body: host.innerHTML, termIds: ids };
}

```

```

<div style="font-size:.7rem;text-transform:uppercase"
<h1 style="color:#fff;font-size:1.6rem;font-weight:
<div style="font-size:.72rem;color:#475569;margin-t
</header>

<div class="export-note">
  Это автономная копия: стили внутри файла, интернет
  Интерактивные тренажёры и анимации в статичный HTML
  они живут в онлайн-версии: <a href="${escapeHtml(op
</div>

<article class="prose prose-invert max-w-none">
${opts.body}
</article>

${opts.gloss}

<footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}

```

```

function triggerDownload(html: string, filename: string) {
  const blob = new Blob([html], { type: "text/html;charset=utf-8" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  a.remove();
  setTimeout(() => URL.revokeObjectURL(url), 2000);
}

```

```

/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
  const num = title.match(/^s*(\d+)/)?.[1];

```



```
-----
const body = `
```

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

-----  
ФАЙЛ 80/87: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

```
-----
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 *   node scripts/audit-coverage.mjs           # отчёт
 *   node scripts/audit-coverage.mjs --md      # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
```

```

});

const has2D = (r) => r.interactive || r.inlineSvg || r.
const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && !
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D
const orphanViz = [...vizIds].filter((id) => !chapters.

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) =>

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
  console.log("| # | Глава | Аналогия | 2D-виз. | SVG |
  console.log("| --- | --- | --- | --- | --- |");
  for (const r of rows) {
    console.log(
      `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy
        ge)} |`
    );
  }
} else {
  console.log(`Глав в пособии: ${rows.length}\n`);
  const dump = (title, list) => {
    console.log(`${title} (${list.length}):`);
    list.forEach((r) => console.log(`  * ${r.title} [$
    console.log();
  };
  dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
  dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
  dump("НЕТ 2D (аналогия есть)", gapViz);
  console.log(`Пропущенные номера билетов 1-24: ${missi
  console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

```

-----
for (const c of chapters) {
  const text = c.content
    .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi)
    .replace(/<[^>]+>/g, " ");

  const perTerm = new Map();
  const perSurface = new Map();
  let highlights = 0;
  const ids = new Set();

  for (const m of text.matchAll(re)) {
    const surface = m[1].toLowerCase();
    const id = map.get(surface);
    if (!id) continue;
    const ut = perTerm.get(id) ?? 0;
    const us = perSurface.get(surface) ?? 0;
    if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
    perTerm.set(id, ut + 1);
    perSurface.set(surface, us + 1);
    highlights += 1;
    ids.add(id);
    used.add(id);
  }

  if (ids.size === 0) bad.push(c.title);
  console.log(
    String(ids.size).padStart(3), "|", String(highlights).padStart(3),
    c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 10).join(", ")
  );
}
console.log("\nГлав без единой подсказки:", bad.length,);
console.log("Терминов в глоссарии:", GLOSSARY.length,);
console.log("Не встретились:", GLOSSARY.filter((g) => !ids.has(g)).length,);

```

```
        CreationDate: new Date(),
      },
    ));

    const stream = fs.createWriteStream(PDF_PATH);
    doc.pipe(stream);

    for (const file of pages) {
      doc.addPage({ size: [A4.width, A4.height], margin: 0 });
      doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
    }

    doc.end();
    await new Promise((resolve, reject) => {
      stream.on("finish", resolve);
      stream.on("error", reject);
    });

    console.log("[3/3] png -> pdf");
    console.log(`      страниц: ${pages.length}`);
    console.log(`      выход:   ${path.relative(ROOT, PDF_PATH)}`);
  }

  main().catch((err) => {
    console.error("Ошибка сборки PDF:", err.message);
    process.exit(1);
  });
```

---

ФАЙЛ 83/87: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМЕР: 1.5 КБ

---

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текстовый файл
```

```

    "Конфигурация проекта",
    "Ядро приложения",
    "Контент и данные пособия",
    "Билеты (учебный контент)",
    "Интерактивные компоненты и симуляторы",
    "Визуализаторы (вложенные)",
    "HTML-разметка (layout)",
    "Утилиты",
    "Скрипты сборки и экспорта",
    "CI / автоматизация",
    "Прочее",
];

function listFiles() {
  let files;
  try {
    files = execFileSync("git", ["ls-files", "--cached"], {
      cwd: ROOT,
      encoding: "utf8",
    })
      .split("\n")
      .filter(Boolean);
  } catch {
    files = [];
    const walk = (dir) => {
      for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
        const abs = path.join(dir, e.name);
        const rel = path.relative(ROOT, abs).split(path.sep);
        if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
        if (e.isDirectory()) walk(abs);
        else files.push(rel);
      }
    };
    walk(ROOT);
  }

  return files
    .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d + path.sep)))

```

```
const push = (s = "") => out.push(s);

const stamp = new Date().toLocaleString("ru-RU", { timeZoned: true });
const totalBytes = files.reduce((s, f) => s + fs.statSync(f).size, 0);

push(HR);
push("                УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ДАННЫХ");
push("                ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ И КОММЕНТАРИИ)");
push(HR);
push();
push(`Дата генерации:                ${stamp}`);
push(`Файлов исходного кода:         ${files.length}`);
push(`Суммарный объём кода:           ${humanSize(totalBytes)}`);
push(`Глав/билетов в пособии:         ${chapters.length}`);
push();

push(HR);
push("                                ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
push(HR);
chapters.forEach((c, i) => {
  push(`  ${String(i + 1).padStart(3, " ")}. ${c.title}`);
});
push();

push(HR);
push("                                ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
push(HR);
let current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push();
    push(`  # ${cat}`);
  }
  push(`  ${String(i + 1).padStart(3, " ")}. ${rel}`);
});
push();
```

```
main();
```

ФАЙЛ 84/87: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМ

```
/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt    (collect-code.mjs)
 * txt  -> page-XXXX.png             (render-pages.mjs)
 * png  -> full_code_all_pages.pdf   (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
  /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
  density: Number(process.env.EXPORT_DENSITY ?? 150),
  /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=1 */
  monochrome: process.env.EXPORT_GRAYSCALE !== "1",
  size: "A4",
  font: "DejaVu-Sans-Mono",
};
```

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
  ROOT,
  PAGES_DIR,
  TXT_PATH,
  PAGE,
  ensureDir,
  humanSize,
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
  const expanded = line.replace(/\t/g, " ").replace(/<\/p>
  if (expanded.length <= PAGE.cols) return [expanded];
  const parts = [];
  const indent = " ".repeat(Math.min((expanded.match(/<\/p>
  let rest = expanded;
  let first = true;
  while (rest.length > 0) {
    const width = first ? PAGE.cols : PAGE.cols - indent;
    parts.push((first ? "" : indent) + rest.slice(0, width));
    rest = rest.slice(width);
    first = false;
  }
  return parts;
}
```



```

    "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
    "-pointsize", String(PAGE.pointsSize),
    `text:${job.txtFile}`,
    "-background", "white",
    "-flatten",
    "-colorspace", "Gray",
    ...(PAGE.monochrome ? ["-monochrome"] : []),
    "-strip",
    "-define", "png:compression-level=9",
    job.pngFile,
  ];

```

```

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

```

```

async function worker() {
  while (cursor < jobs.length) {
    const job = jobs[cursor++];
    await execFileAsync(im, args(job));
    fs.rmSync(job.txtFile, { force: true });
    done += 1;
    if (done % 25 === 0 || done === total) {
      process.stdout.write(`      отрендерено ${done}\n`);
    }
  }
}

```

```

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

```

```

const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length) throw new Error(`Не отрендерены следующие файлы: ${missing.map(j => j.pngFile).join(", ")}`);

```

```

const bytes = jobs.reduce((s, j) => s + fs.statSync(j.pngFile).size, 0);
console.log("[2/3] txt -> png");
console.log(`      страниц: ${total}`);
console.log(`      выход:   ${path.relative(ROOT, PAGE.pngFile)}`);

```

```
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
deploy:
```

## Универсальное пособие • полный исходный код

```
-----
#
#                                     └─ full_code_all_page
#
# Готовые TXT и PDF коммитятся обратно в ветку (их отда
# промежуточные PNG сохраняются как artifacts запуска.

on:
  push:
    branches:
      - "**"
    paths-ignore:
      # чтобы коммит самого workflow не запускал бескон
      - "public/export/**"
      - "**/*.md"
  workflow_dispatch:
    inputs:
      density:
        description: "DPI растеризации страниц (по умол
        required: false
        default: "150"

concurrency:
  group: rebuild-pdf-`${github.ref}`
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[
    steps:
      - name: Checkout
        uses: actions/checkout@v6
```

- ```
run: npx tsc --noEmit
continue-on-error: true
```
- name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`public/export/full\_code\_all\_pages`"  
 echo "| \`public/export/full\_code\_all\_pages`"  
 echo "| PNG-страниц | `\${ls tmp/export-pages}`"  
 } >> "\$GITHUB\_STEP\_SUMMARY"
  - name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7  
with:  
 name: full-code-export  
 path: |  
 public/export/full\_code\_all\_pages.pdf  
 public/export/full\_code\_all\_pages.txt  
 if-no-files-found: error  
 retention-days: 30
  - name: Upload artifacts (PNG-страницы)  
uses: actions/upload-artifact@v7  
with:  
 name: full-code-pages-png  
 path: tmp/export-pages/\*.png  
 if-no-files-found: error  
 retention-days: 7
  - name: Commit rebuilt export back to the branch  
run: |  
 git config user.name "github-actions[bot]"  
 git config user.email "41898282+github-action@users.noreply.github.com"  
 git add -- public/export/full\_code\_all\_pages.pdf